Gaussian Mixture Model

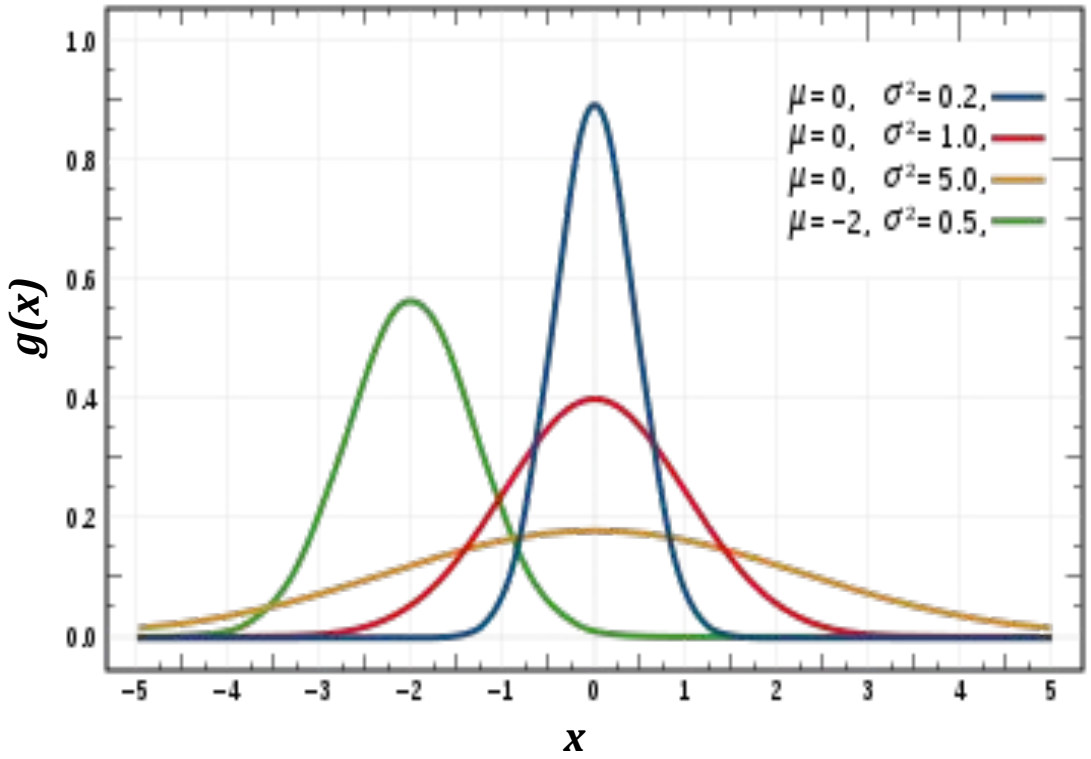


Prof. S. Umesh

DEPARTMENT OF ELECTRICAL ENGINEERING
INDIAN INSTITUTE OF TECHNOLOGY MADRAS



Gaussian Distribution



Plot shows the variation of normal distribution curve for various **μ** and **σ^2**

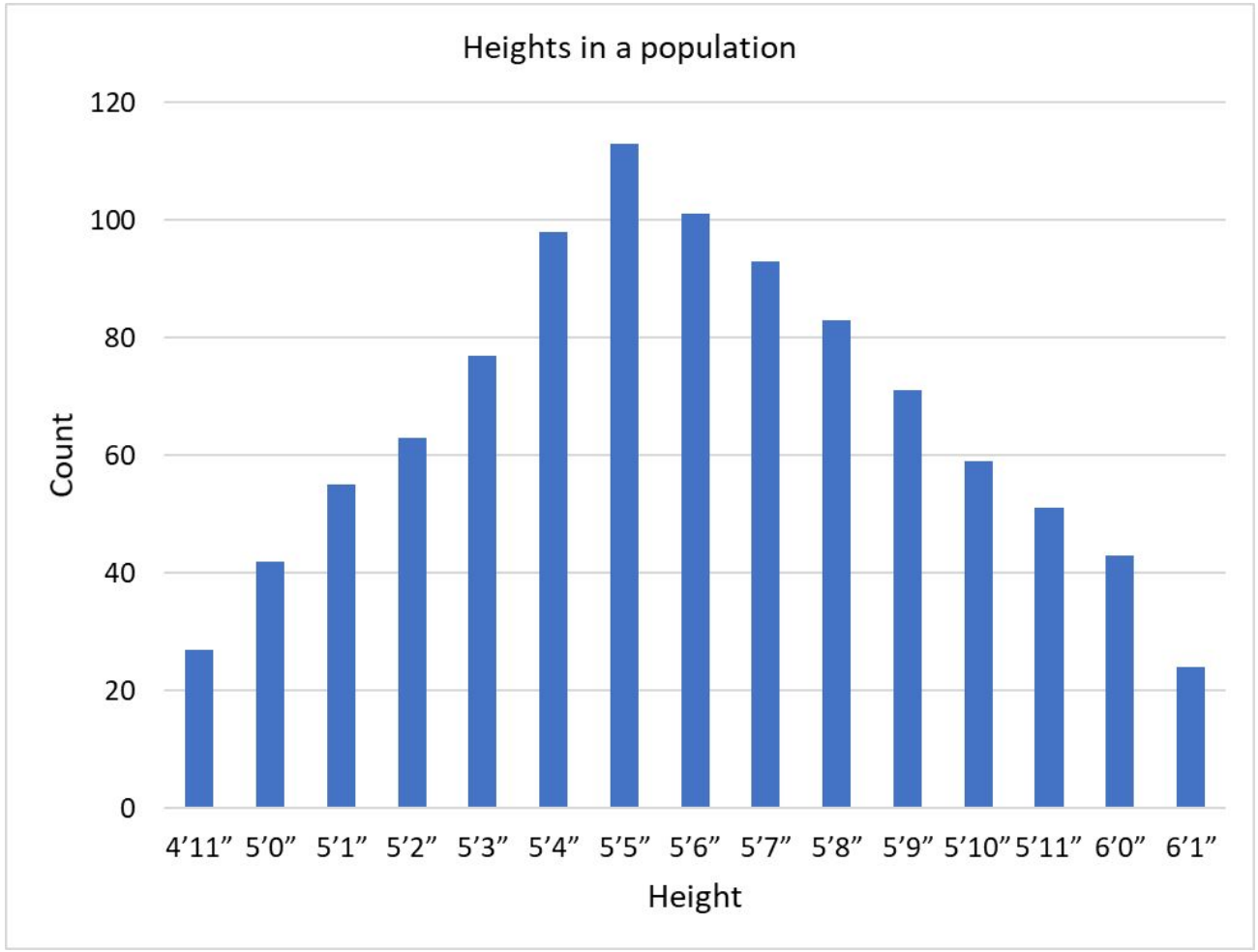
$$f_x(x;\mu,\sigma) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{1}{2}\frac{(x-\mu)^2}{\sigma^2}\right)$$

μ :- Mean

σ :- Standard Deviation

Gaussian Distribution is one of the most important distribution in statistics because it fits many natural phenomena like **age, height, test-scores, IQ scores, sum of the rolls of two dice** and so on.

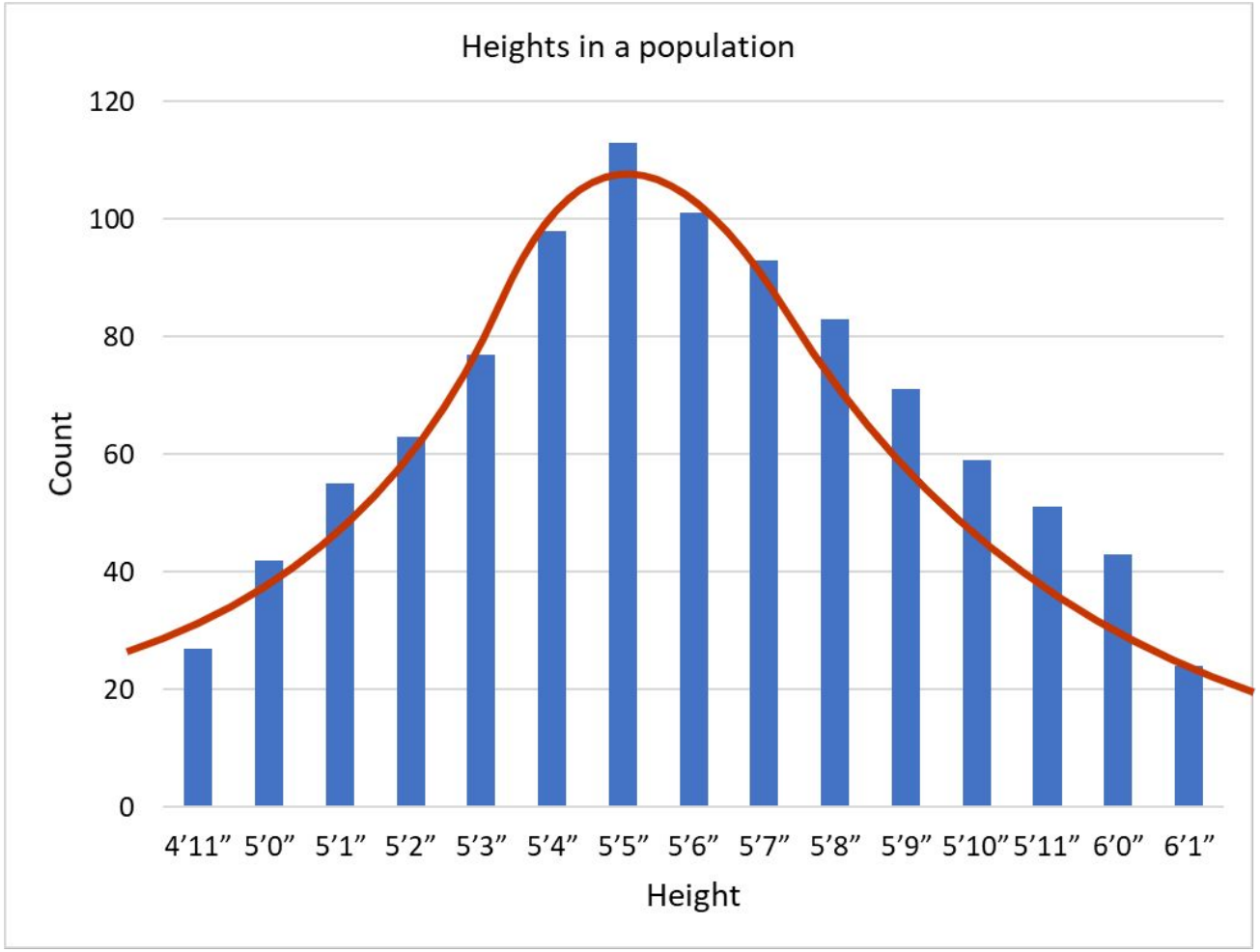
Histogram of an Example Dataset



Heights	No. of People
4'11"	27
5'0"	42
5'1"	55
5'2"	63
5'3"	77
5'4"	98
5'5"	113
5'6"	101
5'7"	93
5'8"	83
5'9"	71
5'10"	59
5'11"	51
6'0"	43
6'1"	24

We can fit Gaussian Distribution on this dataset

Fitting a Gaussian Distribution



Heights	No. of People
4'11"	27
5'0"	42
5'1"	55
5'2"	63
5'3"	77
5'4"	98
5'5"	113
5'6"	101
5'7"	93
5'8"	83
5'9"	71
5'10"	59
5'11"	51
6'0"	43
6'1"	24

Q 1: How do we fit the Gaussian Distribution on our dataset ?

Ans 1: We use our dataset to compute **Mean μ** and **Standard Deviation σ** of the gaussian distribution.

Q 2: How do we compute Mean and Standard Deviation of the gaussian distribution, given the dataset ?

Ans2: We use some **estimation techniques** to compute **Mean μ** and **Standard Deviation σ** for the gaussian distribution.



Estimating μ and σ

Estimation : the process of finding the parameters (**μ and σ**) of the distribution ***f(x)***, which have most "likely" generated the observations.

For the given dataset (**heights dataset**), we have 1000 height measurements.

Using **Maximum Likelihood Estimation (MLE)** we can estimate **μ** and **σ** as:

$$\mu = \frac{1}{1000} \sum_{i=1}^{1000} x_i \qquad \sigma = \sqrt{\frac{1}{1000} \sum_{i=1}^{1000} (x_i - \mu)^2}$$

Maximum Likelihood Estimation (MLE)

Likelihood: For a given set of observations, Likelihood is defined by taking the **joint probability** across the whole set of observations.

$$L(\mu, \sigma) = \mathrm{f_x}\left(\mathrm{x}_1, \ldots\ldots, x_{1000}; \mu, \sigma\right)$$

MLE: It's a technique to estimate parameters (**μ and σ**) of the assumed distribution ***f(x)***, given the data by **maximizing the likelihood function**.

$$\mu^*, \sigma^* = \underset{\mu,\sigma}{\operatorname{argmax}}(L(\mu, \sigma))$$



Using MLE to estimate μ and σ for Height Dataset

As each of the observations are independent, we can say :-

$$f_x(x_1, \ldots\ldots, x_{1000}; \mu, \sigma) = \prod_{i=1}^{1000} f_x(x_i; \mu, \sigma)$$

$$\mu^*, \sigma^* = \underset{\mu,\sigma}{\operatorname{argmax}} \prod_{i=1}^{1000} f_x(x_i; \mu, \sigma)$$

To find argmax we have to take partial derivative w.r.t **μ** and **σ**. But usually **it's easier to take derivative of sum of functions instead of product of functions**. So to achieve this we can use ***log*** function because it's **monotonic**.



$$\mu^*, \sigma^* = \underset{\mu,\sigma}{\operatorname{argmax}} \sum_{i=1}^{1000} \log\left(f_x\left(x_i; \mu, \sigma\right)\right)$$ ← *log-likelihood*

$$G(\mu, \sigma) = \sum_{i=1}^{1000} \log\left(f_x\left(x_i; \mu, \sigma\right)\right) \quad \text{where} \quad f_x(x_i; \mu, \sigma) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{1}{2}\frac{(x_i - \mu)^2}{\sigma^2}\right)$$

To find **μ*** and **σ*** we will take **derivative of G(μ,σ) w.r.t to μ and σ and solve for μ and σ respectively**

$$\frac{\partial G(\mu, \sigma)}{\partial \mu} = 0, \frac{\partial G(\mu, \sigma)}{\partial \sigma} = 0$$



Using Gaussian Model for Classification Problem

height	gender
5’4”	Male
5’9”	Female
…	…
6’0”	Female

Given a new dataset (**D**) which contain heights and genders of 1000 people.

height	gender
5’4”	Male
5’8”	Male
…	…
6’1”	Male

D_M

height	gender
5’9”	Female
5’7”	Female
…	…
6’0”	Female

D_F

We divide **dataset D into two subset D_M and D_F based on gender** (which means D_M contain all the height observations for which gender is male and D_F contain all the height observations for which gender is female) and $D_M \cup D_F = D$

Using MLE to estimate μ and σ for Height Dataset

As each of the observations are independent, we can say :-

$$f_x(x_1, \ldots\ldots, x_{1000}; \mu, \sigma) = \prod_{i=1}^{1000} f_x(x_i; \mu, \sigma)$$

$$\mu^*, \sigma^* = \underset{\mu,\sigma}{\operatorname{argmax}} \prod_{i=1}^{1000} f_x(x_i; \mu, \sigma)$$

To find argmax we have to take partial derivative w.r.t **μ** and **σ**. But usually **it's easier to take derivative of sum of functions instead of product of functions**. So to achieve this we can use ***log*** function because it's **monotonic**.

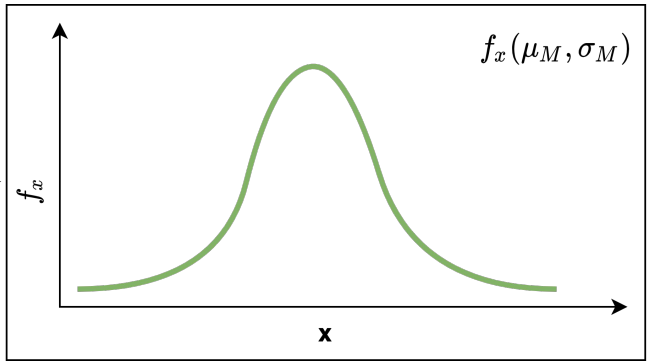


Using Gaussian Model for Classification Problem

height	gender
5'4"	Male
5'9"	Female
…	…
6'0"	Female

height	gender
5'4"	Male
5'8"	Male
…	…
6'1"	Male

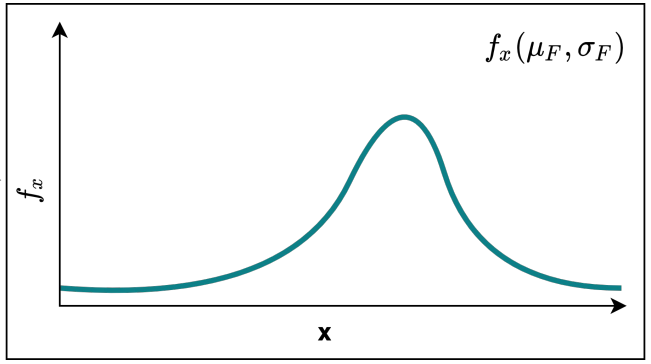
D_M



Gaussian Model for Male

height	gender
5'9"	Female
5'7"	Female
…	…
6'0"	Female

D_F

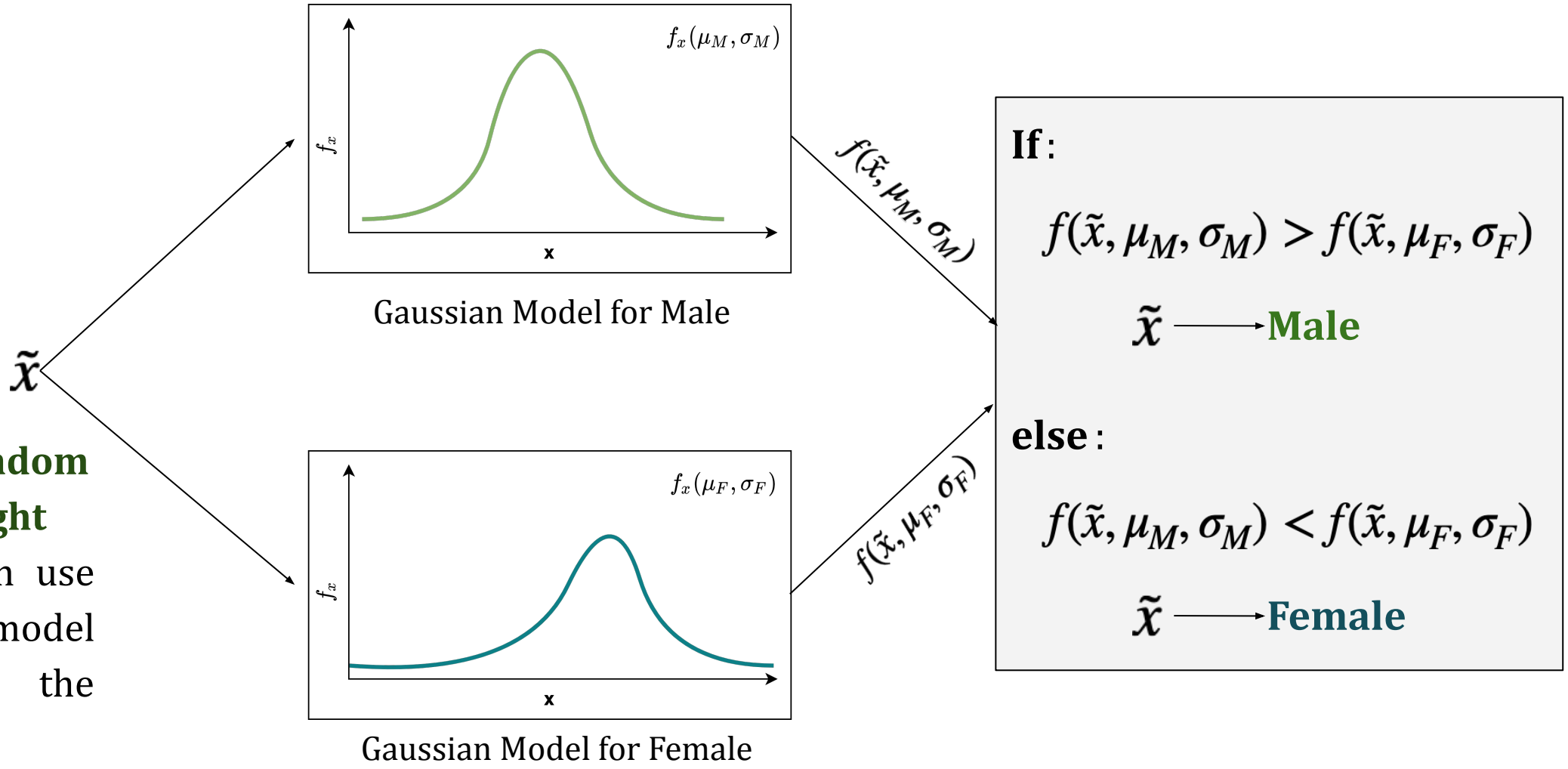


Gaussian Model for Female

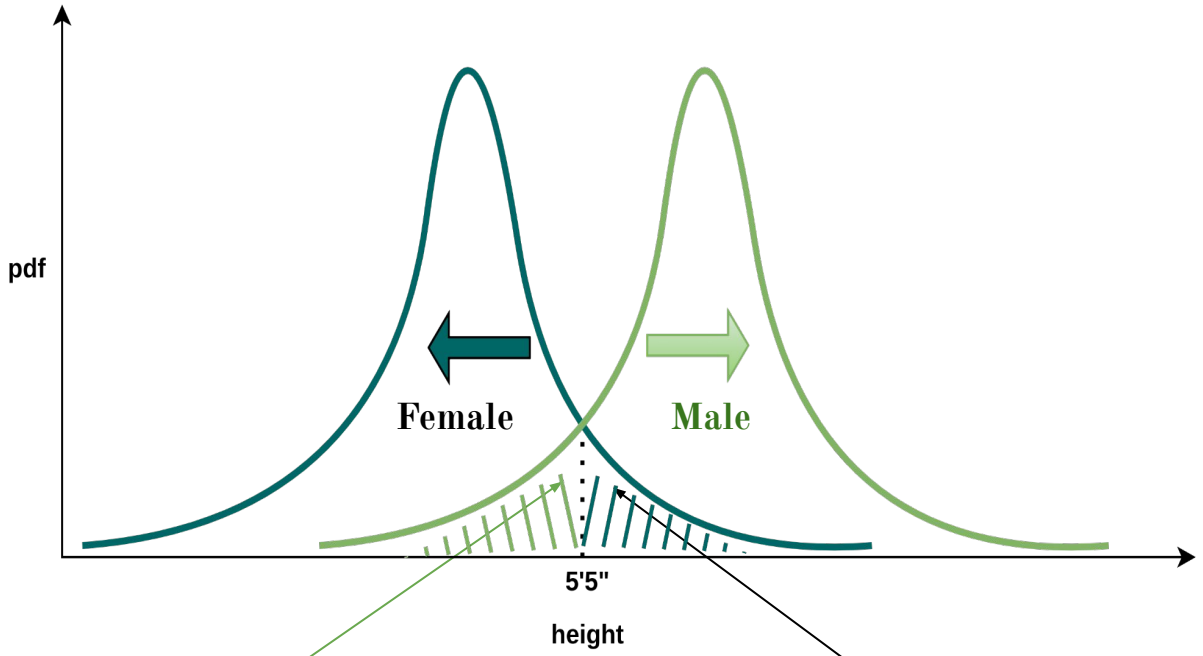
We use **D_M to estimate μ_M and σ_M for Male Gaussian model** and we use **D_F to estimate μ_F and σ_F for Female Gaussian model** using Maximum Likelihood Estimation (**MLE**)

Using Gaussian Model for Classification Problem

Given **a random person height** ($\tilde{x}$) we can use gaussian model to predict the gender.



Error Analysis



$$\int_{-\infty}^{5'5''} f_x\left(\mu_M, 1.2\right)$$

Prob of classifying males as female

$$\int_{5'5''}^{\infty} f_x\left(\mu_F, 1 \cdot 2\right)$$

Prob of classifying females as males

Error Analysis

- When females are classified as males and males are classified as females, these are called misclassifications.
- For a given test dataset and a given model, the misclassification rate is defined as

$$\text{misclassification rate} = \frac{\text{no. of misclassifications}}{\text{total no. of predictions}}$$

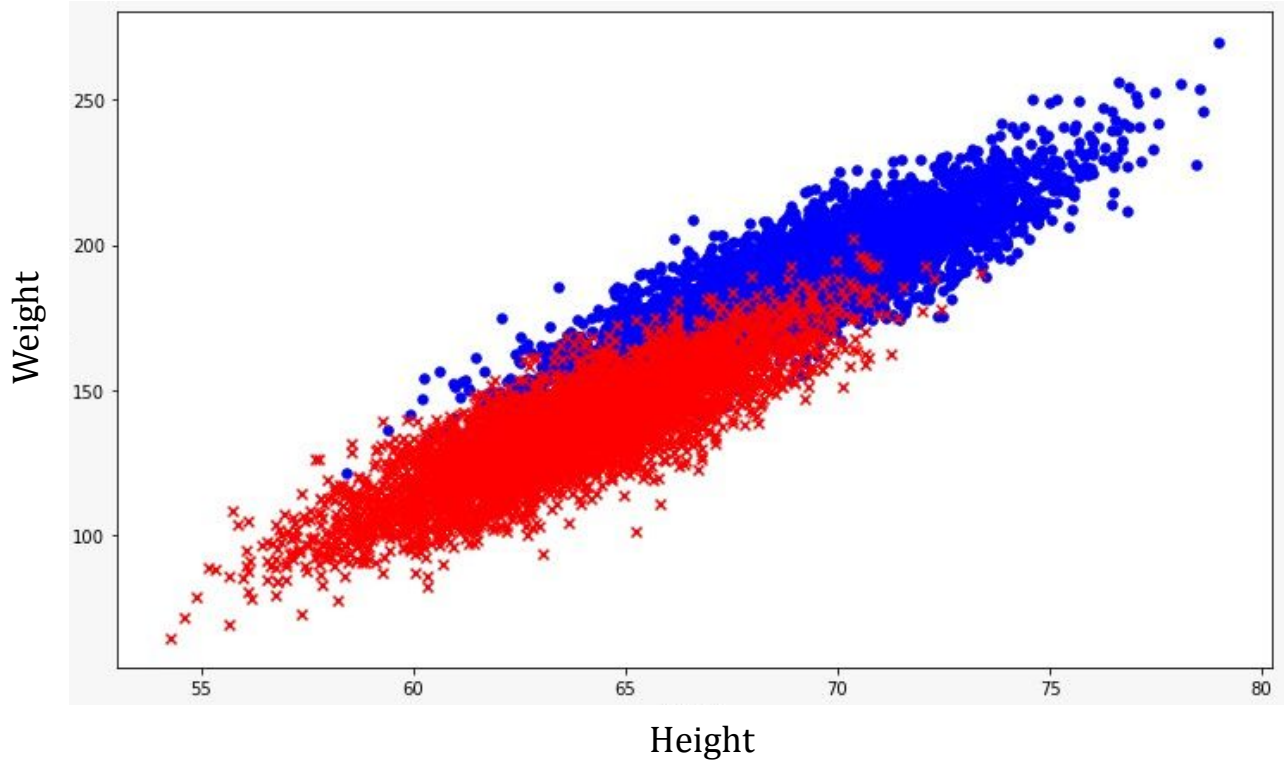
- For example, if the test data for heights dataset contains heights of 100 persons and 10 females are classified as males, 15 males are classified as females and the remaining 75 persons are correctly classified, the misclassification rate will be 0.25



Multivariate Gaussian Model



Example of the Bivariate Gaussian Model



A scatter plot of height and weight measurements of 10000 men and women is shown here.

Men are shown by blue dots and women are shown by red dots.

Here the height and weight are correlated features.

- taller ↔ heavier
- shorter ↔ lighter

Multivariate Gaussian Model

A generalization of the one-dimensional (univariate) normal distribution to higher dimensions (n).

$$p(\mathbf{x};\mu,\mathbf{\Sigma}) = \frac{1}{(2\pi)^{\frac{n}{2}}|\mathbf{\Sigma}|^{\frac{1}{2}}}\exp\left(-\frac{1}{2}(\mathbf{x}-\mu)^T\mathbf{\Sigma}^{-1}(\mathbf{x}-\mu)\right)$$

- Here $\mu \in \mathrm{R}^{\mathrm{n}}$ is the mean and $\mathbf{\Sigma} \in \mathrm{S}^{\mathrm{n}}_{++}$ is covariance matrix.

$\mathrm{S}^{\mathrm{n}}_{++}$: nxn space of symmetric positive definite matrices

Multivariate Gaussian Model

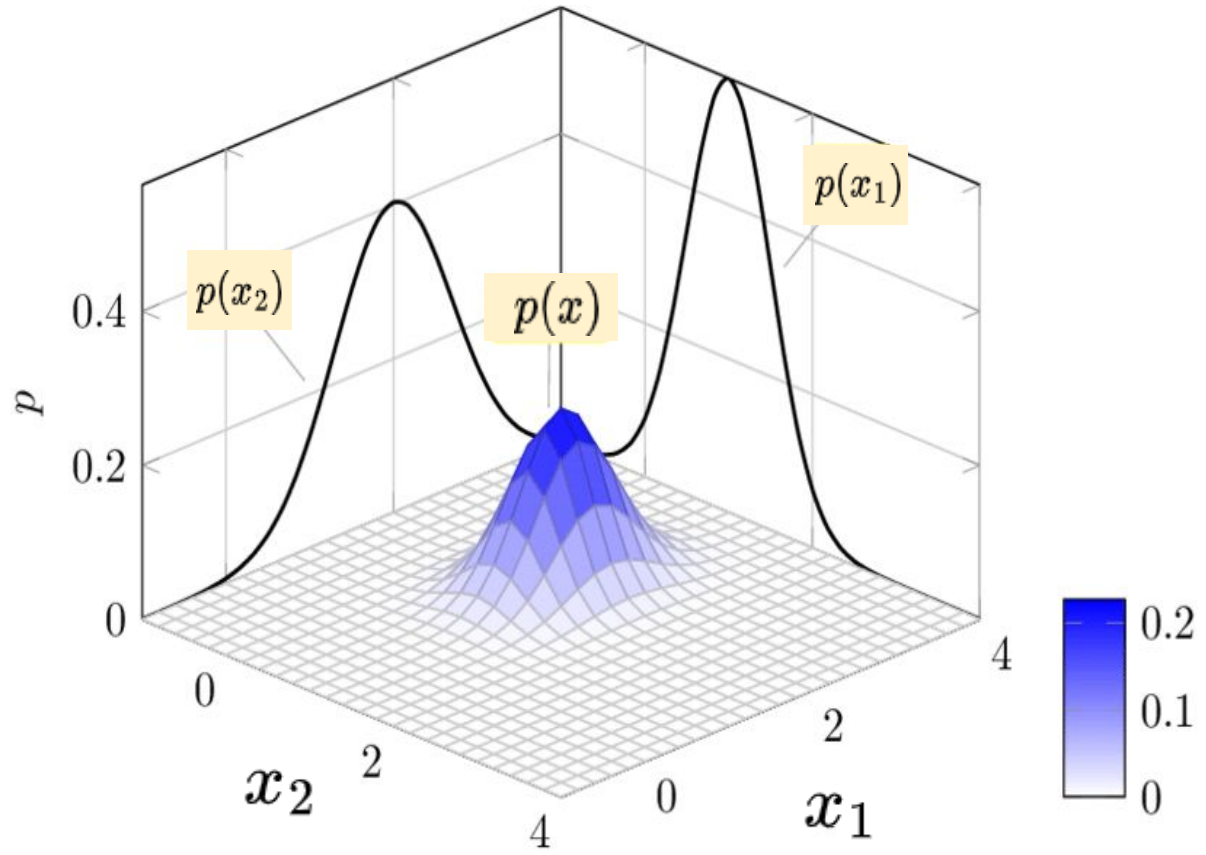
For Dimension n= 2,

$$x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

With Gaussian Densities

$$x_1 \rightarrow \mathcal{N}(\mu_1, \sigma_1)$$

$$x_2 \rightarrow \mathcal{N}(\mu_2, \sigma_2)$$



Bivariate Gaussian Plot

Bivariate Gaussian Model

Where, mean and variance of bivariate gaussian model are,

$$\mu = \begin{bmatrix} \mu_1 \\ \mu_2 \end{bmatrix}, \Sigma = \begin{bmatrix} \sigma_{11} & \sigma_{12} \\ \sigma_{21} & \sigma_{22} \end{bmatrix}$$

μ_1= mean(x_1)
μ_2= mean(x_2)
σ_{12} = Cov(x_1,x_2),
σ_{11} = Var(x_1), σ_{22} = Var(x_2)

Here the data is 2 dimensional so,

- The density function that models this data will be three dimensional
- μ will be a **2D vector**
- Σ will be a **2x2 matrix**



Bivariate Gaussian Model

Covariance of x_1 and x_2 is :

$$Cov[X_1, X_2] = E[(X_1 - E[X_1])(X_2 - E[X_2])] = E[X_1 X_2] - E[X_1]E[X_2]$$

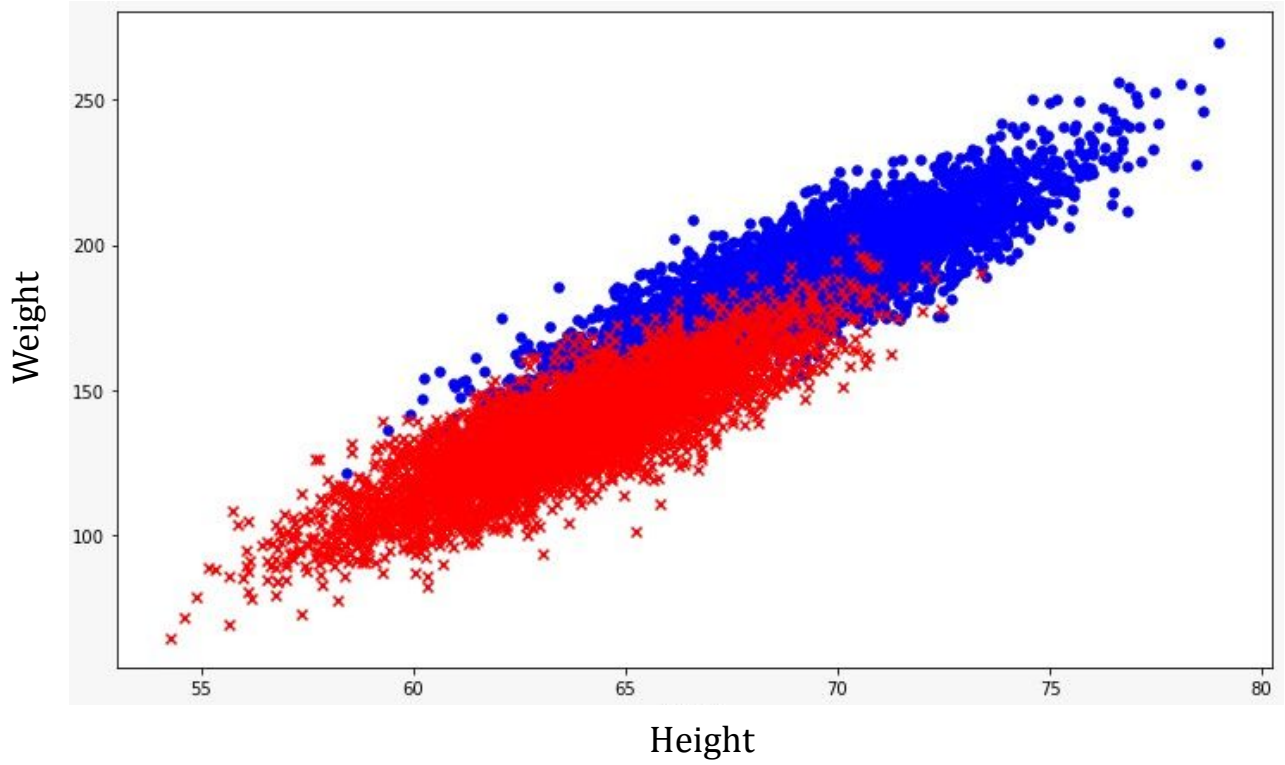
- A square matrix giving the covariance between each pair of elements of a given random vector
- Generalization of variances in multi-dimension.
- For the case of n=2,

$$\mathbf{\Sigma} = \begin{bmatrix} cov[X_1, X_1] & cov[X_1, X_2] \\ cov[X_1, X_2] & cov[X_2, X_2] \end{bmatrix}$$



If X_1 and X_2 are uncorrelated, $cov[X_1,X_2] = 0$ and $\mathbf{\Sigma}$ is a diagonal matrix.

Example of the Bivariate Gaussian Model



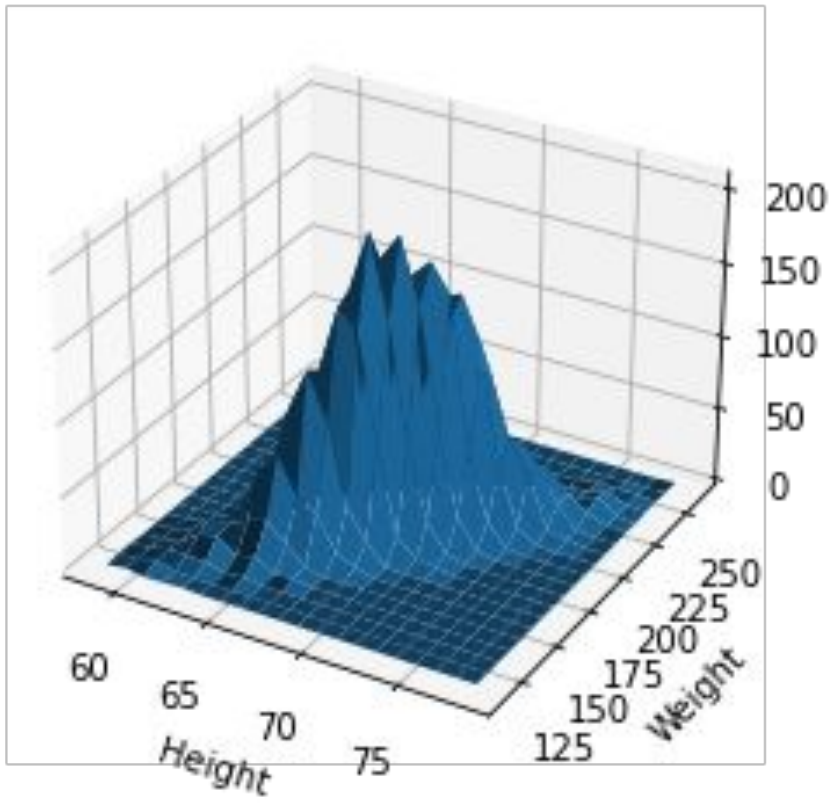
A scatter plot of height and weight measurements of 10000 men and women is shown here.

Men are shown by blue dots and women are shown by red dots.

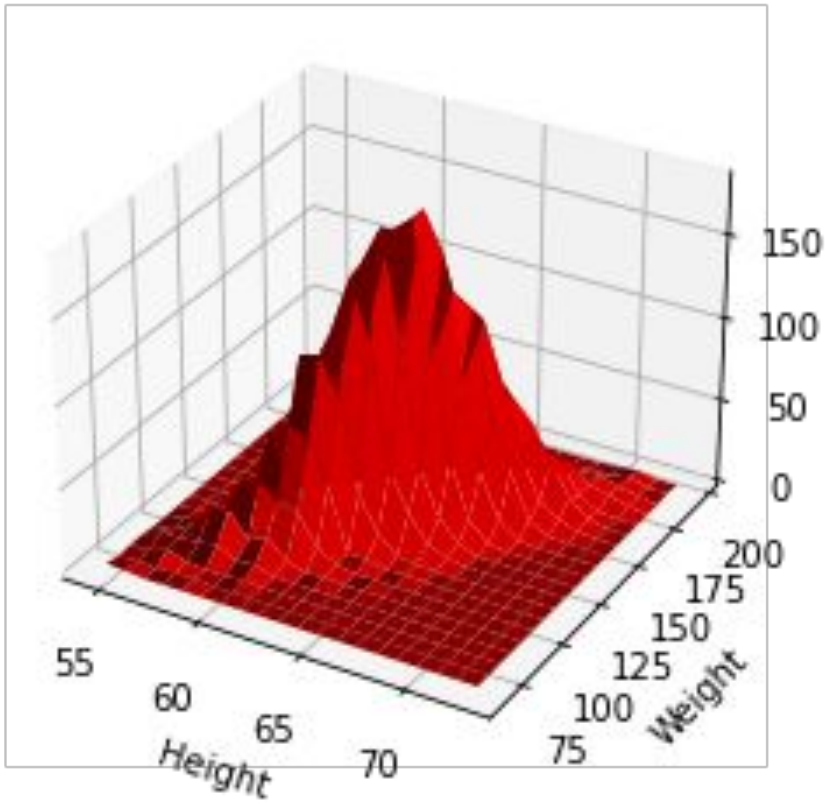
Here the height and weight are correlated features.

- taller ↔ heavier
- shorter ↔ lighter

Example of the Bivariate Gaussian Model



2D histogram of height and weight distribution for male

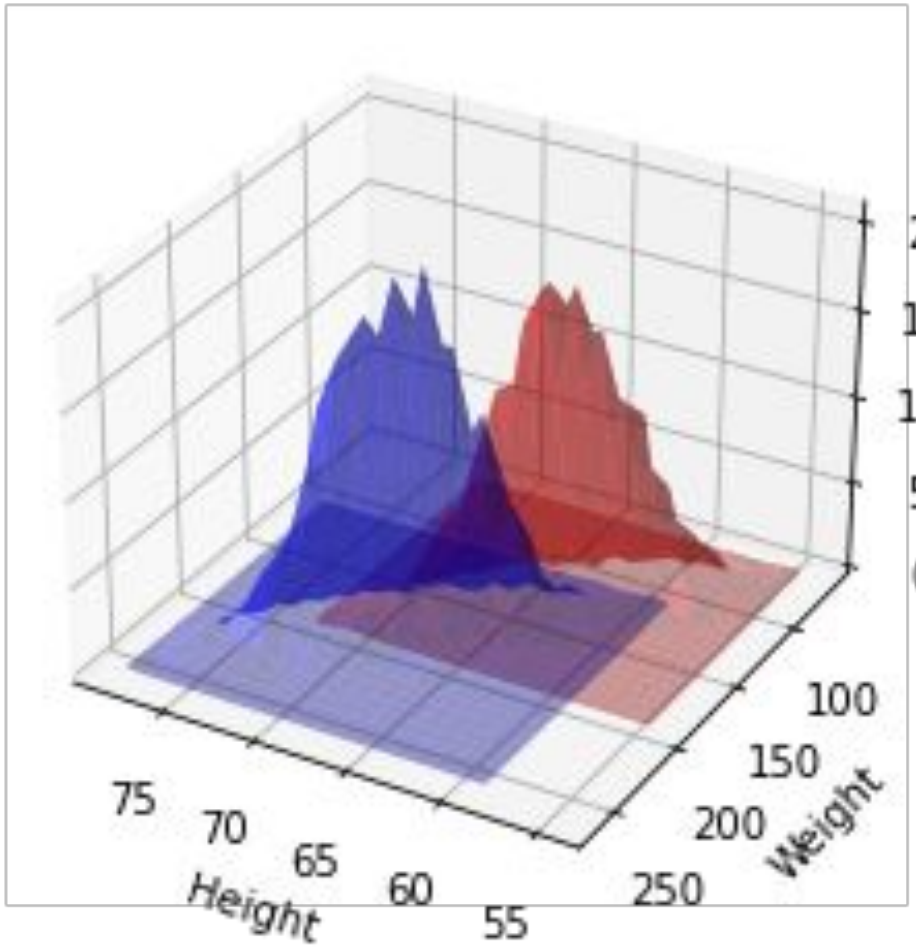


2D histogram of height and weight distribution for female

Using MLE, estimate Gaussian distribution, (μ_m, Σ_m) and (μ_f, Σ_f) for male and female height -weight distribution data, respectively

Using multivariate Gaussian model for classification

$\tilde{x}$ →



$$\frac{p(\tilde{\mathbf{x}}; \mu_\mathbf{m}, \Sigma_\mathbf{m})}{p(\tilde{\mathbf{x}}; \mu_\mathbf{f}, \Sigma_\mathbf{f})}$$

If:

$$p(\tilde{\mathbf{x}}; \mu_\mathbf{m}, \Sigma_\mathbf{m}) > p(\tilde{\mathbf{x}}; \mu_\mathbf{f}, \Sigma_\mathbf{f})$$

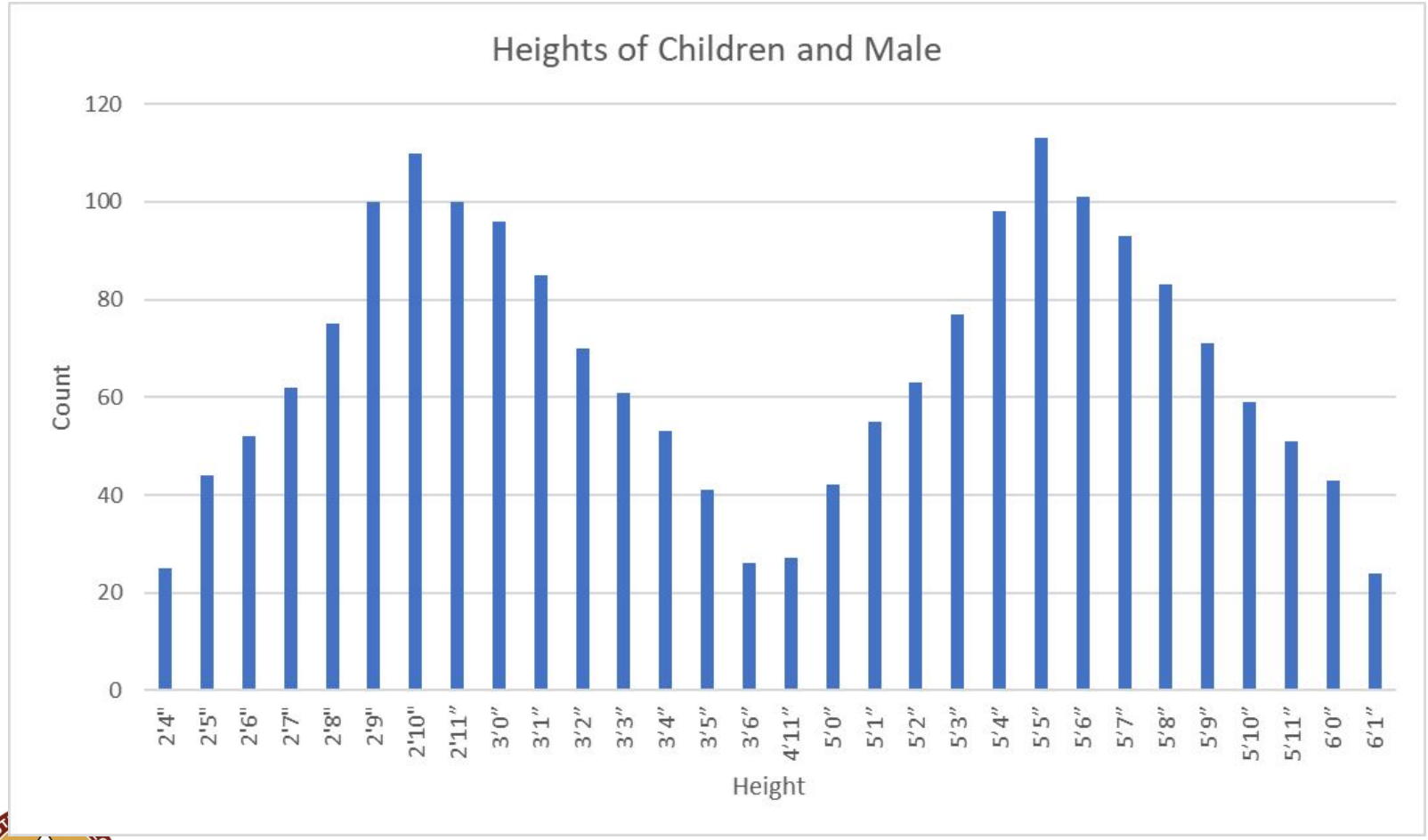
$\tilde{x}$ → **Male**

else:

$$p(\tilde{\mathbf{x}}; \mu_\mathbf{f}, \Sigma_\mathbf{f}) > p(\tilde{\mathbf{x}}; \mu_\mathbf{m}, \Sigma_\mathbf{m})$$

$\tilde{x}$ → **Female**

Gaussian Mixture Model (GMM)

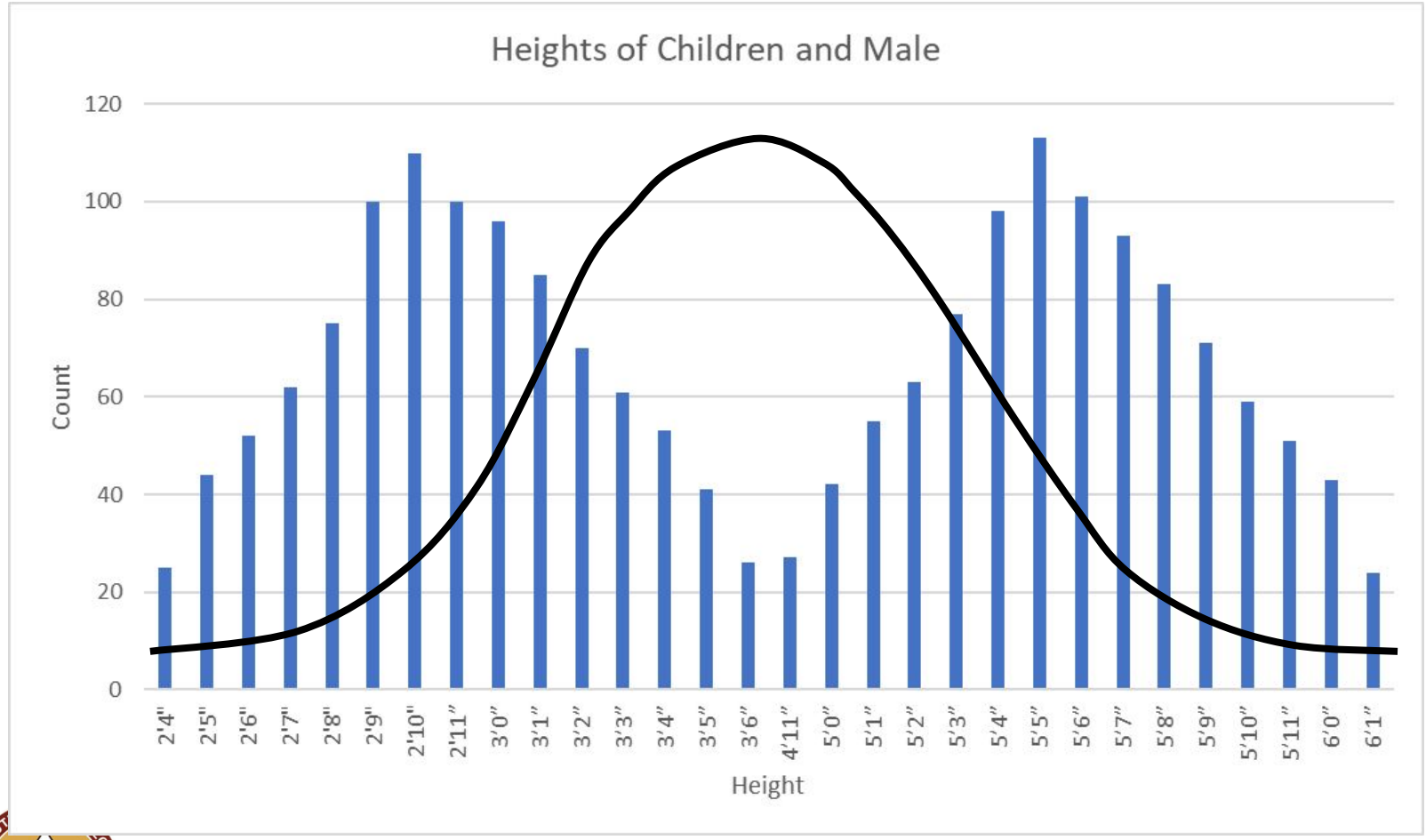


Heights	No. of People
2'4"	25
2'5"	44
2'6"	52
2'7"	62
2'8"	75
2'9"	100
2'10"	110
2'11"	100
3'0"	96
3'1"	85
3'2"	70
3'3"	61
3'4"	53
3'5"	41
3'6"	26

Heights	No. of People
4'11"	27
5'0"	42
5'1"	55
5'2"	63
5'3"	77
5'4"	98
5'5"	113
5'6"	101
5'7"	93
5'8"	83
5'9"	71
5'10"	59
5'11"	51
6'0"	43
6'1"	24

Let's try to fit a Gaussian Distribution on this dataset

Gaussian Mixture Model (GMM)

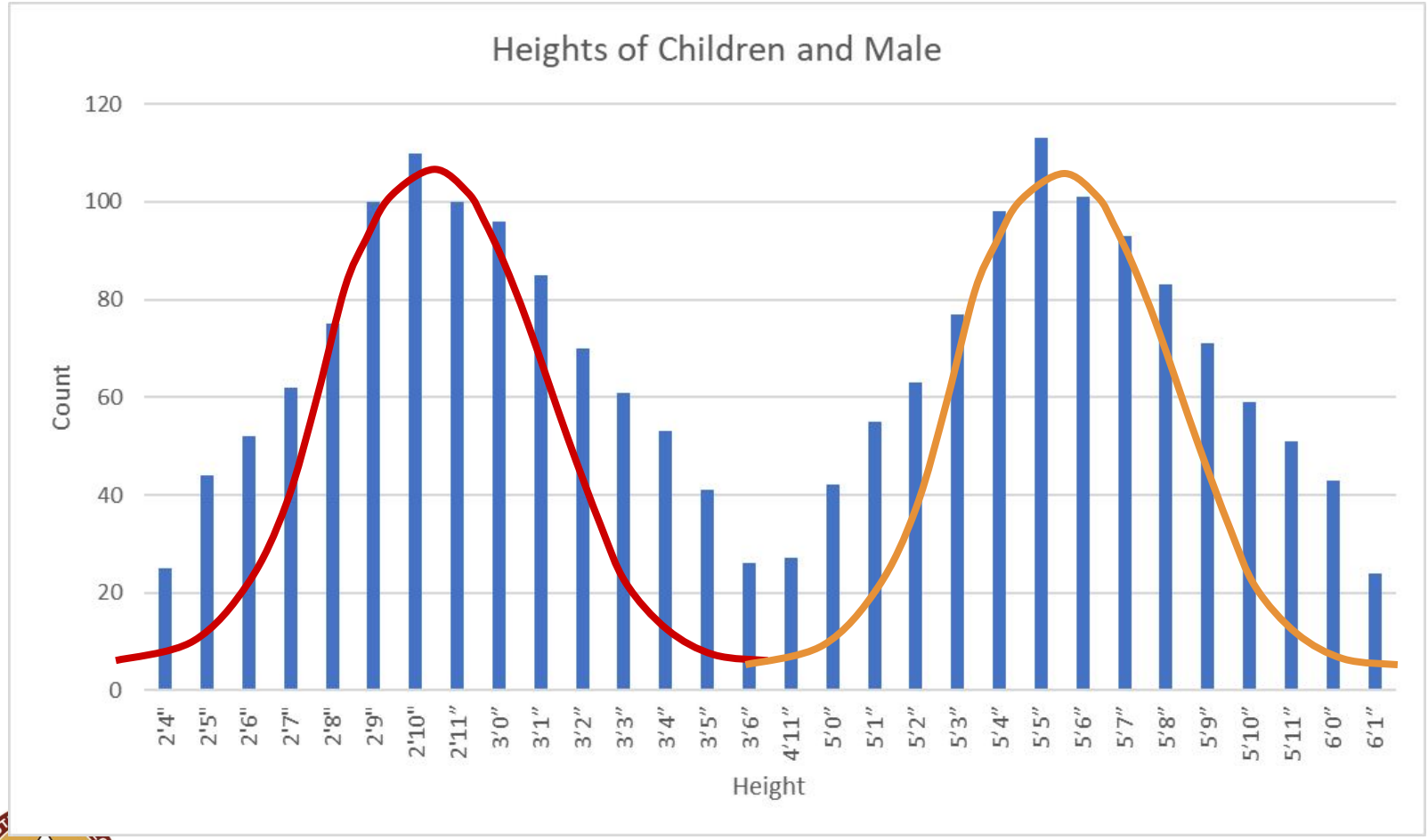


Heights	No. of People
2'4"	25
2'5"	44
2'6"	52
2'7"	62
2'8"	75
2'9"	100
2'10"	110
2'11"	100
3'0"	96
3'1"	85
3'2"	70
3'3"	61
3'4"	53
3'5"	41
3'6"	26

Heights	No. of People
4'11"	27
5'0"	42
5'1"	55
5'2"	63
5'3"	77
5'4"	98
5'5"	113
5'6"	101
5'7"	93
5'8"	83
5'9"	71
5'10"	59
5'11"	51
6'0"	43
6'1"	24

But, this is not good. The fitted Gaussian curve is not modelling the data well

Gaussian Mixture Model (GMM)



Heights	No. of People
2'4"	25
2'5"	44
2'6"	52
2'7"	62
2'8"	75
2'9"	100
2'10"	110
2'11"	100
3'0"	96
3'1"	85
3'2"	70
3'3"	61
3'4"	53
3'5"	41
3'6"	26

Heights	No. of People
4'11"	27
5'0"	42
5'1"	55
5'2"	63
5'3"	77
5'4"	98
5'5"	113
5'6"	101
5'7"	93
5'8"	83
5'9"	71
5'10"	59
5'11"	51
6'0"	43
6'1"	24

Here, we used 2 Gaussian curves to fit the data and this models the data better than the previous one. This is called mixture of Gaussians or GMM.

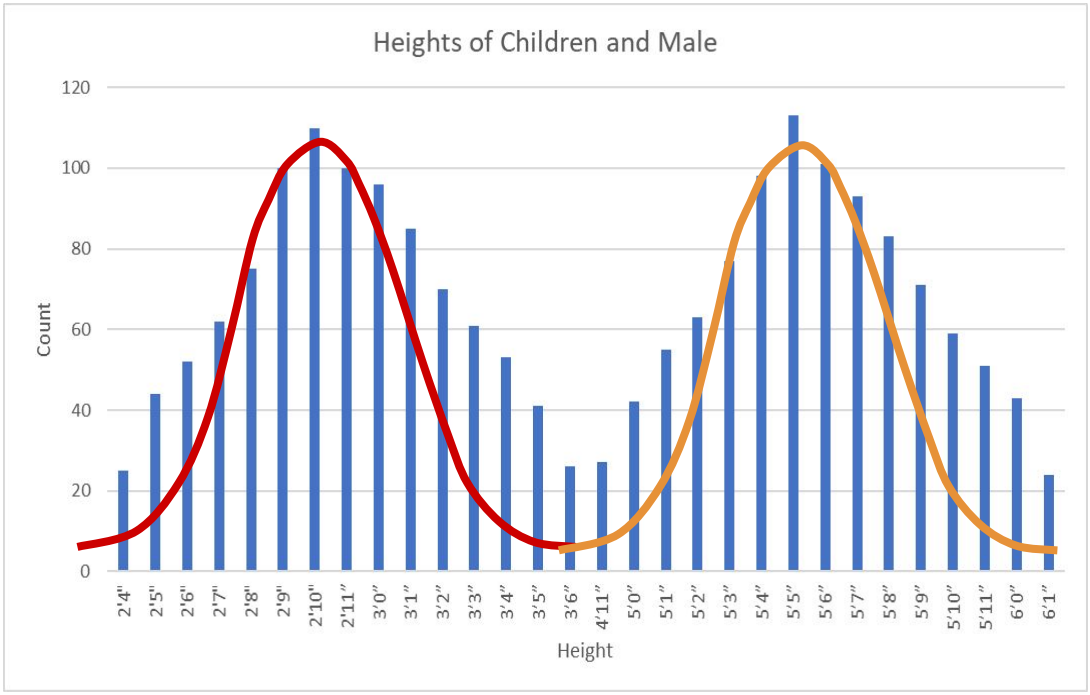
Gaussian Mixture Model

- For the given data,

$$f_X(x) = \pi_1 \mathcal{N}\left(x, \mu_c, \sigma^2\right) + \pi_2 \mathcal{N}\left(x, \mu_m, \sigma^2\right)$$

is better than

$$f_X(x) = \mathcal{N}\left(x, \mu, \sigma^2\right)$$



- Given the histogram representing the data, we would like to fit a mixture of Gaussians which is a parametric form.
- Data - known, **parameters - unknown**
- This problem is called **likelihood problem**

Gaussian Mixture Model

- We can fit **any number of Gaussians** depending on the data.
- Each Gaussian curve is called a **component of the mixture**.
- Each component is associated with a weight(π_i) and **the weights sum up to 1**.
- For a K component GMM, the p.d.f is given by

$$f_X(x) = \sum_{k=1}^{K} \pi_k \mathcal{N}(x \mid \mu_k, \Sigma_k)$$

$$\text{such that } \Sigma_{i=1}^{K} \pi_i = 1$$

- The component weights help in modelling the data where the mixture **components are not equally likely**. For example, number of children is less than the number of males(or adults).
  In practice, the number of components in the Gaussian mixture is unknown and it is a **hyperparameter**. It is found **empirically** or through **domain knowledge**.

Gaussian Mixture Model

- Estimating the parameters of a **GMM is an unsupervised learning** problem
- Every x could have been generated by any component of the mixture
- But in practice, we do not have access to the component information (i.e) we **do not have labels**
- First, let's learn to fit a GMM when we have the labels
- Assume the following data

Observation	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	x_{10}	x_{11}	x_{12}
Label	C	M	C	F	F	M	F	C	M	F	F	C



Gaussian Mixture Model

Let's assume the following:

Children(C) - Class 1, Female(F) - Class 2, Male(M) - Class 3

Since there are three classes, we will take **K=3**

$$f_X(x) = \pi_1 \mathcal{N}\left(x, \mu_1, \sigma_1^2\right) + \pi_2 \mathcal{N}\left(x, \mu_2, \sigma_2^2\right) + \pi_3 \mathcal{N}\left(x, \mu_3, \sigma_3^2\right)$$

Unknowns: $\mu_1, \mu_2, \mu_3, \sigma_1^2, \sigma_2^2, \sigma_3^2, \pi_1, \pi_2, \pi_3$

So, we can easily estimate the mean and variance of each component using **MLE**!



Gaussian Mixture Model

Observation	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	x_{10}	x_{11}	x_{12}
Label	C	M	C	F	F	M	F	C	M	F	F	C

$$\hat{\pi}_1 = \frac{\text{number of children}}{\text{total measurements}} = \frac{4}{12}$$

$$\hat{\pi}_2 = \frac{\text{number of females}}{\text{total measurements}} = \frac{5}{12}$$

$$\hat{\pi}_3 = \frac{\text{number of males}}{\text{total measurements}} = \frac{3}{12}$$



Gaussian Mixture Model

Observation	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	x_{10}	x_{11}	x_{12}
Label	C	M	C	F	F	M	F	C	M	F	F	C

$$\hat{\mu_1} = \frac{x_1 + x_3 + x_8 + x_{12}}{4}$$

$$\hat{\mu_2} = \frac{x_4 + x_5 + x_7 + x_{10} + x_{11}}{5}$$

$$\hat{\mu_3} = \frac{x_2 + x_6 + x_9}{3}$$



Similarly, we can easily compute $\hat{\sigma_1}, \hat{\sigma_2}, \hat{\sigma_3}$

Gaussian Mixture Model

- But, the labels(z_i) are actually **hidden** information or **latent** variables.
- If we know the underlying GMM component information, this is an easy problem.
- In practice, we only have the measurements and **no category information.**
- **How do we go about it? How do we find which observation belongs to which density?**
- Make initial guess of parameters: $\theta^g = \pi_1^g, \pi_2^g, \pi_3^g, \mu_1^g, \mu_2^g, \mu_3^g, \sigma_1^g, \sigma_2^g, \sigma_3^g$
- Given an observation x_i and parameter θ^g, the probability of x_i coming from j^{th} component is

$$p(z_i = j | x_i; \theta^g) = \frac{p(z_i = j, x_i; \theta^g)}{p(x_i; \theta^g)}$$

Observation	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	x_{10}	x_{11}	x_{12}
Hidden/Latent Variable	z_1	z_2	z_3	z_4	z_5	z_6	z_7	z_8	z_9	z_{10}	z_{11}	z_{12}

Gaussian Mixture Model

$$= \frac{p(x_i|z_i = j; \theta^g) \cdot p(z_i = j)}{\sum_{k=1}^{3} p(x_i|z_i = k; \theta^g) \cdot p(z_i = k)}$$

- All the probabilities in the above equation can be easily computed since we have fixed the parameters
- Then we can find the label/hidden variable as

$$z_i = \underset{j}{\operatorname{argmax}}\, p(z_i = j|x_i; \theta)$$

- This is called **hard allocation**
- Once we have the labels, we know how to estimate the parameters.



Gaussian Mixture Model

- Instead of hard allocation, we can also use **soft assignment**.
- (i.e) we will not assign a component label to a point, instead it will belong to all the components with some probability
- Average probability that a sample belongs to component j is given by

$$\hat{\pi_j}^{new} = \frac{1}{N}\Sigma_{i=1}^{N} p(z_i = j|x_i; \theta^g)$$

- Similarly means and variances can be computed by taking **weighted average** as follows

$$\hat{\mu_j}^{new} = \frac{\Sigma_{i=1}^{N} x_i \cdot p(z_i = j|x_i; \theta^g)}{\Sigma_{i=1}^{N} p(z_i = j|x_i; \theta^g)}$$

$$(\hat{\sigma_j^2})^{new} = \frac{\Sigma_{i=1}^{N} (x_i - \hat{\mu_j})^2 \cdot p(z_i = j|x_i; \theta^g)}{\Sigma_{i=1}^{N} p(z_i = j|x_i; \theta^g)}$$

Expectation-Maximization

It is an elegant method for finding maximum likelihood solutions for models with latent variables.

Here, were looking to model a GMM having *K* components.

A GMM represents a distribution as: $$p(\mathbf{x}) = \sum_{k=1}^{K} \pi_k \mathcal{N}(\mathbf{x}|\mu_k, \sigma_k)$$

with π_k representing the mixing coefficients, such that:

$$\sum_{k=1}^{K} \pi_k = 1 \qquad \text{and} \qquad \pi_k \geq 0, \forall k$$

Through the EM algorithm, we're looking to efficiently estimate the parameters of the GMM:

$$\Theta = \{\pi_k, \mu_k, \sigma_k\}, k \in \{1, 2, \ldots, K\}$$



The Latent Variables

What we have: A set of data points $\{x^{(1)}, x^{(2)}, \ldots, x^{(N)}\}$

What we wish to do: Model this data using a GMM with K component mixtures.

How do we do that: Using the EM algorithm to estimate the parameters.

What we don't know: Which component Gaussian is a given data point sampled from.

Variables that are always unobserved are called latent variables, or hidden variables.

In a mixture model, the identity of the component that generated a given data point is a latent variable.



Our Latent Variable

In the case of the GMM, we had a latent variable z which would represent the Gaussian responsible for generating our observation $\boldsymbol{x}$, with some probability.

Let $z \sim Categorical(\pi),\ (\text{where } \pi_k \geq 0, \sum_{k}^{K} \pi_k = 1)$

Then,

$$p(\mathbf{x}) = \sum_{k=1}^{K} p(\mathbf{x}, z = k)$$

$$= \sum_{k=1}^{K} \underbrace{p(z = k)}_{\pi_k} \underbrace{p(\mathbf{x}|z = k)}_{\mathcal{N}(\mathbf{x}|\mu_k, \Sigma_k)}$$



Reformulating the problem

Gaussian mixture distribution: $p(\mathbf{x}) = \sum_{k=1}^{K} \pi_k \mathcal{N}(\mathbf{x}|\mu_k, \Sigma_k)$

Where, $z \sim \text{Categorical}(\boldsymbol{\pi})$ (where $\pi_k \geq 0, \quad \sum_k \pi_k = 1$)

The joint distribution that is used in further in maximum likelihood: $p(\mathbf{x}, \mathbf{z}) = p(\mathbf{z})p(\mathbf{x}|\mathbf{z})$

We replace the probabilities with the logarithm of the probabilities in the likelihood equation. This is done for mathematical simplicity (optimizing a function with summation inside the logarithm is hard). Also, it is justified to apply the logarithm function as it is monotonically increasing.

Results in a log-likelihood: $\ell(\boldsymbol{\pi}, \mu, \Sigma) = \ln p(\mathbf{X}|\pi, \mu, \Sigma) = \sum_{n=1}^{N} \ln p(\mathbf{x}^{(n)}|\pi, \mu, \Sigma)$

$$= \sum_{n=1}^{N} \ln \sum_{z^{(n)}=1}^{K} p(\mathbf{x}^{(n)}|\, z^{(n)}; \mu, \Sigma) p(z^{(n)}|\, \boldsymbol{\pi})$$

The simpler estimation problem

If we knew $z^{(n)}$ for every $x^{(n)}$ the MLE problem is easy. The log-likelihood would simplify to:

$$\ell(\boldsymbol{\pi}, \mu, \Sigma) = \sum_{n=1}^{N} \ln p(x^{(n)}, z^{(n)} | \pi, \mu, \Sigma) = \sum_{n=1}^{N} \ln p(\mathbf{x}^{(n)} | \, z^{(n)}; \mu, \Sigma) + \ln p(z^{(n)} | \, \boldsymbol{\pi})$$

Maximizing the above function (taking the first derivative to zero w.r.t each of the parameters) gives:

$$\mu_k = \frac{\sum_{n=1}^{N} 1_{[z^{(n)}=k]} \, \mathbf{x}^{(n)}}{\sum_{n=1}^{N} 1_{[z^{(n)}=k]}}$$

$$\Sigma_k = \frac{\sum_{n=1}^{N} 1_{[z^{(n)}=k]} \, (\mathbf{x}^{(n)} - \mu_k)(\mathbf{x}^{(n)} - \mu_k)^T}{\sum_{n=1}^{N} 1_{[z^{(n)}=k]}}$$

$$\pi_k = \frac{1}{N} \sum_{n=1}^{N} 1_{[z^{(n)}=k]}$$



Estimating the parameters with more unknowns

When there is no information about which Gaussian generated a particular data point, EM algorithm helps.

An intuition of the 2 important steps (E-step and M-step) of the algorithm:

1. E-step: Compute the posterior probability over z given our current model - i.e, how much we think each Gaussian generates each datapoint.
2. M-step: Assuming that the data really was generated this way, change the parameters of each Gaussian to maximize the probability that it would generate the data it is currently responsible for.



EM for GMM: Overview

1) E-step:
   a) In order to adjust the parameters of the GMM, we must first solve the inference problem: Which Gaussian generated each datapoint?
   b) We cannot be sure, so it is a distribution over all possibilities, given by:

$$\gamma_k^{(n)} = p(z^{(n)} = k|\mathbf{x}^{(n)}; \pi, \mu, \Sigma)$$

2) M-step:
   a) Each Gaussian gets a certain amount of posterior probability for each datapoint.
   b) We fit each Gaussian to the weighted data points.
   c) We derive closed form updates for all the parameters.



EM Derivation

The log-likelihood that we have: $\ell(\mathbf{X}, \Theta) = \sum_i \log(P(\mathbf{x}^{(i)}; \Theta)) = \sum_i \log \left(\sum_j P(\mathbf{x}^{(i)}, z^{(i)} = j; \Theta) \right)$

Optimizing such a function is hard because of the sum inside the log. Here, Θ denotes all of our parameters.

We have the Jensen's inequality for concave functions, given by:

$$f\left(\sum_i p_i x_i\right) \geq \sum_i p_i f(x_i) \quad \text{, where } \sum_i p_i = 1$$

As log function is concave, we rewrite our log-likelihood before applying Jensen's.

$$\ell(\mathbf{X}, \Theta) = \sum_i \log \left(\sum_j q_j \frac{P(\mathbf{x}^{(i)}, z^{(i)} = j; \Theta)}{q_j} \right) \text{ , where } \sum_j q_j = 1$$



EM Derivation

Applying Jensen's on the re-written log-likelihood yields:

$$\sum_i \log \left(\sum_j q_j \frac{P(\mathbf{x}^{(i)}, z^{(i)} = j; \Theta)}{q_j} \right) \geq \sum_i \sum_j q_j \log \left(\frac{P(\mathbf{x}^{(i)}, z^{(i)} = j; \Theta)}{q_j} \right)$$

Though our aim is to maximize the log-likelihood, we work towards maximizing the lower bound (RHS) (modified log-likelihood) and thereby force our likelihood to increase. This is because, maximizing the log-likelihood is a difficult problem to solve.

In order to maximize the modified log-likelihood, we need good estimates of q_j's as well.



EM Derivation

We got the sum outside, but we have an inequality:

$$\ell(\mathbf{X}, \Theta) \geq \sum_i \sum_j q_j \log \left(\frac{P(\mathbf{x}^{(i)}, z^{(i)} = j; \Theta)}{q_j} \right)$$

We fix the current parameters to Θ^{old} and try to find a good q_i.

When we pick $q_j = p(z^{(i)} = j | x^{(i)}, \Theta^{old})$, we get $\frac{P(\mathbf{x}^{(i)}, z^{(i)}; \Theta)}{p(z^{(i)} = j | x^{(i)}, \Theta^{old})} = P(\mathbf{x}^{(i)}; \Theta^{old})$

and the inequality becomes an equality.

We now define and optimize,

$$Q(\Theta) = \sum_i \sum_j p(z^{(i)} = j | x^{(i)}, \Theta^{old}) \log \left(P(\mathbf{x}^{(i)}, z^{(i)} = j; \Theta) \right)$$

$$= \mathbb{E}_{P(z^{(i)} | \mathbf{x}^{(i)}, \Theta^{old})} [\log \left(P(\mathbf{x}^{(i)}, z^{(i)}; \Theta) \right)]$$



What just happened?

- Conceptually: we don't know $z^{(i)}$, so we average them given the current model.
- Practically: we define a function $Q(\Theta) = \mathbb{E}_{P(z^{(i)}|\mathbf{x}^{(i)},\Theta^{old})}[\log\left(P(\mathbf{x}^{(i)}, z^{(i)}; \Theta)\right)]$ that lower bounds the desired function and is equal at our current guess.
- If we now optimize Θ we get a better lower bound.

$$\log(P(\mathbf{X}|\Theta^{old})) = Q(\Theta^{old}) \leq Q(\Theta^{new}) \leq \log(P(\mathbf{X}|\Theta^{new}))$$



The Algorithm

1. Initialize Θ^{old}
2. E-step: Evaluate $p(\mathbf{Z}|\mathbf{X}, \Theta^{old})$ and compute,

$$Q(\Theta, \Theta^{old}) = \sum_{z} p(\mathbf{Z}|\mathbf{X}, \Theta^{old}) \ln p(\mathbf{X}, \mathbf{Z}|\Theta)$$

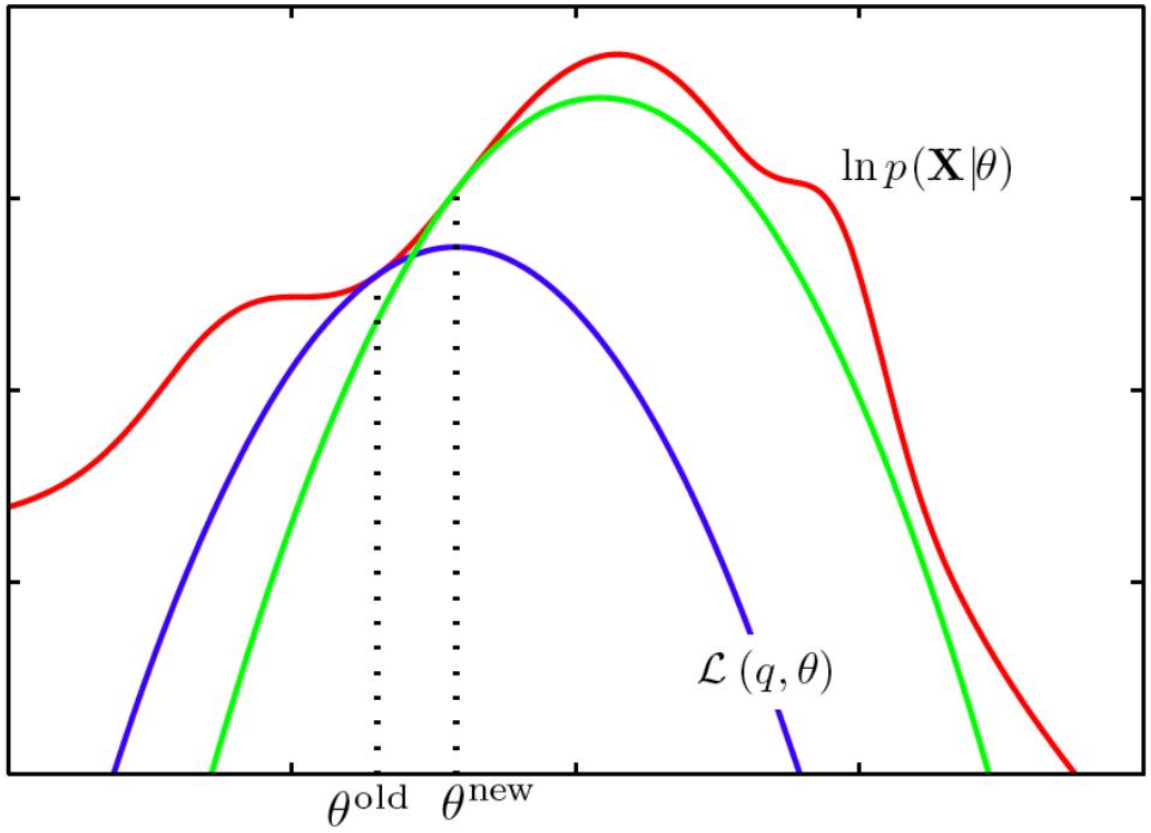
3. M-step: Maximize

$$\Theta^{new} = arg \max_{\Theta} Q(\Theta, \Theta^{old})$$

4. Evaluate log-likelihood and check for convergence. If not converged, set and go to step 2.



Visualization of the EM algorithm



The EM algorithm involves alternately computing a lower bound on the log-likelihood for the current parameter values and then maximizing this bound to obtain the new parameter values.

GMM E-step

Lets see how the EM algorithm works on GMM:

Conditional probability of ***z*** given ***x*** is given by,

$$\begin{aligned}\gamma_k = p(z=k|\mathbf{x}) &= \frac{p(z=k)p(\mathbf{x}|z=k)}{p(\mathbf{x})} \\ &= \frac{p(z=k)p(\mathbf{x}|z=k)}{\sum_{j=1}^{K} p(z=j)p(\mathbf{x}|z=j)} \\ &= \frac{\pi_k \mathcal{N}(\mathbf{x}|\mu_k, \Sigma_k)}{\sum_{j=1}^{K} \pi_j \mathcal{N}(\mathbf{x}|\mu_j, \Sigma_j)}\end{aligned}$$

γ_k can be viewed as responsibility of cluster k towards ***x*** and is similar to the q_j 's we computed earlier.

GMM E-step

Once we computed $\gamma_k^{(i)} = p(z^{(i)} = k|\mathbf{x}^{(i)})$ we can compute the expected likelihood:

$$\mathbb{E}_{P(z^{(i)}|\mathbf{x}^{(i)})}\left[\sum_i \log(P(\mathbf{x}^{(i)}, z^{(i)}|\Theta))\right]$$

$$= \sum_i \sum_k \gamma_k^{(i)} \left(\log(P(z^i = k|\Theta)) + \log(P(\mathbf{x}^{(i)}|z^{(i)} = k, \Theta))\right)$$

$$= \sum_i \sum_k \gamma_k^{(i)} \left(\log(\pi_k) + \log(\mathcal{N}(\mathbf{x}^{(i)}; \mu_k, \Sigma_k))\right)$$

$$= \sum_k \sum_i \gamma_k^{(i)} \log(\pi_k) + \sum_k \sum_i \gamma_k^{(i)} \log(\mathcal{N}(\mathbf{x}^{(i)}; \mu_k, \Sigma_k))$$

When we need to fit k Gaussians, we just need to weight examples by γ_k



GMM M-step

Need to optimize

$$\sum_k \sum_i \gamma_k^{(i)} \log(\pi_k) + \sum_k \sum_i \gamma_k^{(i)} \log(\mathcal{N}(\mathbf{x}^{(i)}; \mu_k, \Sigma_k))$$

Solving for μ_k and Σ_k is like fitting k separate Gaussians but with weights $\gamma_k^{(i)}$

Solution is similar to what we've seen earlier:

$$\mu_k = \frac{1}{N_k} \sum_{n=1}^{N} \gamma_k^{(n)} \mathbf{x}^{(n)}$$

$$\Sigma_k = \frac{1}{N_k} \sum_{n=1}^{N} \gamma_k^{(n)} (\mathbf{x}^{(n)} - \mu_k)(\mathbf{x}^{(n)} - \mu_k)^T$$

$$\pi_k = \frac{N_k}{N} \quad \text{with} \quad N_k = \sum_{n=1}^{N} \gamma_k^{(n)}$$



EM Algorithm for GMM

Initialize the μ_k, Σ_k, π_k and iterate over the following steps until convergence:

1. E-step: Evaluate the responsibilities given the current parameters

$$\gamma_k^{(n)} = p(z^{(n)}|\mathbf{x}) = \frac{\pi_k \mathcal{N}(\mathbf{x}^{(n)}|\mu_k, \Sigma_k)}{\sum_{j=1}^{K} \pi_j \mathcal{N}(\mathbf{x}^{(n)}|\mu_j, \Sigma_j)}$$

2. M-step: Re-estimate the parameters given the current responsibilities

$$\mu_k = \frac{1}{N_k} \sum_{n=1}^{N} \gamma_k^{(n)} \mathbf{x}^{(n)}$$

$$\Sigma_k = \frac{1}{N_k} \sum_{n=1}^{N} \gamma_k^{(n)} (\mathbf{x}^{(n)} - \mu_k)(\mathbf{x}^{(n)} - \mu_k)^T$$

$$\pi_k = \frac{N_k}{N} \quad \text{with} \quad N_k = \sum_{n=1}^{N} \gamma_k^{(n)}$$

3. Evaluate the log-likelihood and check for convergence

$$\ln p(\mathbf{X}|\pi, \mu, \Sigma) = \sum_{n=1}^{N} \ln \left(\sum_{k=1}^{K} \pi_k \mathcal{N}(\mathbf{x}^{(n)}|\mu_k, \Sigma_k) \right)$$

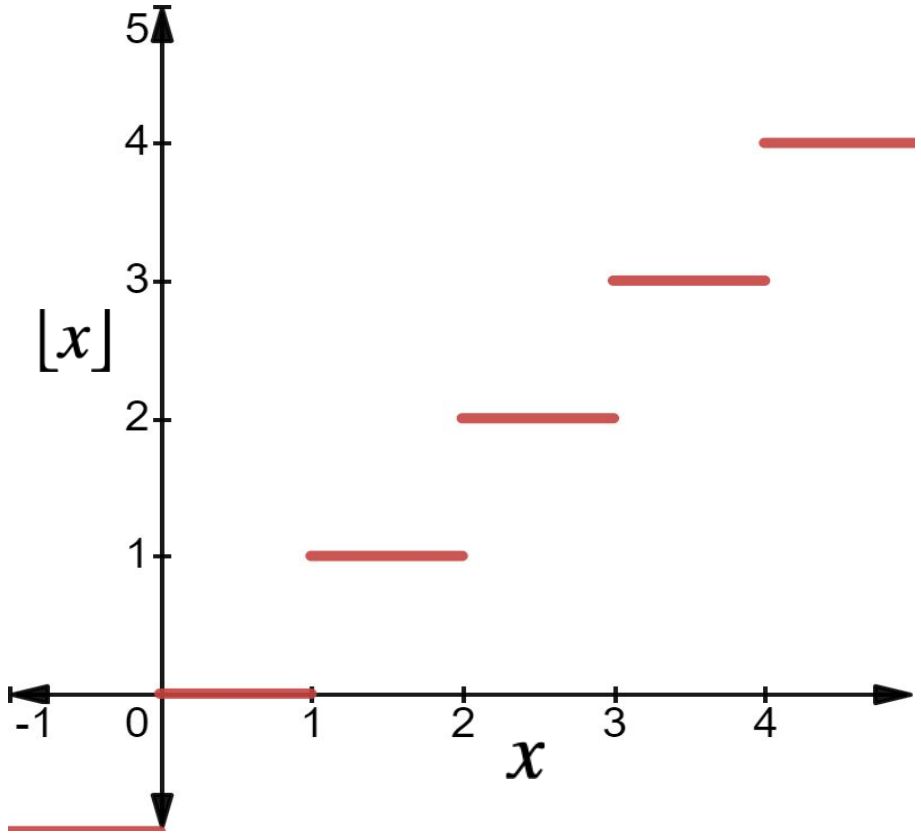


GMM and EM recap

- GMM provides a probabilistic view of clustering - each cluster corresponds to a different Gaussian
- Modeled using latent variables
- Estimation of parameters is done using the EM algorithm
- EM is a general algorithm for optimizing latent variable models
- EM iteratively computes a lower bound and then optimizes it
- EM converges usually to a local optima



Quantization



floor function

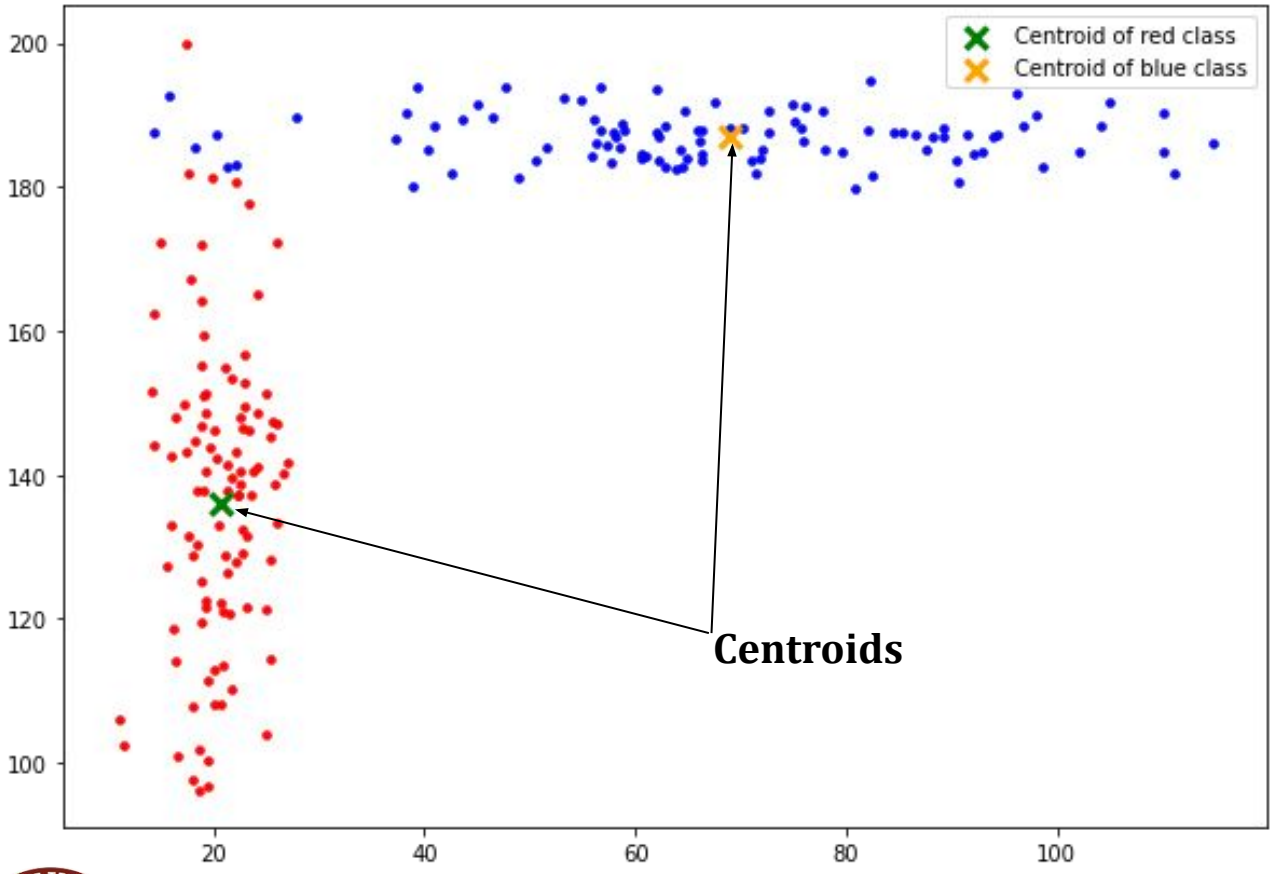
Quantization of Number: Idea behind quantization of numbers is to **represent numbers using some form of encodings**.

For example:- we can represent 0 to 1024 numbers using 10 bits binary encoding.

Quantization of Real Numbers: We can use functions like ***floor*** **(Least Integer Function)** to quantize real numbers between some range into integers.

For example:- $x \in [0,1), \lfloor x \rfloor = 0$

Vector Quantization



- Idea behind vector quantization is to divide a large set of vectors {**V**} into a small group of vectors {**G**} using **clustering techniques.**
- Each vector in {**G**} is the centroid of the group of vectors that it represents.
- It's a technique used to c**ompress information**.

Methods for Vector Quantization

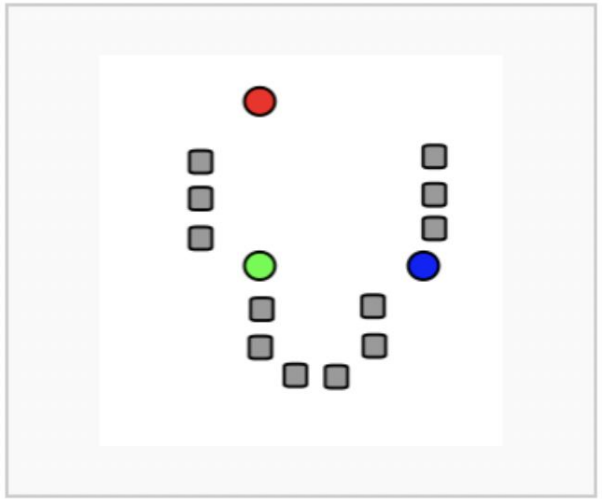
We will discuss two common methods of clustering:

1. **K-means**

2. **Linde-Buzo-Gray(LBG) Algorithm**

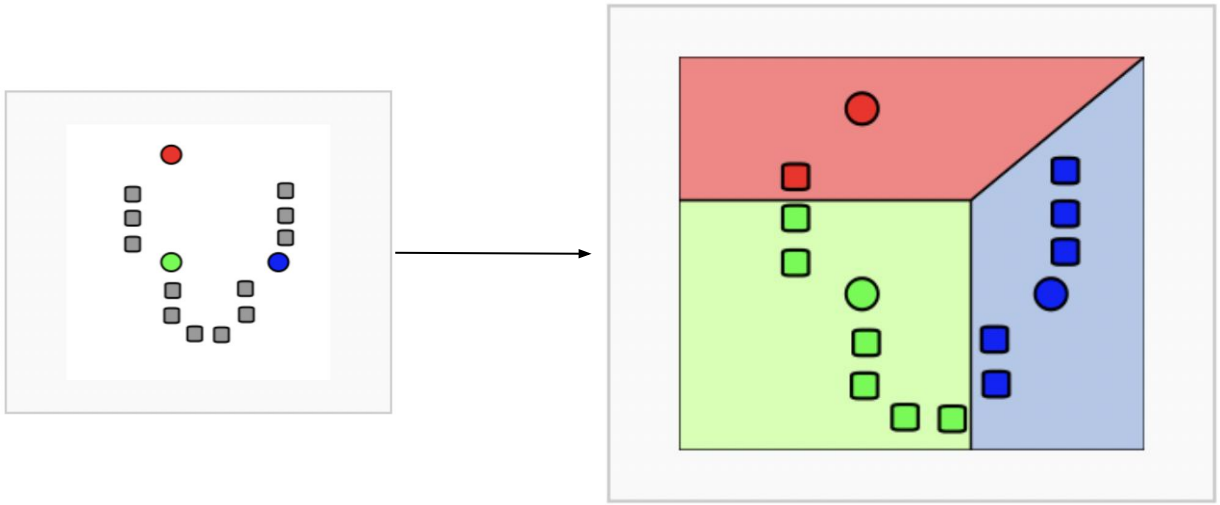


K-Means: Step 1



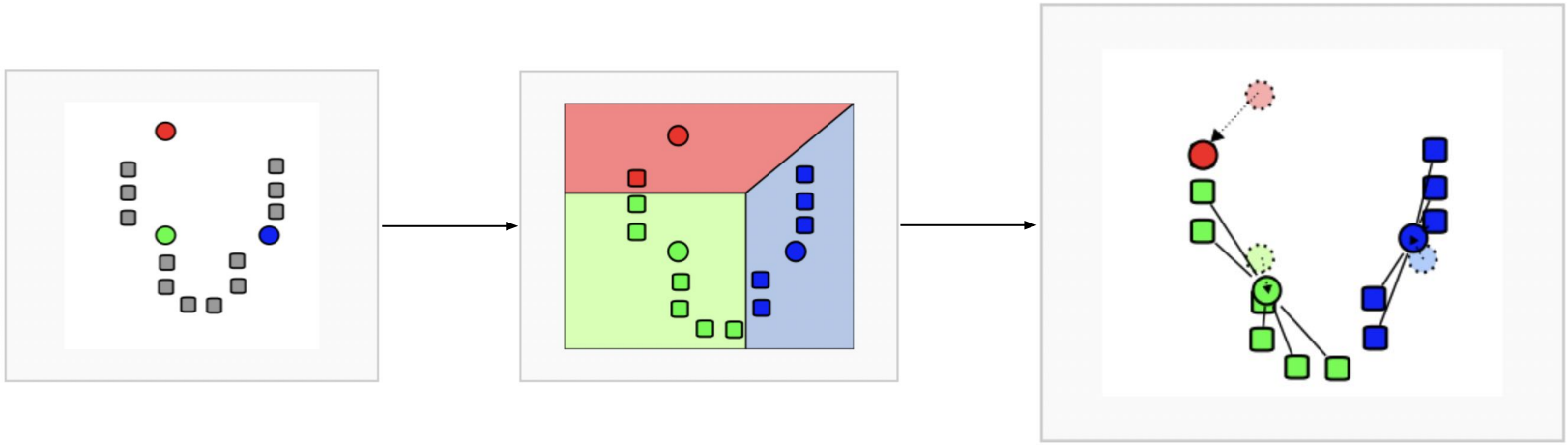
Given a large set of vectors (▢), we decide the **number of clusters (K)** that we want (In our case K = 3). First step is to randomly initialize some set of points as c**luster centroids** (● ● ●)

K-Means: Step 2



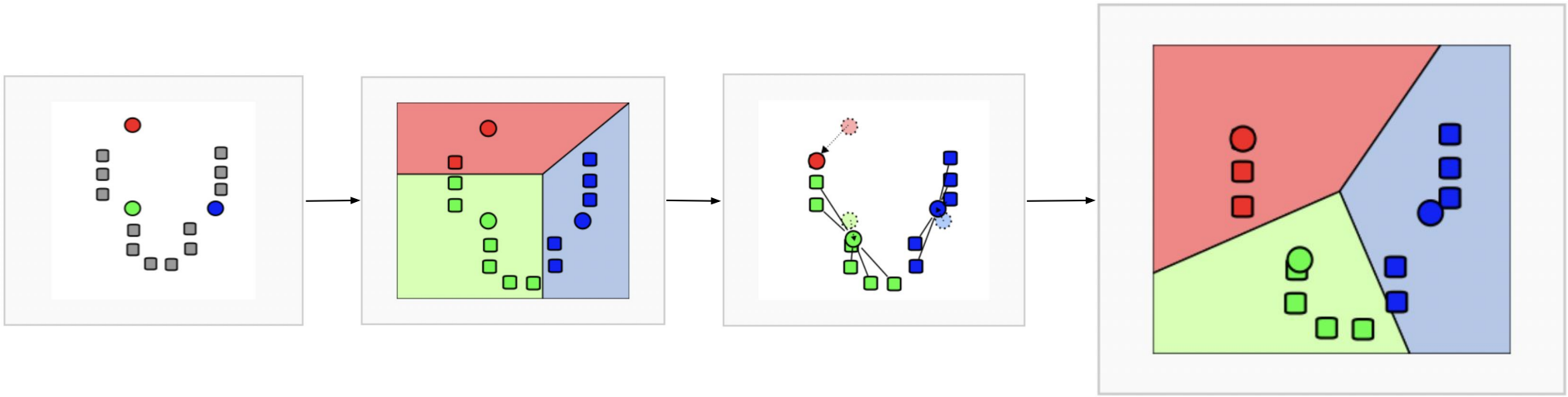
Based on the 3 random cluster centroids, we create 3 clusters by **associating every vector with nearest centroid** (We use **Euclidean Distance** to measure the distance between two vector)

K-Means: Step 3



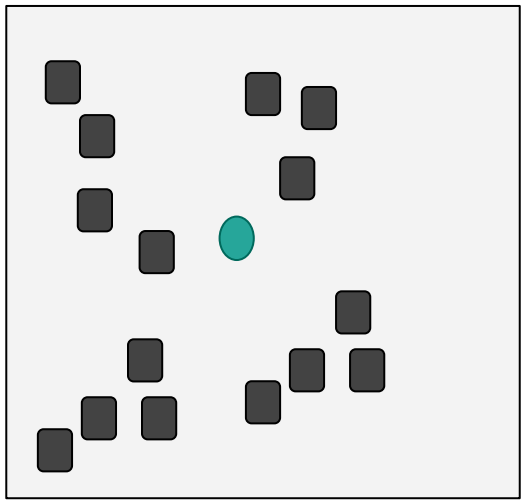
We reestimate the centroids using the vectors that are **associated with each cluster.**

K-Means: Step 4



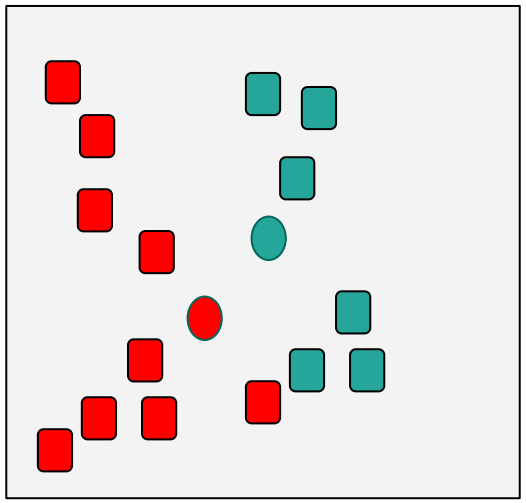
Finally Step 2 and Step 3 are repeated until **convergence is reached**

Linde-Buzo-Gray Algorithm (LBG): Step 1



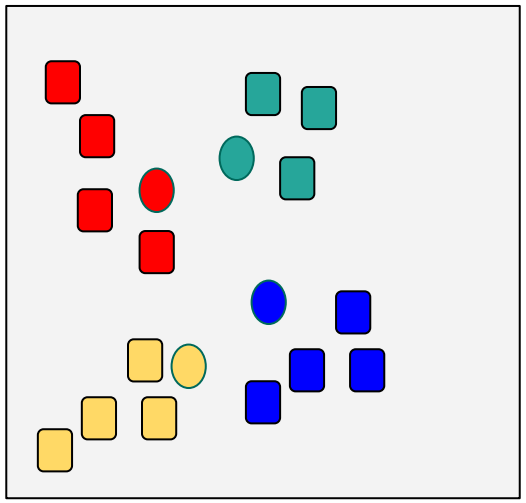
Given a large set of vectors (■), the first step is to calculate the centroid (●) across all the vectors.

Linde-Buzo-Gray Algorithm (LBG): Step 2



Given a centroid (**c**) we define 2 new centroids **c - δ** and **c + δ** (● ●) , where **δ** is some **small distortion**. We create clusters based on assigning each vector to nearest centroid.

Linde-Buzo-Gray Algorithm (LBG): Step 3



Repeat the **Step 1** and **Step 2** until the **computed distortion after splitting the centroids becomes greater than before splitting the centroids.**

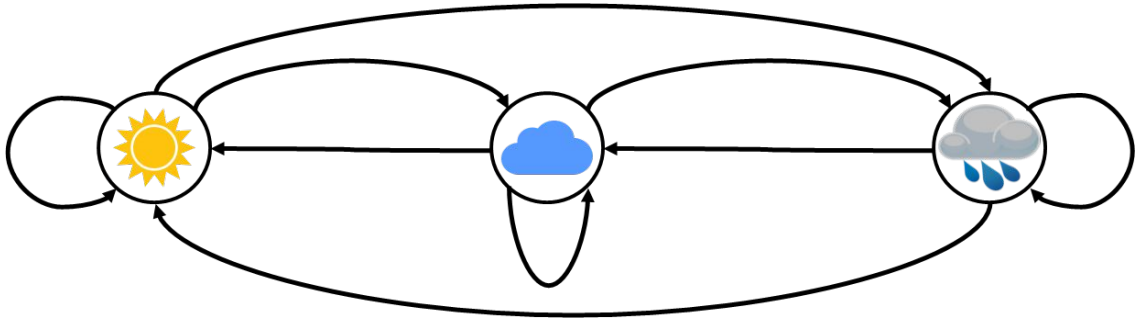
Modelling Sequential Data

- Until now, we learned how to model non-sequential data (i.e) data that has no order information. **Speech is sequential data** in which different sounds are being produced at different instances of time in a particular order
- Examples of sequential data: **weather, stock market, DNA sequence, text document**
- Let's see how sequential data can be modelled



Markov Chain - Example

- Assume that there are **three possible weather conditions** namely **sunny**, **cloudy** and **rainy**
- The sequence of weather conditions can be pictured as a **state machine** where each state represents a distinct weather condition
- The arrows from one state to other states represent the state transitions and are associated with **state transition probabilities**
- Today's weather can always be predicted solely by the knowledge of previous day's weather in **First order Markov chain**



Markov Chain - Example

- Let's say that we have observed the following weather sequence:

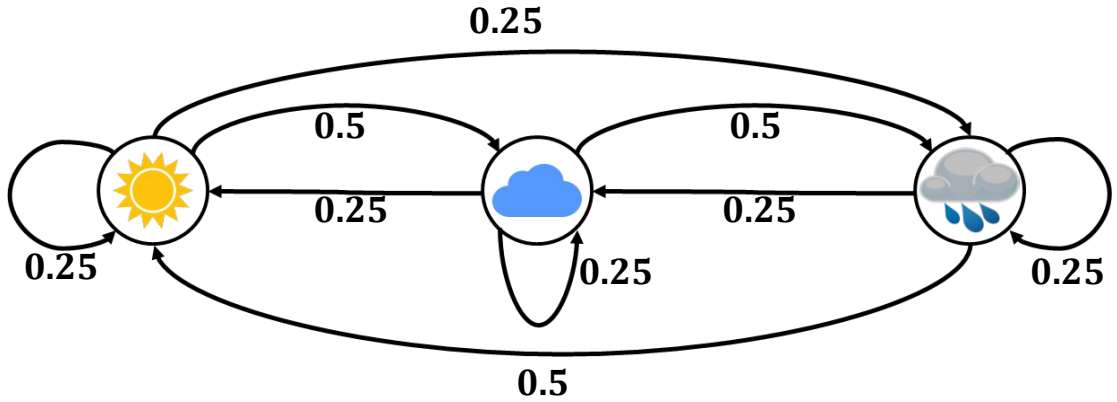
  [sunny, cloudy, rainy, rainy, cloudy, sunny, sunny, rainy, sunny, cloudy, cloudy, rainy, sunny]
- We can train the Markov chain using this data (i.e) we can compute the transition probability matrix
- For example, let's calculate **the probability of today being sunny given that yesterday was cloudy** which will correspond to one entry in the transition probability matrix
- In the given sequence, **how many times sunny day follows a cloudy day**? One time
- How many **cloudy days are there in total**? Four

$\therefore P(today=sunny \mid yesterday=cloudy) = ¼ = 0.25$

Markov Chain - Example

- Similarly, we can compute all the N^2 transition probabilities when there are N states. In this example, we have 3 states and hence 9 probabilities.
- Each probability corresponds to a state transition.
- The computed transition probability matrix will be

		Today		
		sunny	cloudy	rainy
Yesterday	sunny	0.25	0.5	0.25
	cloudy	0.25	0.25	0.5
	rainy	0.5	0.25	0.25



- Once we have the transition probability matrix, we can solve various problems related to Markov chain

Probability of a weather sequence

- Given yesterday was cloudy, the probability of observing the following sequence **sunny, sunny, cloudy, rainy, cloudy** is

$$P(sunny \mid cloudy) \cdot P(sunny \mid sunny) \cdot P(cloudy \mid sunny) \cdot P(rainy \mid cloudy) \cdot P(cloudy \mid rainy) =$$
$$0.25 * 0.25 * 0.5 * 0.5 * 0.25$$

		Today		
		sunny	cloudy	rainy
Yesterday	sunny	0.25	0.5	0.25
	cloudy	0.25	0.25	0.5
	rainy	0.5	0.25	0.25

Predicting weather condition of next 3 days

Yesterday \ Today	Sunny	Cloudy	Rainy
Sunny	0.5	0.375	0.125
Cloudy	0.25	0.125	0.625
Rainy	0.25	0.075	0.675

- Let's assume the above probability transition matrix.
- Suppose **Day 0 is Sunny**, we want to find **what weather is on Day 1**?
- We look at the transition probability matrix and find out the maximum probability out of these three: **max{P(Sunny|Sunny), P(Cloudy|Sunny), P(Rainy|Sunny)}**
- We find out that **P(Sunny|Sunny)** is maximum (i.e) 0.5 and so **Day 1 being Sunny is most probable.**

Predicting weather condition of next 3 days

- Now suppose **Day 1 is Sunny**, **what will be Day 2?**
- We again find maximum out of these three:
  max{P(Sunny|Sunny), P(Cloudy|Sunny), P(Rainy|Sunny)}
- We find out that **P(Sunny|Sunny)** is maximum and hence **Day 2 will be Sunny** is most probable.
- Similarly, we can find out that **Day 3 being Sunny** is most probable.
- This approach is called **Greedy approach**.
- Greedy approach does not give the best possible sequence
- We can compute the probability of all possible sequences and choose the best



		Today		
		Sunny	**Cloudy**	**Rainy**
Yesterday	**Sunny**	0.5	0.375	0.125
	Cloudy	0.25	0.125	0.625
	Rainy	0.25	0.075	0.675

Predicting weather condition of next 3 days

Day 1	Day 2	Day 3	Probability of the sequence		
Sunny	Sunny	Sunny	P(Sunny|Sunny)*P(Sunny|Sunny)*P(Sunny|Sunny)	0.5*0.5*0.5	0.125
Sunny	Sunny	Cloudy	P(Sunny|Sunny)*P(Sunny|Sunny)*P(Cloudy|Sunny)	0.5*0.5*0.375	0.09375
Sunny	Sunny	Rainy	P(Sunny|Sunny)*P(Sunny|Sunny)*P(Rainy|Sunny)	0.5*0.5*0.125	0.03125
Sunny	Cloudy	Sunny	P(Sunny|Sunny)*P(Cloudy|Sunny)*P(Sunny|Cloudy)	0.5*0.375*0.25	0.046875
Sunny	Cloudy	Cloudy	P(Sunny|Sunny)*P(Cloudy|Sunny)*P(Cloudy|Cloudy)	0.5*0.375*0.125	0.0234375
Sunny	Cloudy	Rainy	P(Sunny|Sunny)*P(Cloudy|Sunny)*P(Rainy|Cloudy)	0.5*0.375*0.625	0.1171875
Sunny	Rainy	Sunny	P(Sunny|Sunny)*P(Rainy|Sunny)*P(Sunny|Rainy)	0.5*0.125*0.25	0.015625
Sunny	Rainy	Cloudy	P(Sunny|Sunny)*P(Rainy|Sunny)*P(Cloudy|Rainy)	0.5*0.125*0.075	0.0046875
Sunny	Rainy	Rainy	P(Sunny|Sunny)*P(Rainy|Sunny)*P(Rainy|Rainy)	0.5*0.125*0.675	0.0421875
Cloudy	Sunny	Sunny	P(Cloudy|Sunny)*P(Sunny|Cloudy)*P(Sunny|Sunny)	0.375*0.25*0.5	0.046875
Cloudy	Sunny	Cloudy	P(Cloudy|Sunny)*P(Sunny|Cloudy)*P(Cloudy|Sunny)	0.375*0.25*0.375	0.03515625
Cloudy	Sunny	Rainy	P(Cloudy|Sunny)*P(Sunny|Cloudy)*P(Rainy|Sunny)	0.375*0.25*0.125	0.01171875
Cloudy	Cloudy	Sunny	P(Cloudy|Sunny)*P(Cloudy|Cloudy)*P(Sunny|Cloudy)	0.375*0.125*0.25	0.01171875
Cloudy	Cloudy	Cloudy	P(Cloudy|Sunny)*P(Cloudy|Cloudy)*P(Cloudy|Cloudy)	0.375*0.125*0.125	0.00585938
Cloudy	Cloudy	Rainy	P(Cloudy|Sunny)*P(Cloudy|Cloudy)*P(Rainy|Cloudy)	0.375*0.125*0.625	0.02929688
Cloudy	Rainy	Sunny	P(Cloudy|Sunny)*P(Rainy|Cloudy)*P(Sunny|Rainy)	0.375*0.625*0.25	0.05859375
Cloudy	Rainy	Cloudy	P(Cloudy|Sunny)*P(Rainy|Cloudy)*P(Cloudy|Rainy)	0.375*0.625*0.075	0.01757813
Cloudy	Rainy	Rainy	P(Cloudy|Sunny)*P(Rainy|Cloudy)*P(Rainy|Rainy)	0.375*0.625*0.675	0.15820313
Rainy	Sunny	Sunny	P(Rainy|Sunny)*P(Sunny|Rainy)*P(Sunny|Sunny)	0.125*0.25*0.5	0.015625
Rainy	Sunny	Cloudy	P(Rainy|Sunny)*P(Sunny|Rainy)*P(Cloudy|Sunny)	0.125*0.25*0.375	0.01171875
Rainy	Sunny	Rainy	P(Rainy|Sunny)*P(Sunny|Rainy)*P(Rainy|Sunny)	0.125*0.25*0.125	0.00390625
Rainy	Cloudy	Sunny	P(Rainy|Sunny)*P(Cloudy|Rainy)*P(Sunny|Cloudy)	0.125*0.075*0.25	0.00234375
Rainy	Cloudy	Cloudy	P(Rainy|Sunny)*P(Cloudy|Rainy)*P(Cloudy|Cloudy)	0.125*0.075*0.125	0.00117188
Rainy	Cloudy	Rainy	P(Rainy|Sunny)*P(Cloudy|Rainy)*P(Rainy|Cloudy)	0.125*0.075*0.625	0.00585938
Rainy	Rainy	Sunny	P(Rainy|Sunny)*P(Rainy|Rainy)*P(Sunny|Rainy)	0.125*0.675*0.25	0.02109375
Rainy	Rainy	Cloudy	P(Rainy|Sunny)*P(Rainy|Rainy)*P(Cloudy|Rainy)	0.125*0.675*0.075	0.00632813
Rainy	Rainy	Rainy	P(Rainy|Sunny)*P(Rainy|Rainy)*P(Rainy|Rainy)	0.125*0.675*0.675	0.05695313

→ **Greedy Solution**

→ **Best Solution**

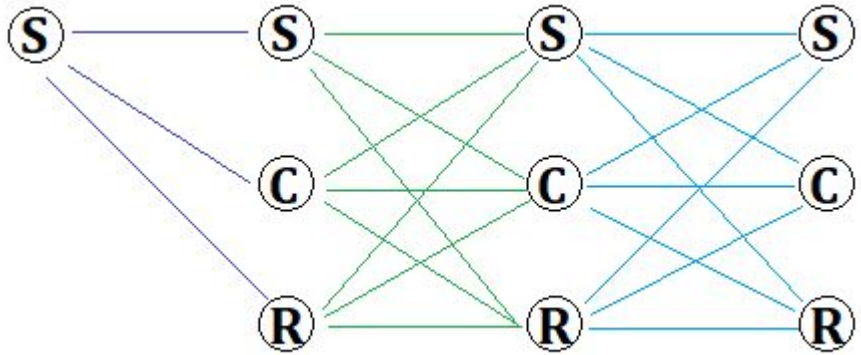


Here we have to do 27 calculations. Choose the sequence with maximum probability. This need not give same solution as the greedy approach

Total no. of paths for 3 days forecasting

D1 D2 D3

3 x 3 x 3 =27 different paths



Total no. of paths for 5 days forecasting

D1 D2 D3 D4 D5

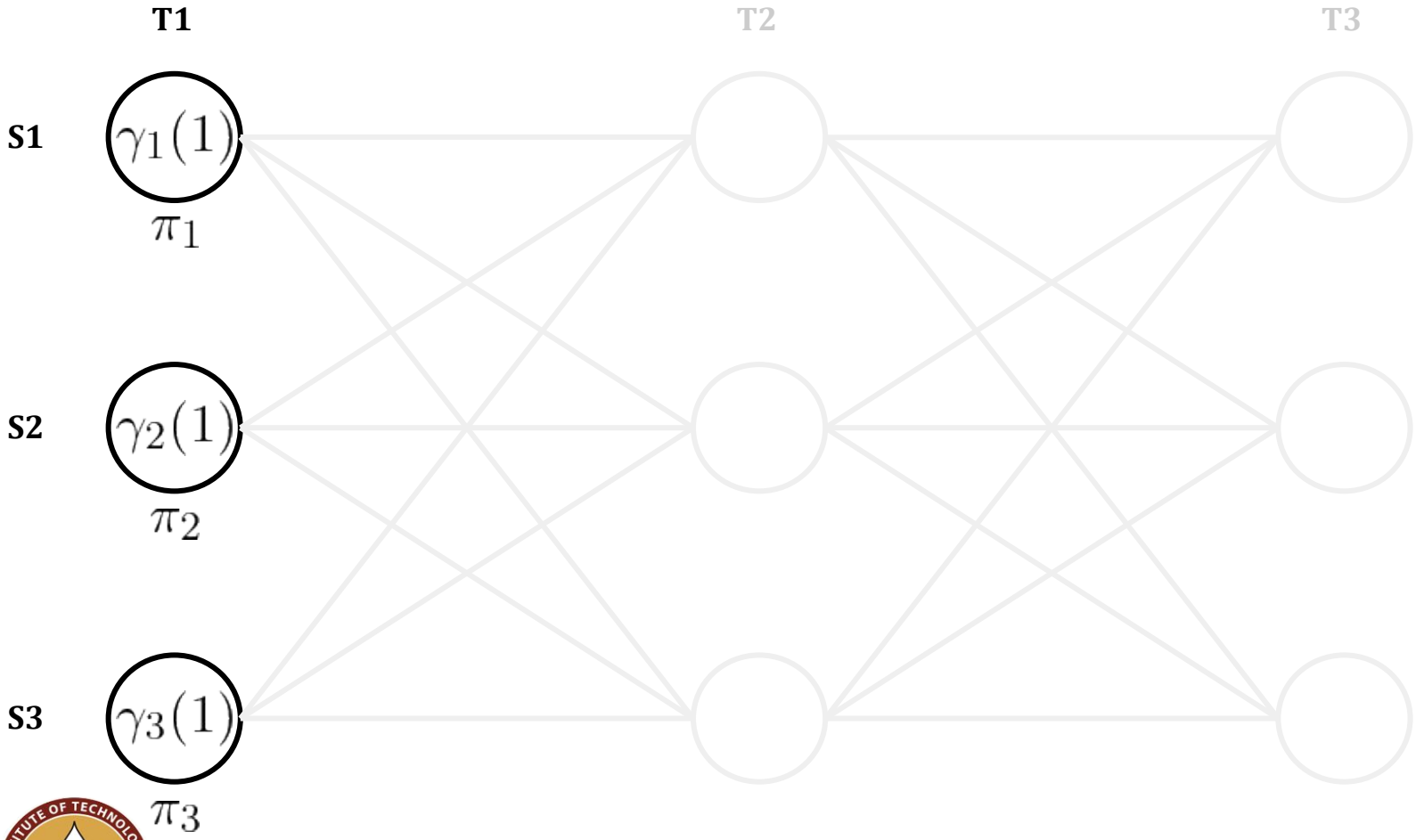
3 x 3 x 3 x 3 x 3 =243 different paths

Total no. of paths

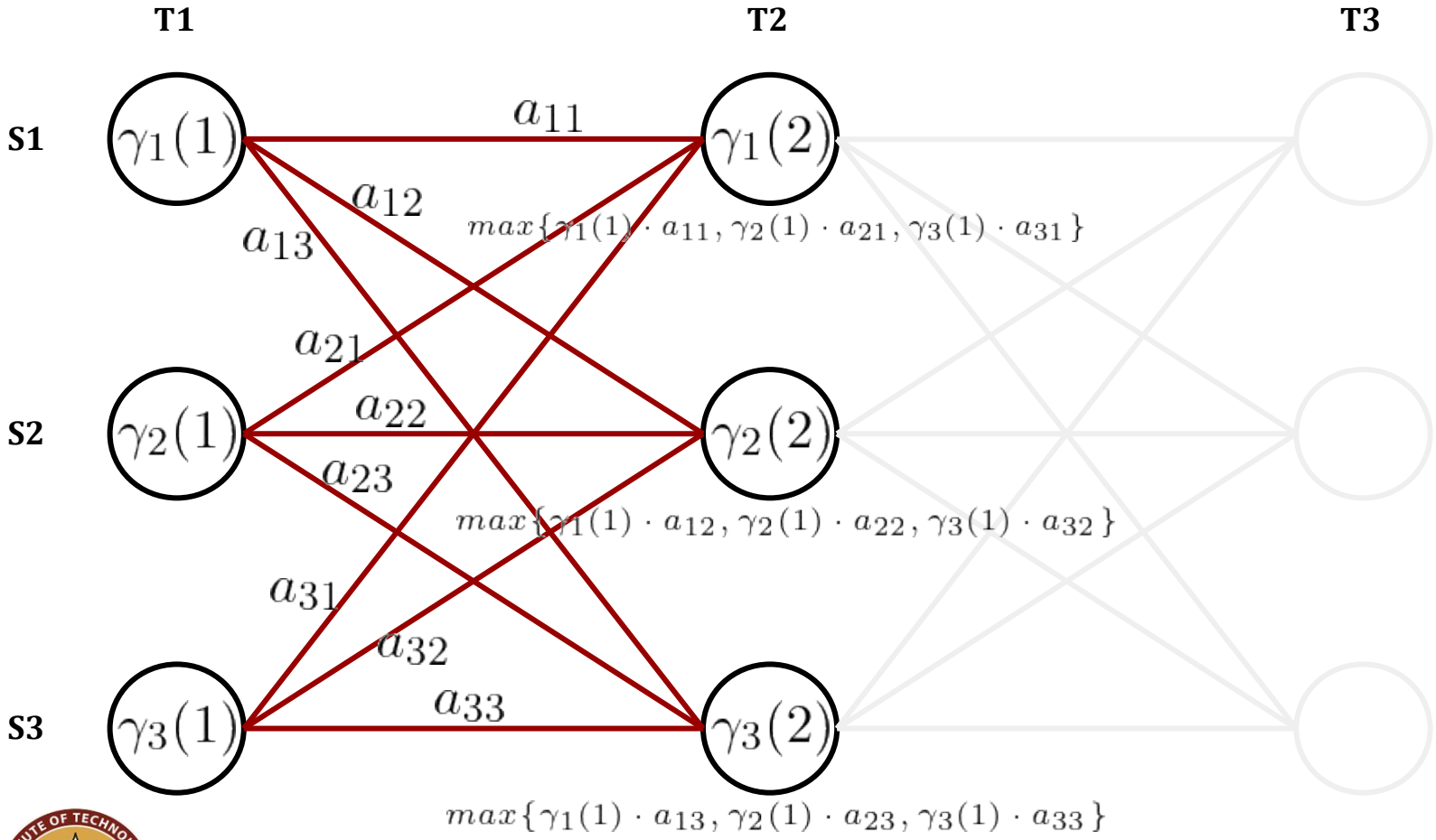
- In general, if there are **N states** and **the length of the sequence is T**, then the total **number of possible paths is N^T**
- As total no. of paths is very large, this process is expensive.
- There is an effective algorithm to find the most probable path in very less time complexity.
- The algorithm is known as **Viterbi Algorithm**. Please note it is **not an approximation.**



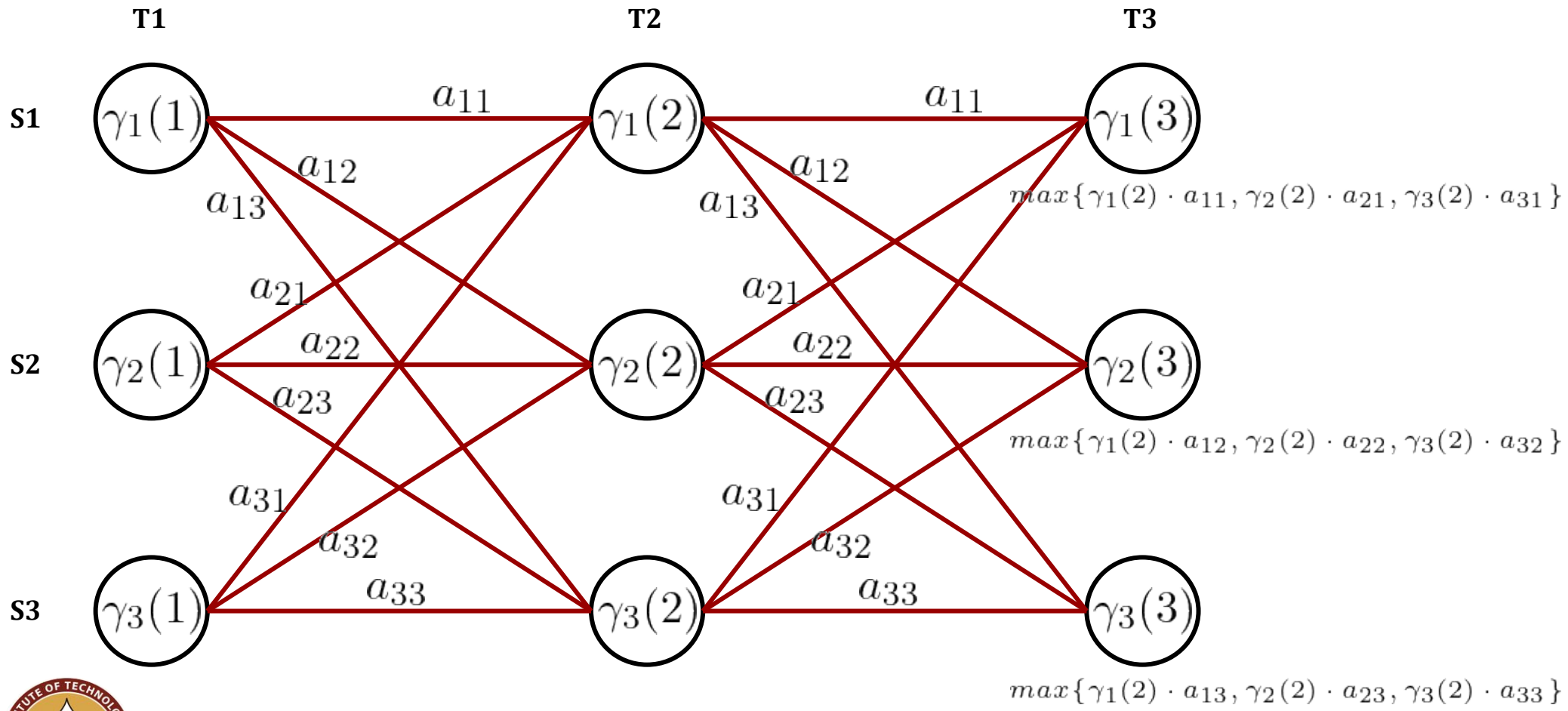
Viterbi Algorithm



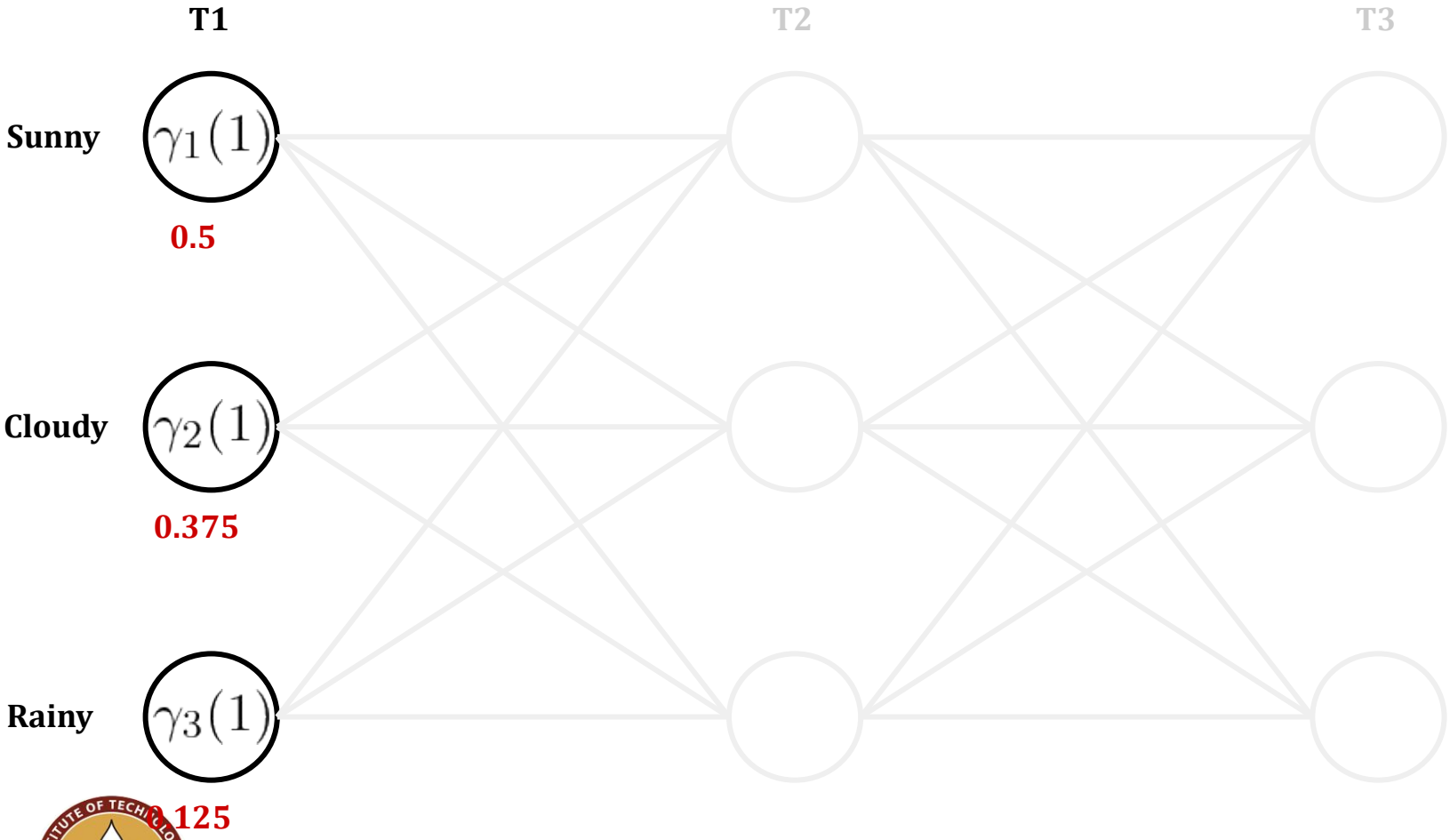
Viterbi Algorithm



Viterbi Algorithm



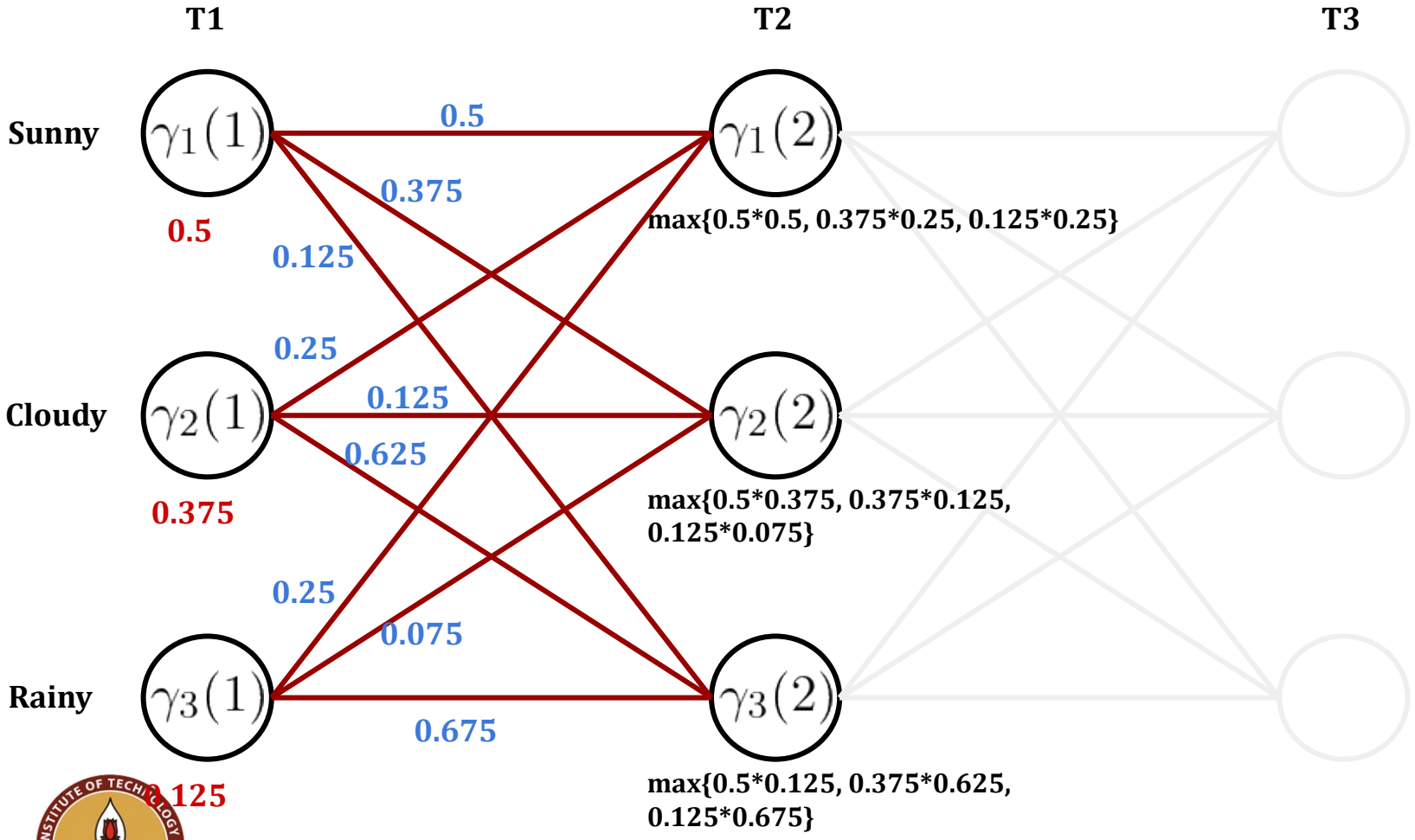
Viterbi Algorithm - Weather Example



- Given that **Day 0 is Sunny**, we want to find the best possible weather sequence for the next 3 days.
- The **first row** of the transition matrix will give us the initial probabilities since we know that Day 0 is Sunny

Yesterday \ Today	Sunny	Cloudy	Rainy
Sunny	0.5	0.375	0.125
Cloudy	0.25	0.125	0.625
Rainy	0.25	0.075	0.675

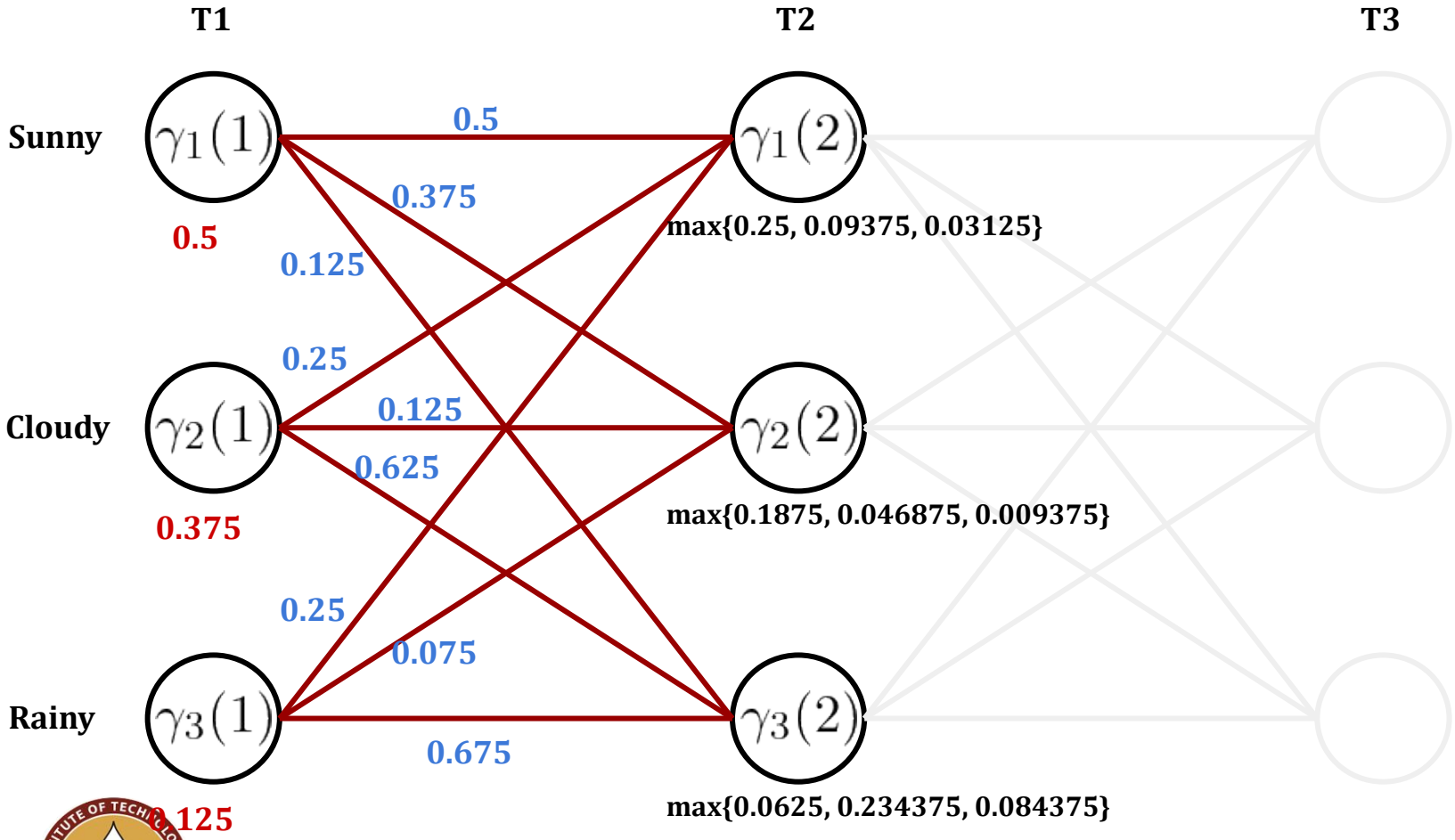
Viterbi Algorithm - Weather Example



Today

Yesterday	Sunny	Cloudy	Rainy
Sunny	0.5	0.375	0.125
Cloudy	0.25	0.125	0.625
Rainy	0.25	0.075	0.675

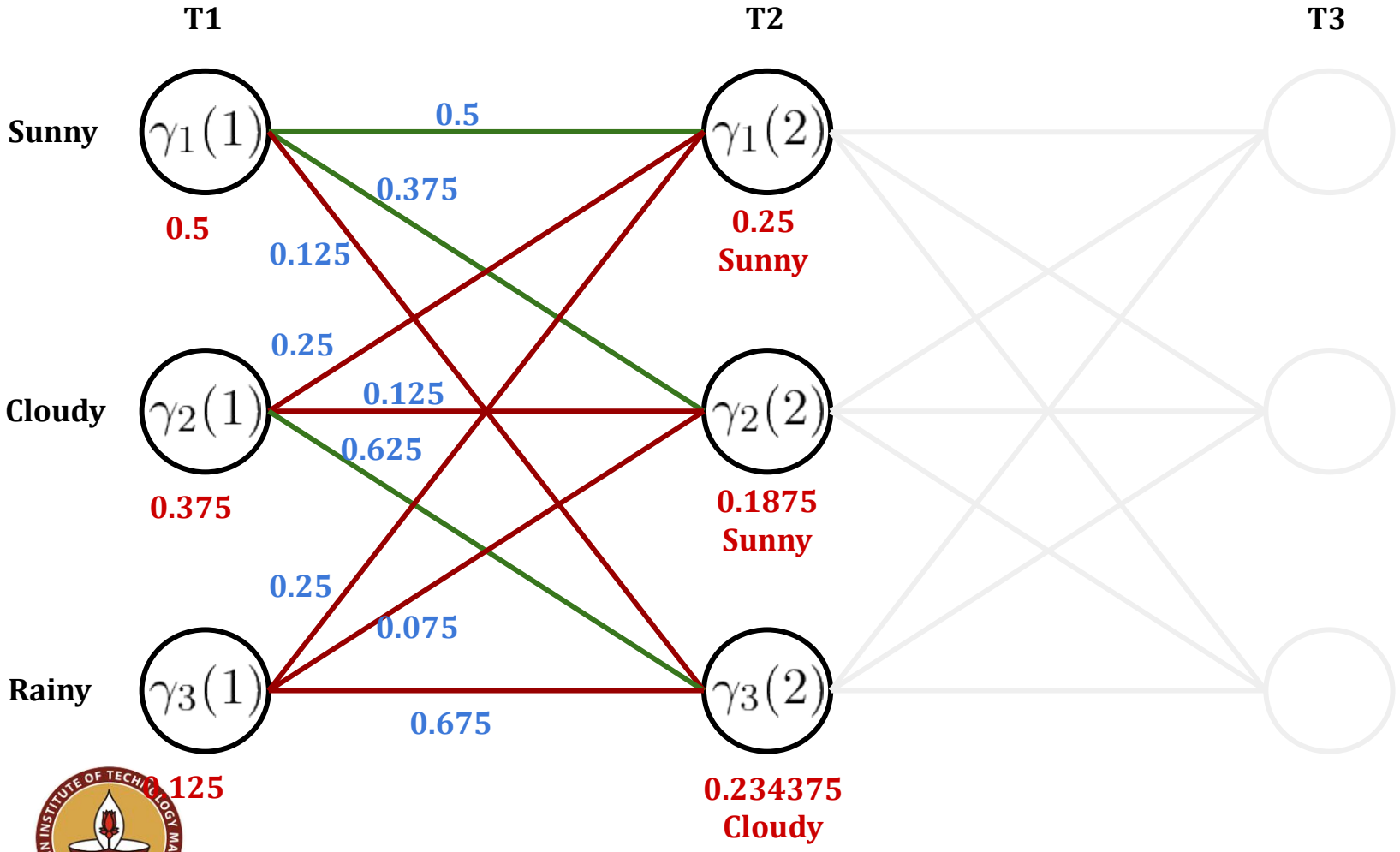
Viterbi Algorithm - Weather Example



Today

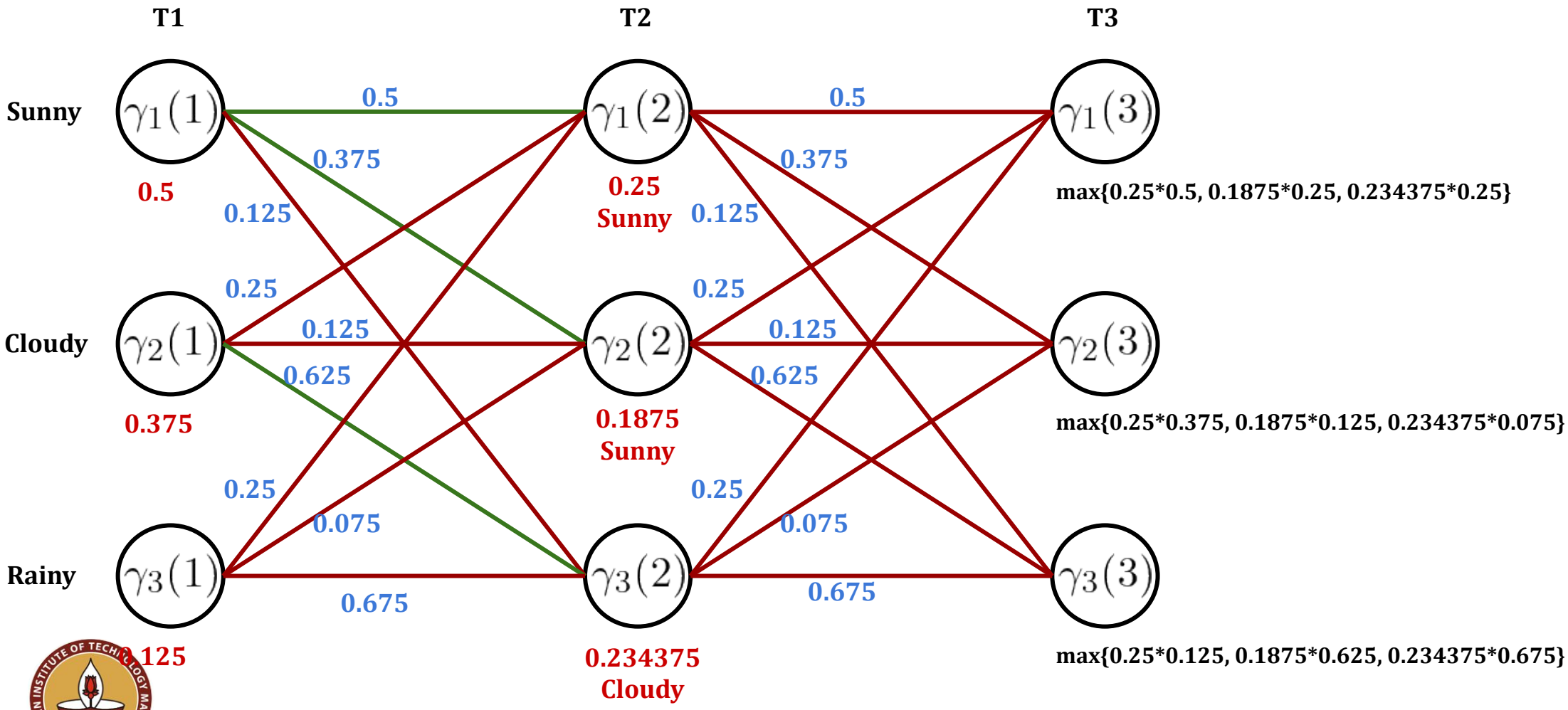
Yesterday	Sunny	Cloudy	Rainy
Sunny	0.5	0.375	0.125
Cloudy	0.25	0.125	0.625
Rainy	0.25	0.075	0.675

Viterbi Algorithm - Weather Example

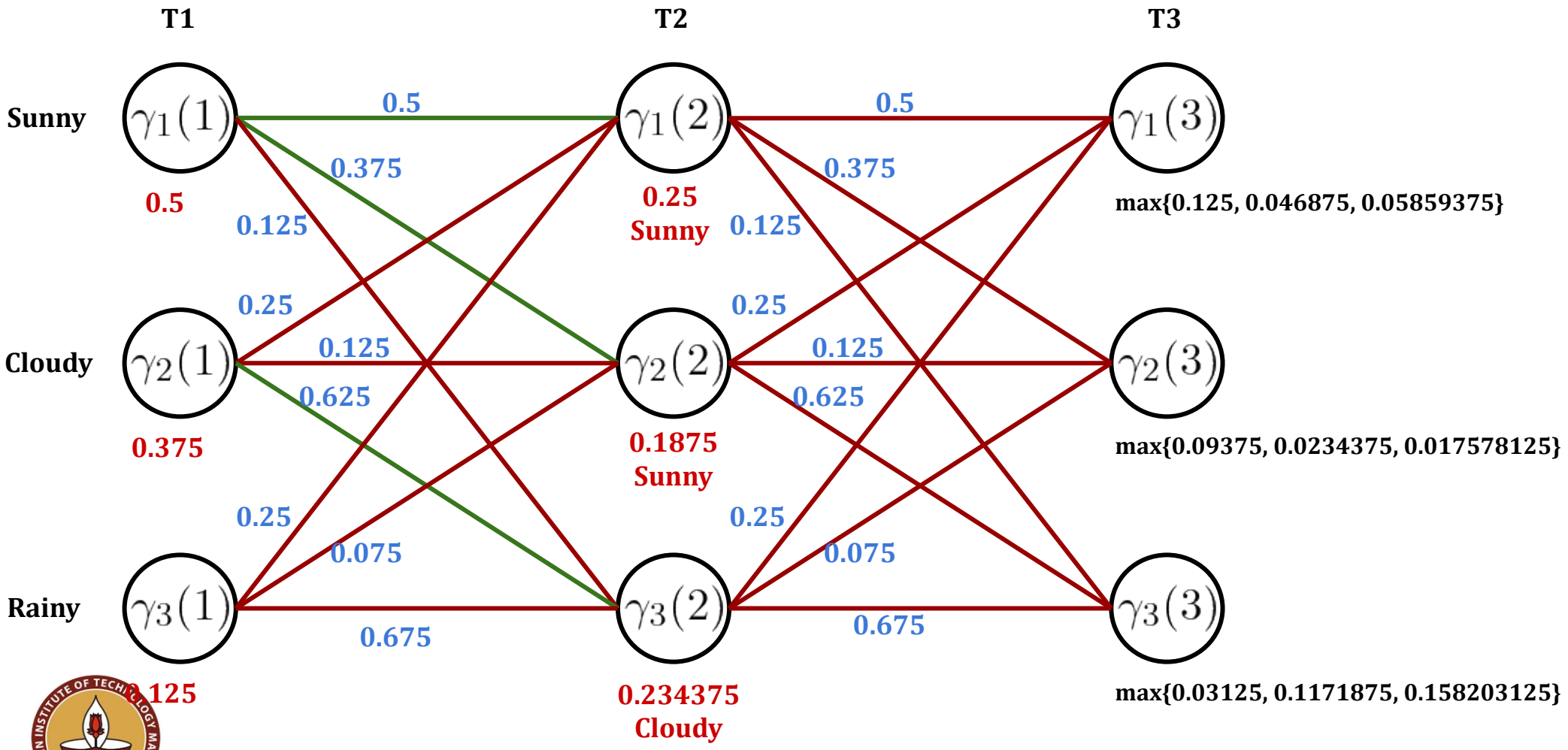


Yesterday \ Today	Sunny	Cloudy	Rainy
Sunny	0.5	0.375	0.125
Cloudy	0.25	0.125	0.625
Rainy	0.25	0.075	0.675

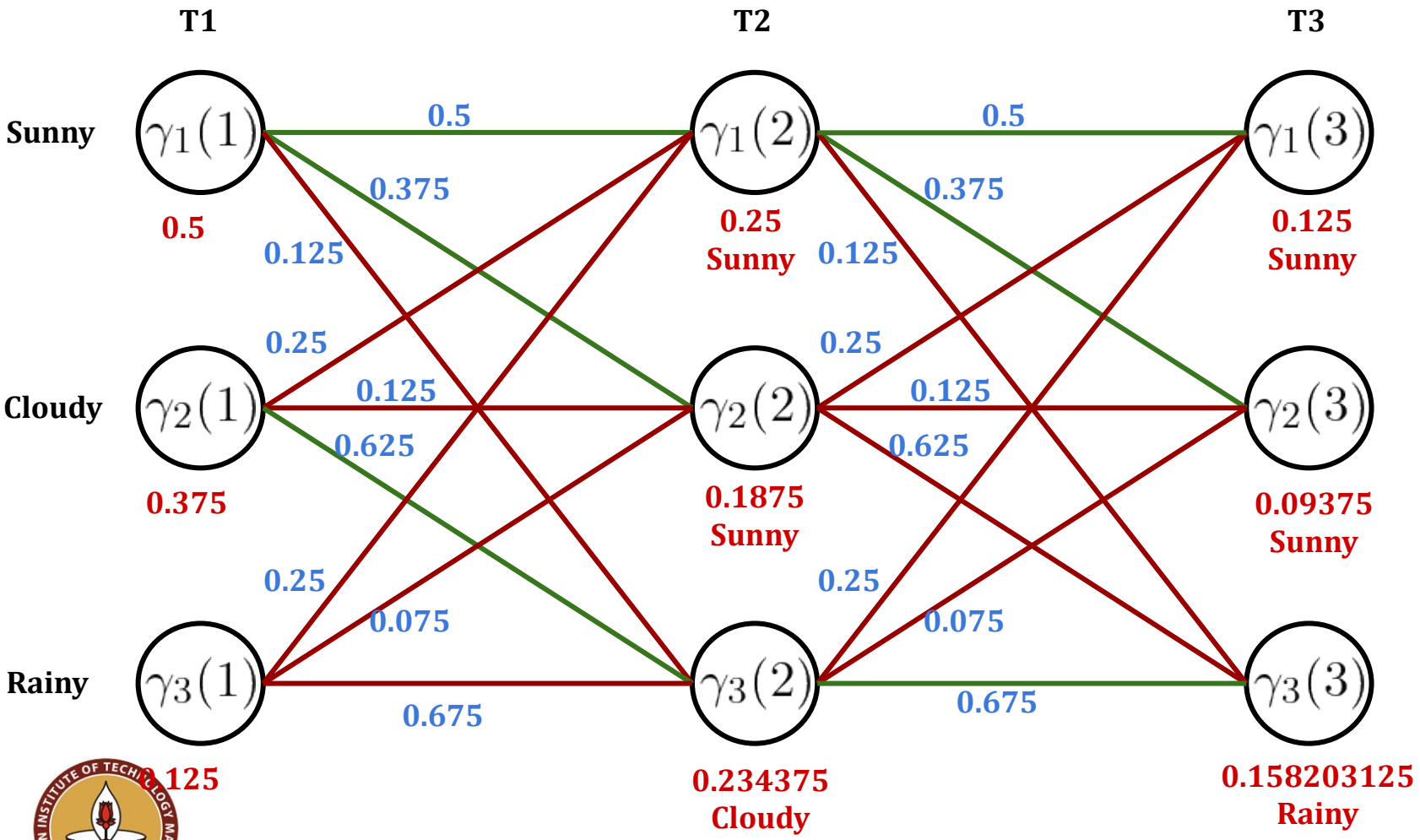
Viterbi Algorithm - Weather Example



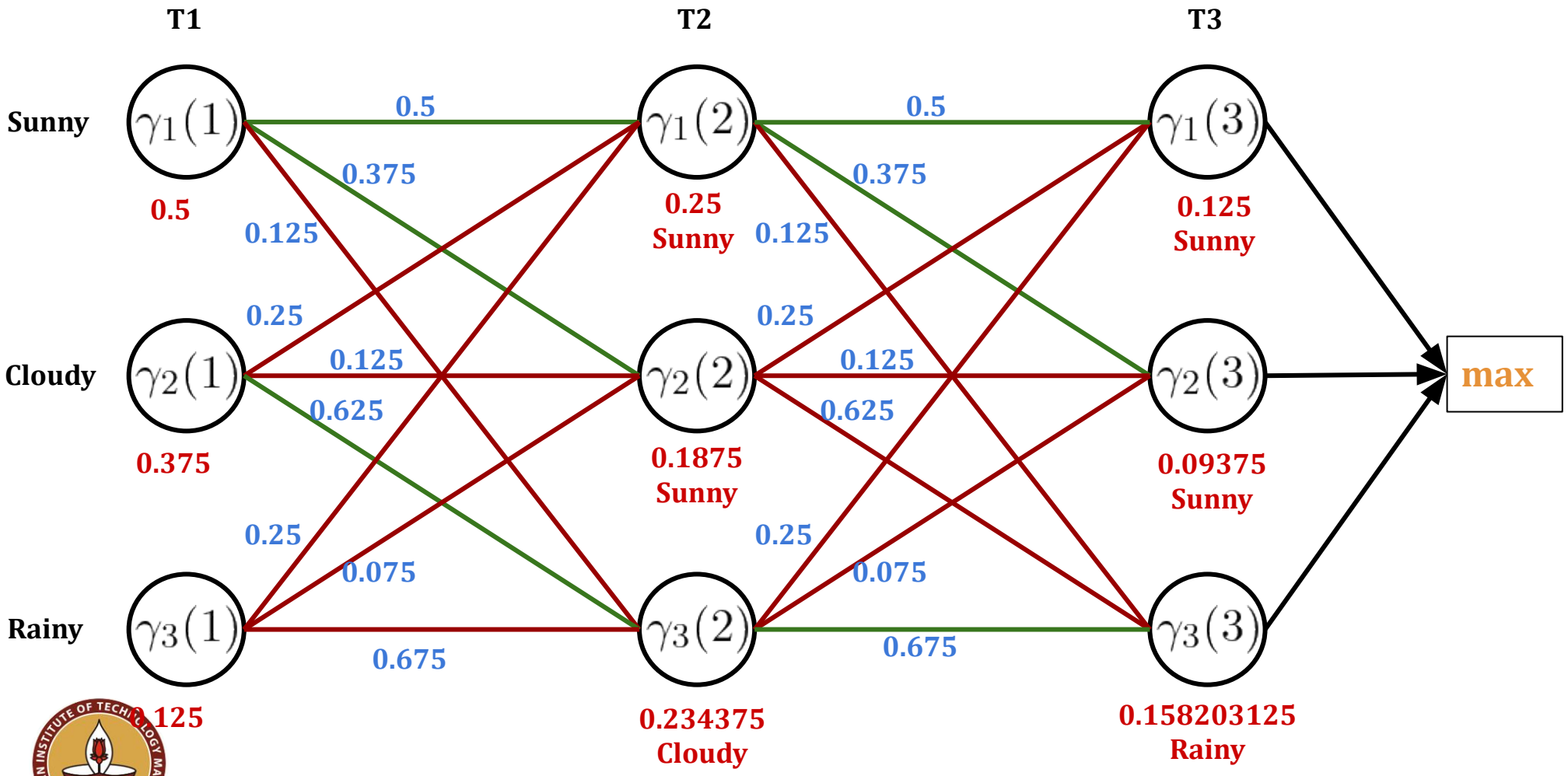
Viterbi Algorithm - Weather Example



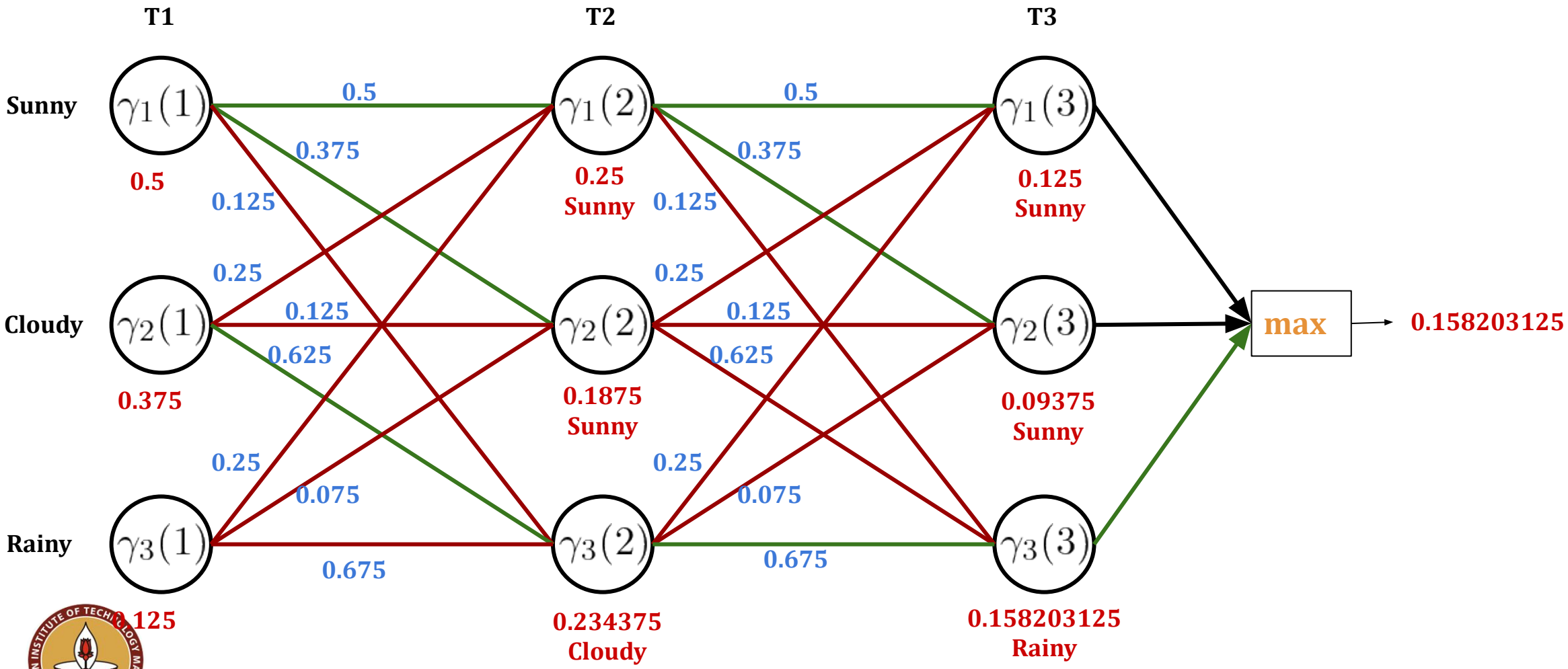
Viterbi Algorithm - Weather Example



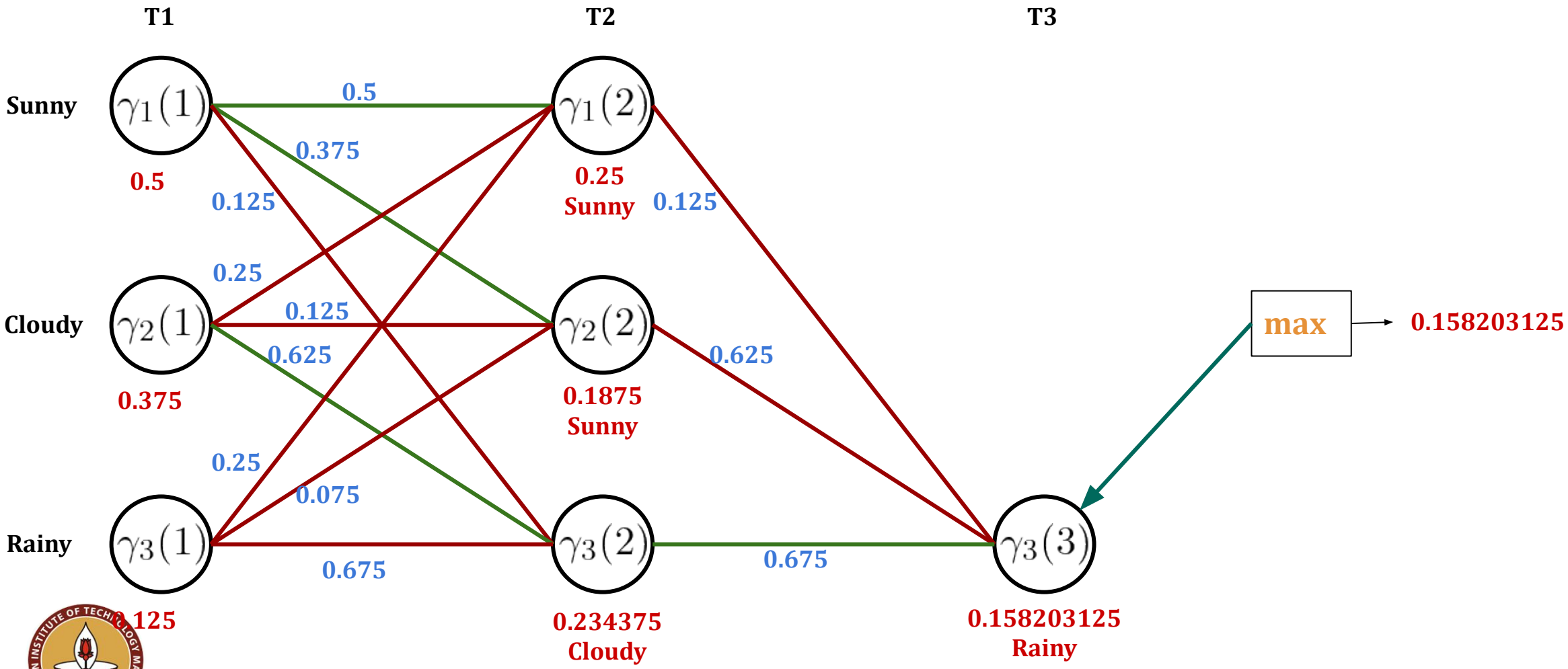
Viterbi Algorithm - Weather Example



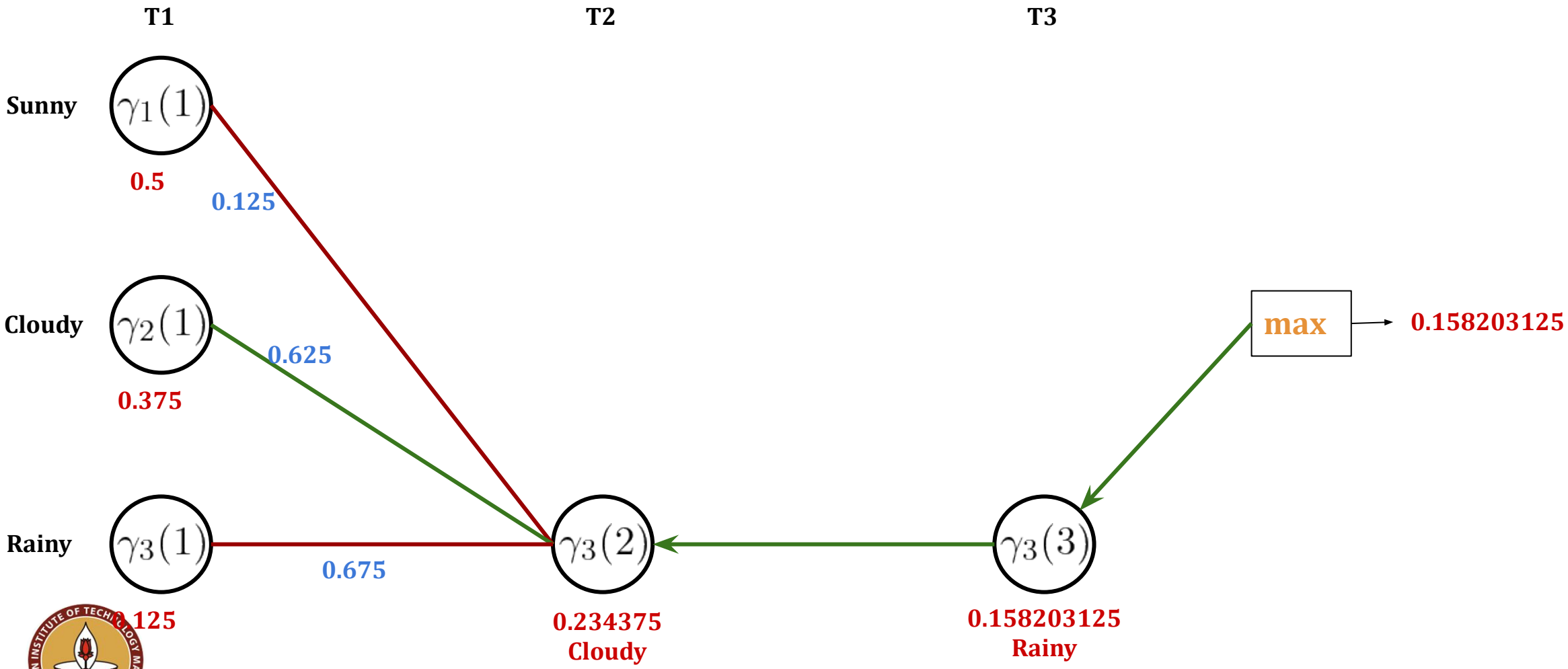
Viterbi Algorithm - Weather Example



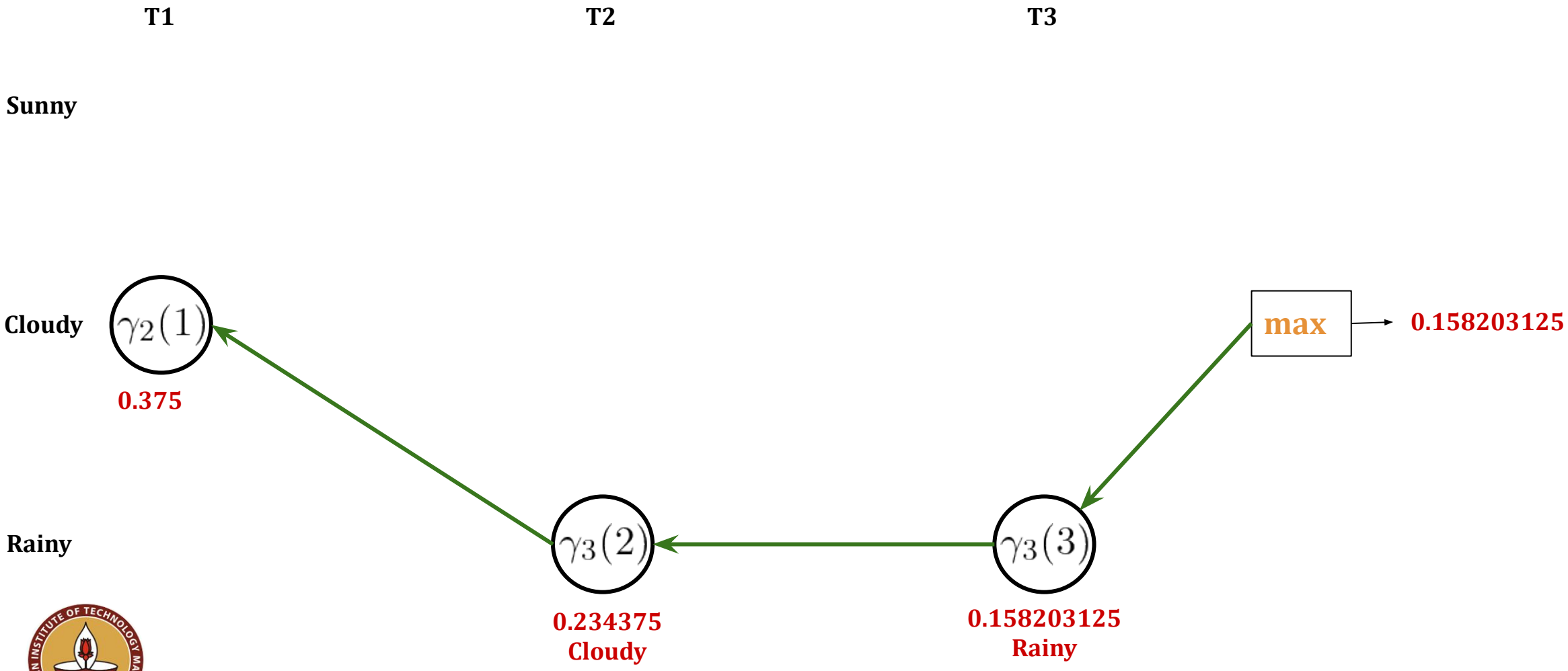
Viterbi Algorithm - Weather Example: Backtracking



Viterbi Algorithm - Weather Example: Backtracking



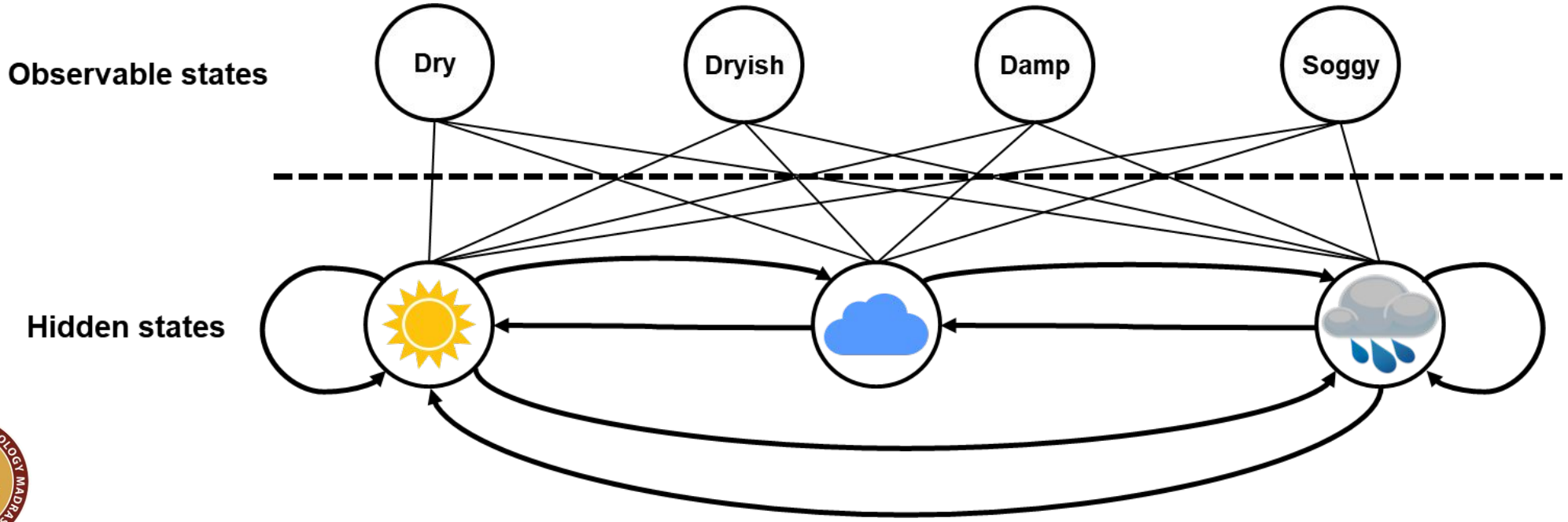
Viterbi Algorithm - Weather Example: Backtracking



Therefore, the optimal sequence is **Cloudy, Rainy, Rainy** and it is the same as the sequence we got using brute force approach

Hidden Markov Model (HMM)

- Let's assume we **can't directly observe** today's weather (i.e) **hidden**
- Rather we observe a **related phenomenon like dryness of grass**
- For example, state of grass can be one of **dry, dryish, damp, soggy**



Hidden Markov Model

Grass

Weather	Dry	Dryish	Damp	Soggy
Sunny	0.60	0.20	0.15	0.05
Cloudy	0.25	0.25	0.25	0.25
Rainy	0.05	0.10	0.35	0.50

- This matrix gives the probability distribution of observations given the weather condition.
- Observation probability distribution is **discrete** (e.g) P(grass is damp|sunny) = 0.15
- Two pieces of information
  - Weather condition the previous day – through transition probability
  - Condition of the grass today – observation probability



Hidden Markov Model

Grass

Weather	Dry	Dryish	Damp	Soggy
Sunny	0.60	0.20	0.15	0.05
Cloudy	0.25	0.25	0.25	0.25
Rainy	0.05	0.10	0.35	0.50

- Given observation for 5 days as **Dry, Dry, Soggy, Dampy, Soggy**
- What is the **most probable sequence of weather**?
- **Use Viterbi Algorithm**
- Viterbi Algorithm for HMM is similar to the one we used for Markov Chain except that HMM has **observation probabilities in addition** to state transition probabilities



Hidden Markov Model

- Given observation for 5 days as Dry, Dry, Soggy, Dampy, Soggy**, which of the below sequences is most probable? (assuming initial cloudy)**
- Sunny, Sunny, Cloudy, Sunny, Sunny Vs. Rainy, Rainy, Cloudy, Cloudy, Rainy

P(Sunny, Sunny, Cloudy, Sunny, Sunny | Dry, Dry, Soggy, Dampy, Soggy)

= P(Dry, Dry, Soggy, Dampy, Soggy | Sunny, Sunny, Cloudy, Sunny, Sunny) * P(Sunny, Sunny, Cloudy, Sunny, Sunny)

= P(Dry|Sunny) P(Dry|Sunny) P(Soggy|Cloudy) P(Dampy|Sunny) P(Soggy|Sunny) P(Sunny|Cloudy) P(Sunny|Sunny) P(Cloudy|Sunny) P(Sunny|Cloudy) P(Sunny|Sunny)

Hidden Markov Model

= 0.6 * 0.6 * 0.25 * 0.15 * 0.05 * 0.25 * 0.5 * 0.375 * 0.25 * 0.5 = **39.551 * e-7**

P(Rainy, Rainy, Cloudy, Cloudy, Rainy | Dry, Dry, Soggy, Dampy, Soggy)

= P(Dry, Dry, Soggy, Dampy, Soggy | Rainy, Rainy, Cloudy, Cloudy, Rainy) * P(Rainy, Rainy, Cloudy, Cloudy, Rainy)

= P(Dry|Rainy) P(Dry|Rainy) P(Soggy|Cloudy) P(Dampy|Cloudy) P(Soggy|Rainy) P(Rainy|Cloudy) P(Rainy|Rainy) P(Cloudy|Rainy) P(Cloudy|Cloudy) P(Rainy|Cloudy)

= 0.05 * 0.05 * 0.25 * 0.25 * 0.5 * 0.625 * 0.675 * 0.075 * 0.125 * 0.625 = **1.931 * e-7**

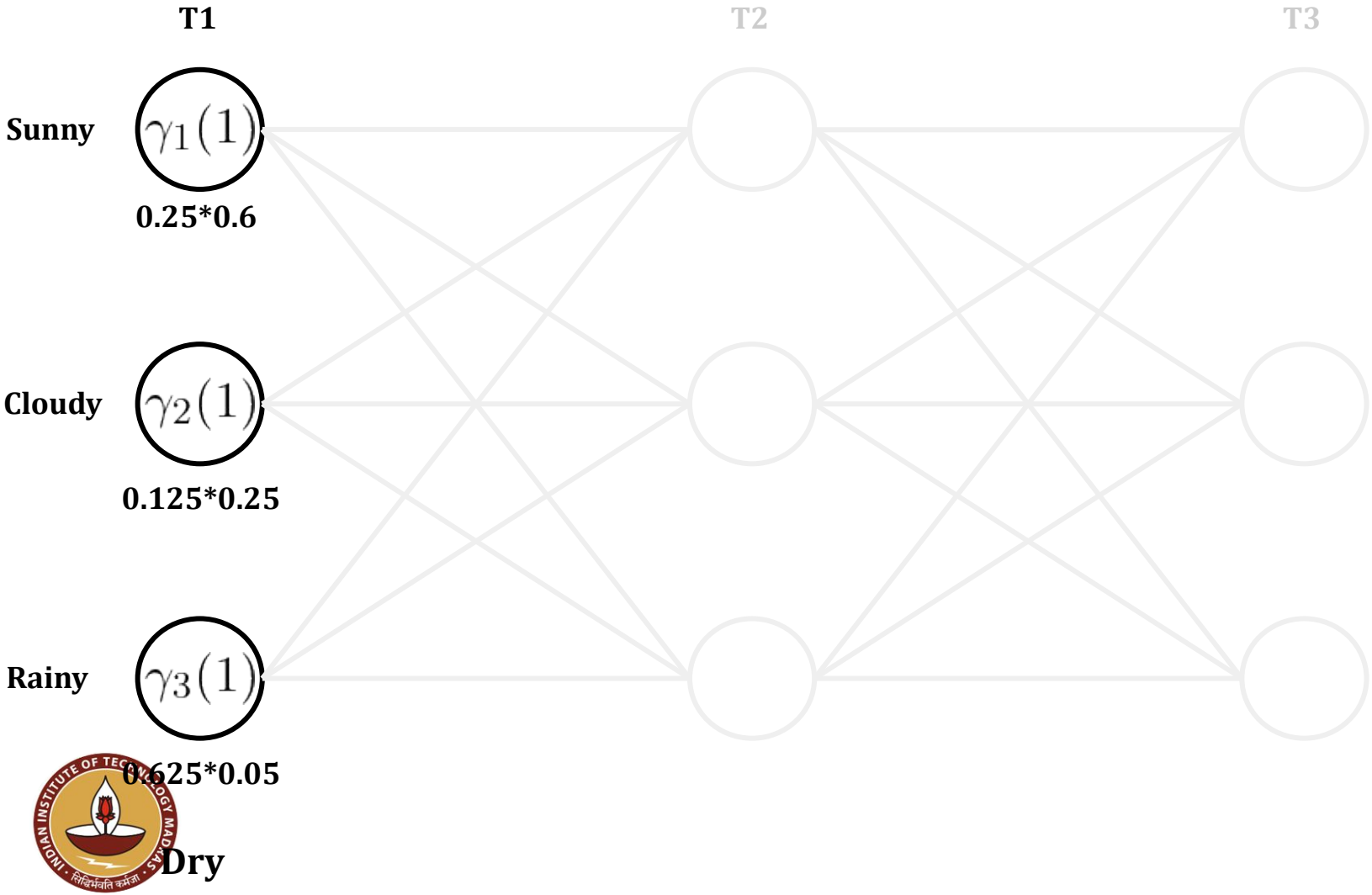
Today

Yesterday	Sunny	Cloudy	Rainy
Sunny	0.5	0.375	0.125
Cloudy	0.25	0.125	0.625
Rainy	0.25	0.075	0.675

Grass

Weather	Dry	Dryish	Damp	Soggy
Sunny	0.60	0.20	0.15	0.05
Cloudy	0.25	0.25	0.25	0.25
Rainy	0.05	0.10	0.35	0.50

Viterbi Algorithm For HMM - Weather Example



- Given that **Day 0 is Cloudy**, and **observed sequence is Dry, Dry, Soggy**, what is the best possible state sequence?
- We have to use both the state transition and observation probabilities.

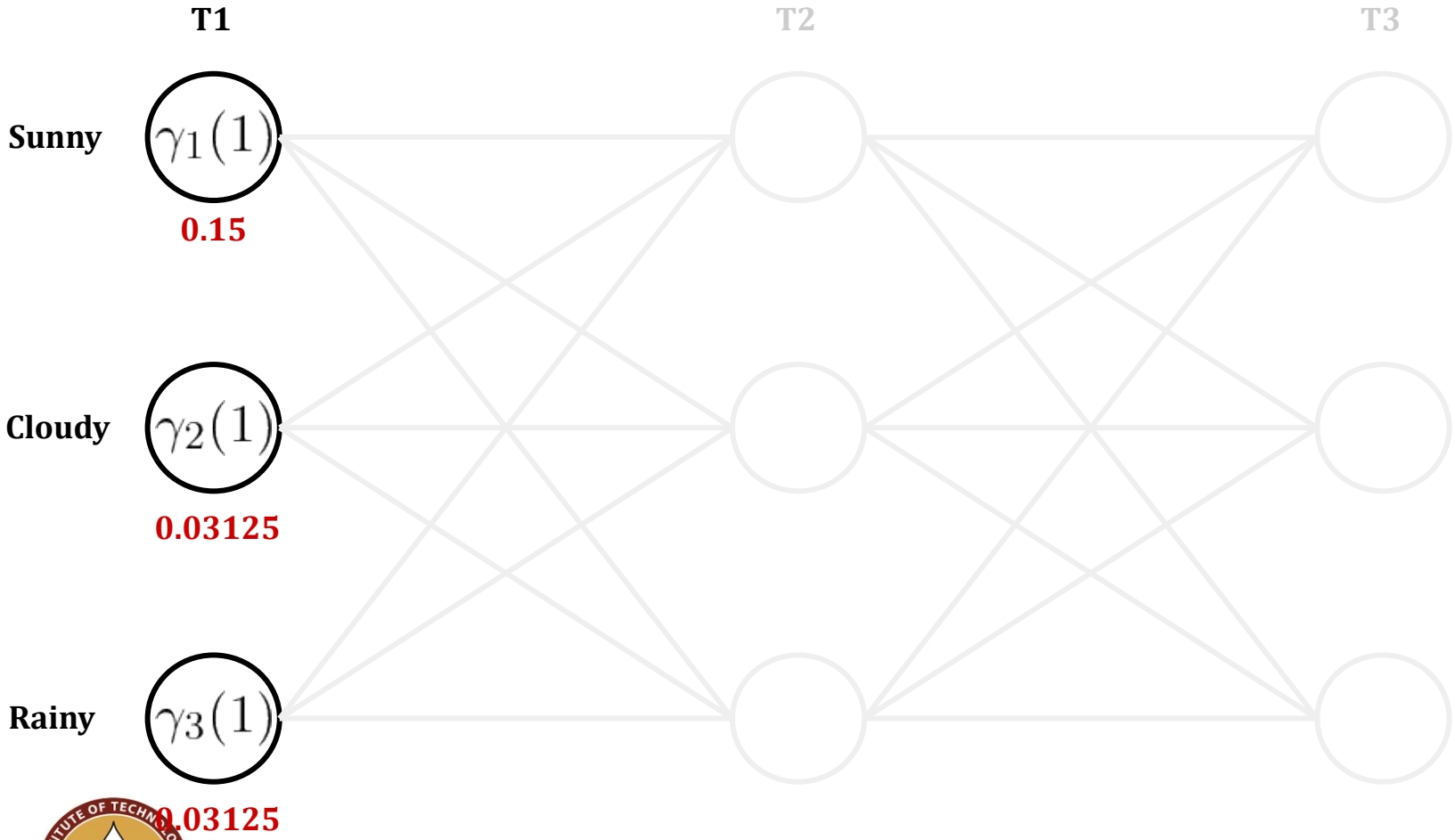
Grass

Weather	Dry	Dryish	Damp	Soggy
Sunny	0.60	0.20	0.15	0.05
Cloudy	0.25	0.25	0.25	0.25
Rainy	0.05	0.10	0.35	0.50

Today

Yesterday	Sunny	Cloudy	Rainy
Sunny	0.5	0.375	0.125
Cloudy	0.25	0.125	0.625
Rainy	0.25	0.075	0.675

Viterbi Algorithm For HMM - Weather Example



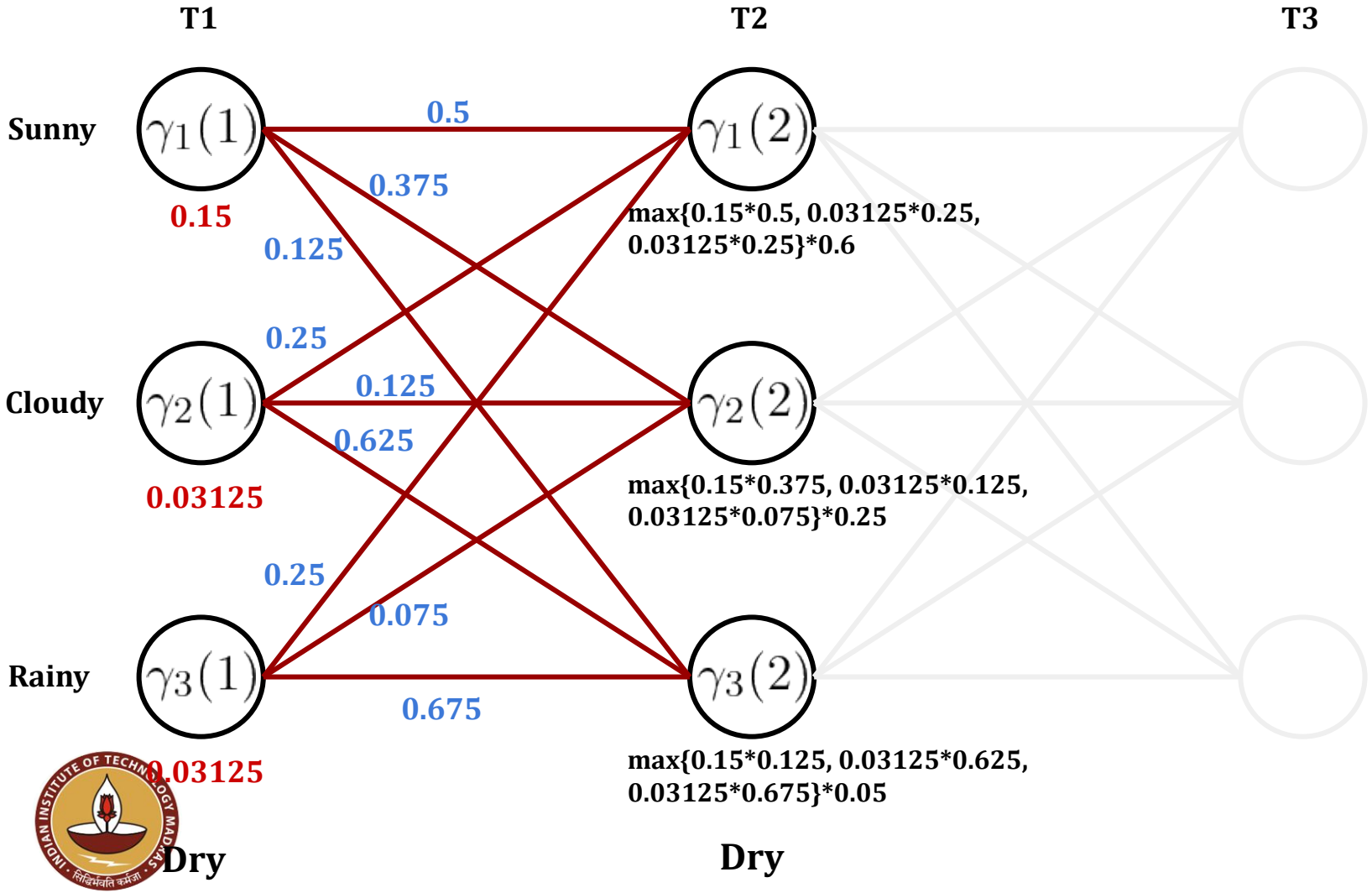
Grass

Weather	Dry	Dryish	Damp	Soggy
Sunny	0.60	0.20	0.15	0.05
Cloudy	0.25	0.25	0.25	0.25
Rainy	0.05	0.10	0.35	0.50

Today

Yesterday	Sunny	Cloudy	Rainy
Sunny	0.5	0.375	0.125
Cloudy	0.25	0.125	0.625
Rainy	0.25	0.075	0.675

Viterbi Algorithm For HMM - Weather Example



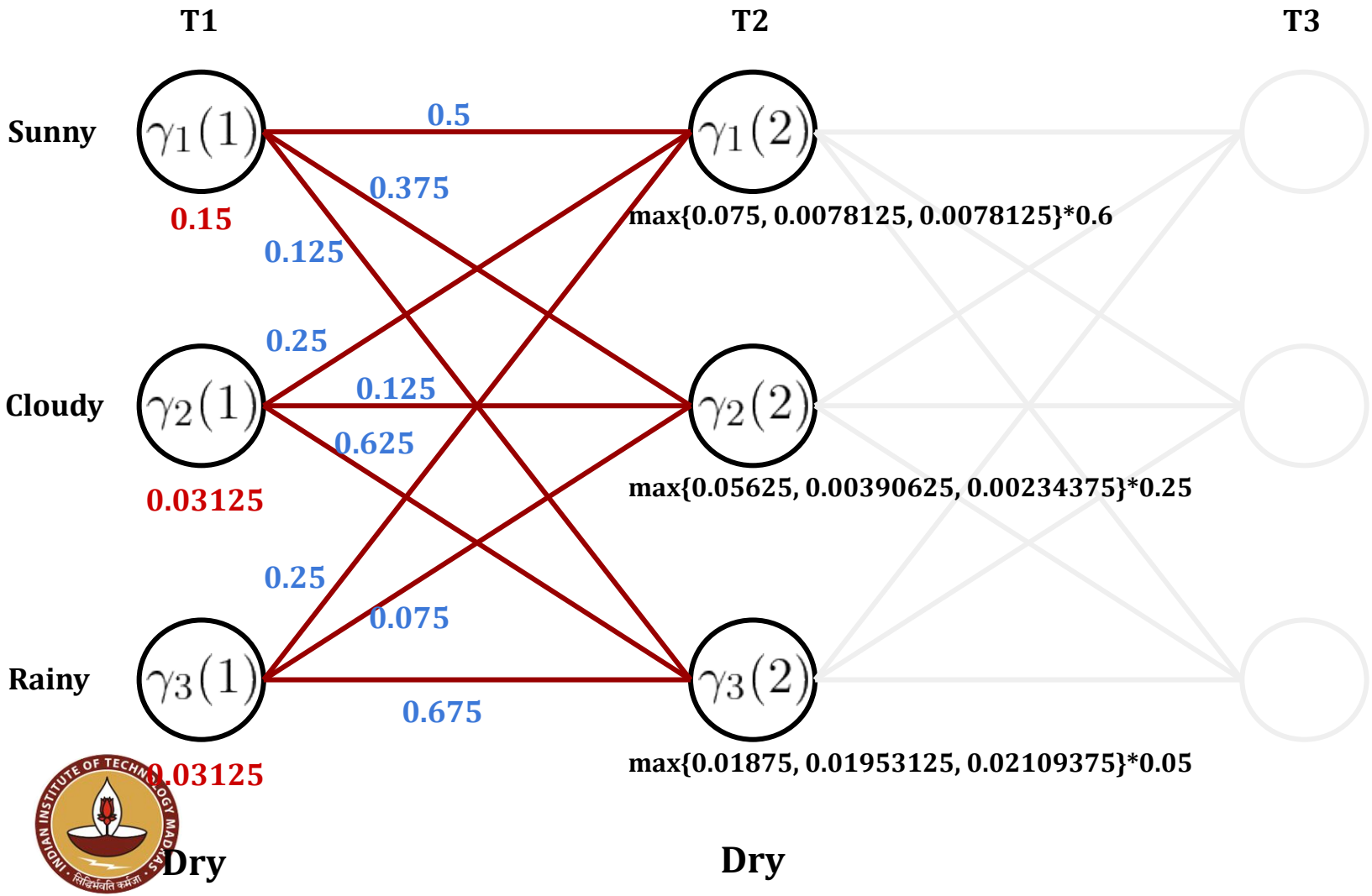
Grass

Weather	Dry	Dryish	Damp	Soggy
Sunny	0.60	0.20	0.15	0.05
Cloudy	0.25	0.25	0.25	0.25
Rainy	0.05	0.10	0.35	0.50

Today

Yesterday	Sunny	Cloudy	Rainy
Sunny	0.5	0.375	0.125
Cloudy	0.25	0.125	0.625
Rainy	0.25	0.075	0.675

Viterbi Algorithm For HMM - Weather Example



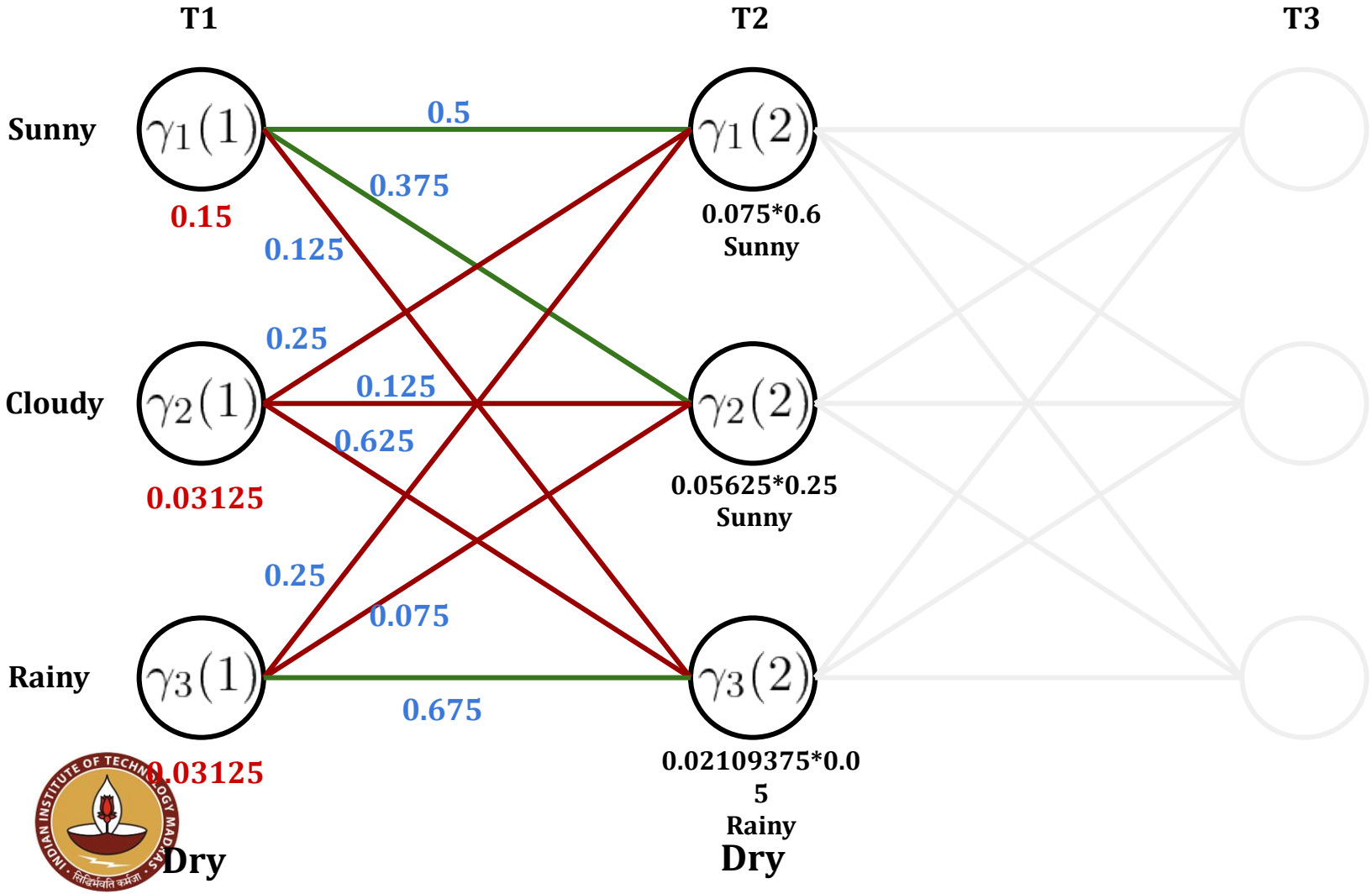
Grass

Weather	Dry	Dryish	Damp	Soggy
Sunny	0.60	0.20	0.15	0.05
Cloudy	0.25	0.25	0.25	0.25
Rainy	0.05	0.10	0.35	0.50

Today

Yesterday	Sunny	Cloudy	Rainy
Sunny	0.5	0.375	0.125
Cloudy	0.25	0.125	0.625
Rainy	0.25	0.075	0.675

Viterbi Algorithm For HMM - Weather Example



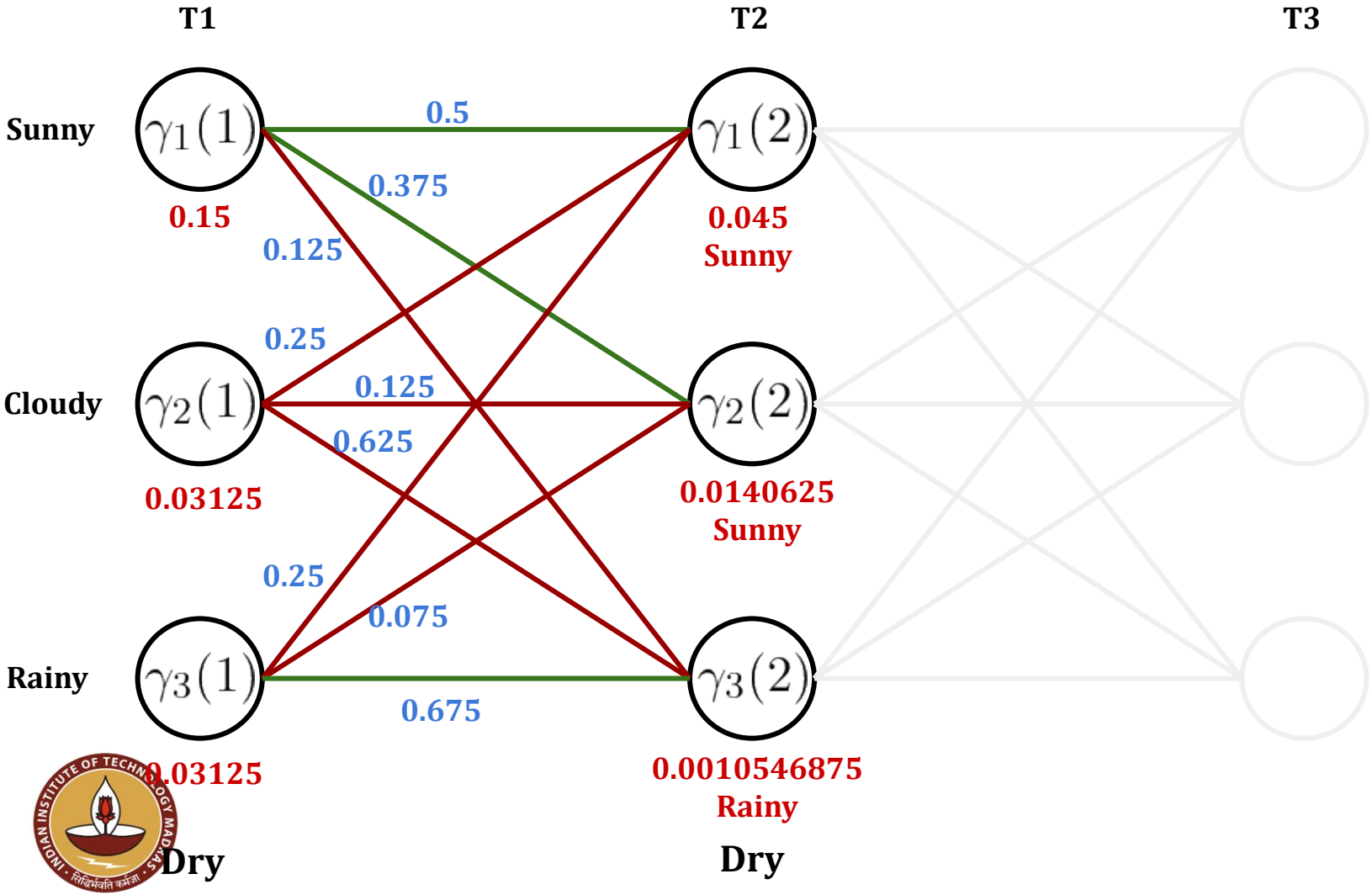
Grass

Weather	Dry	Dryish	Damp	Soggy
Sunny	0.60	0.20	0.15	0.05
Cloudy	0.25	0.25	0.25	0.25
Rainy	0.05	0.10	0.35	0.50

Today

Yesterday	Sunny	Cloudy	Rainy
Sunny	0.5	0.375	0.125
Cloudy	0.25	0.125	0.625
Rainy	0.25	0.075	0.675

Viterbi Algorithm For HMM - Weather Example



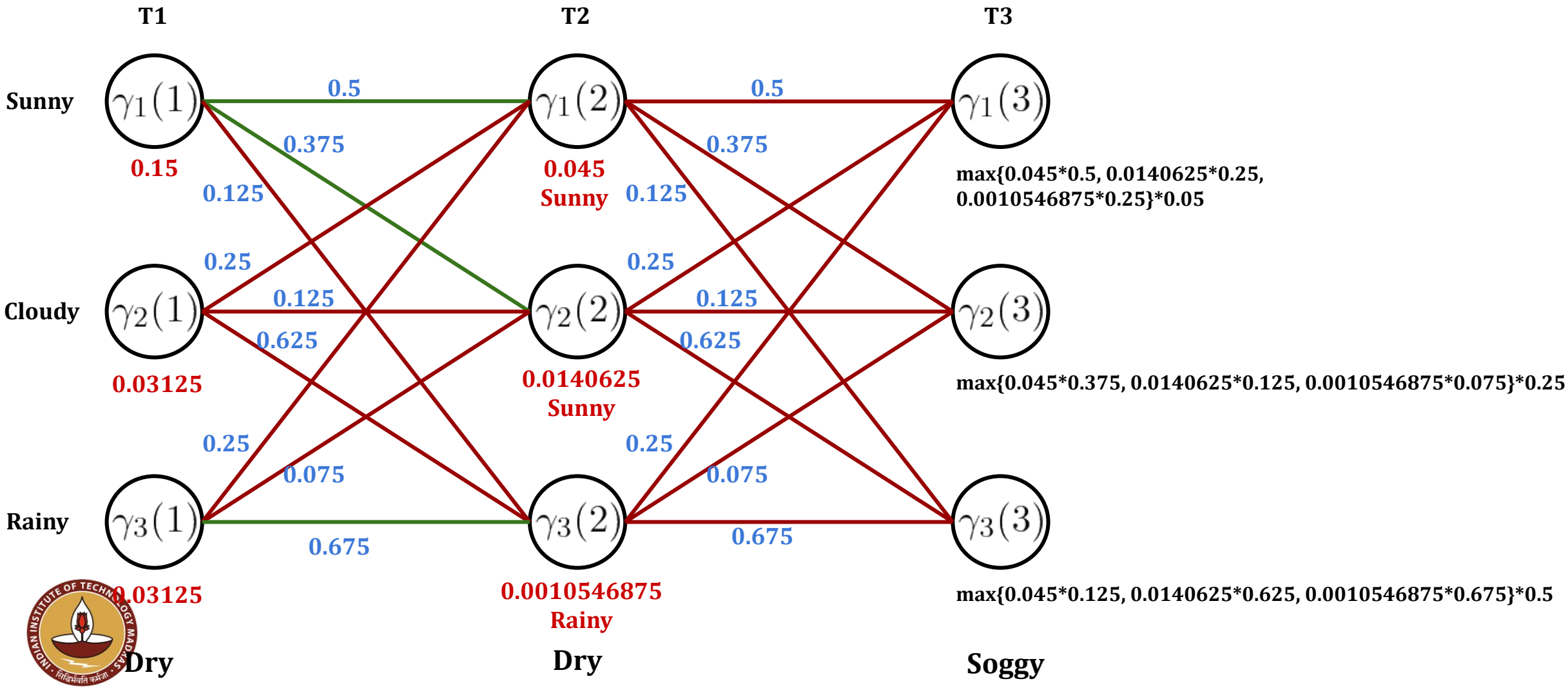
Grass

Weather	Dry	Dryish	Damp	Soggy
Sunny	0.60	0.20	0.15	0.05
Cloudy	0.25	0.25	0.25	0.25
Rainy	0.05	0.10	0.35	0.50

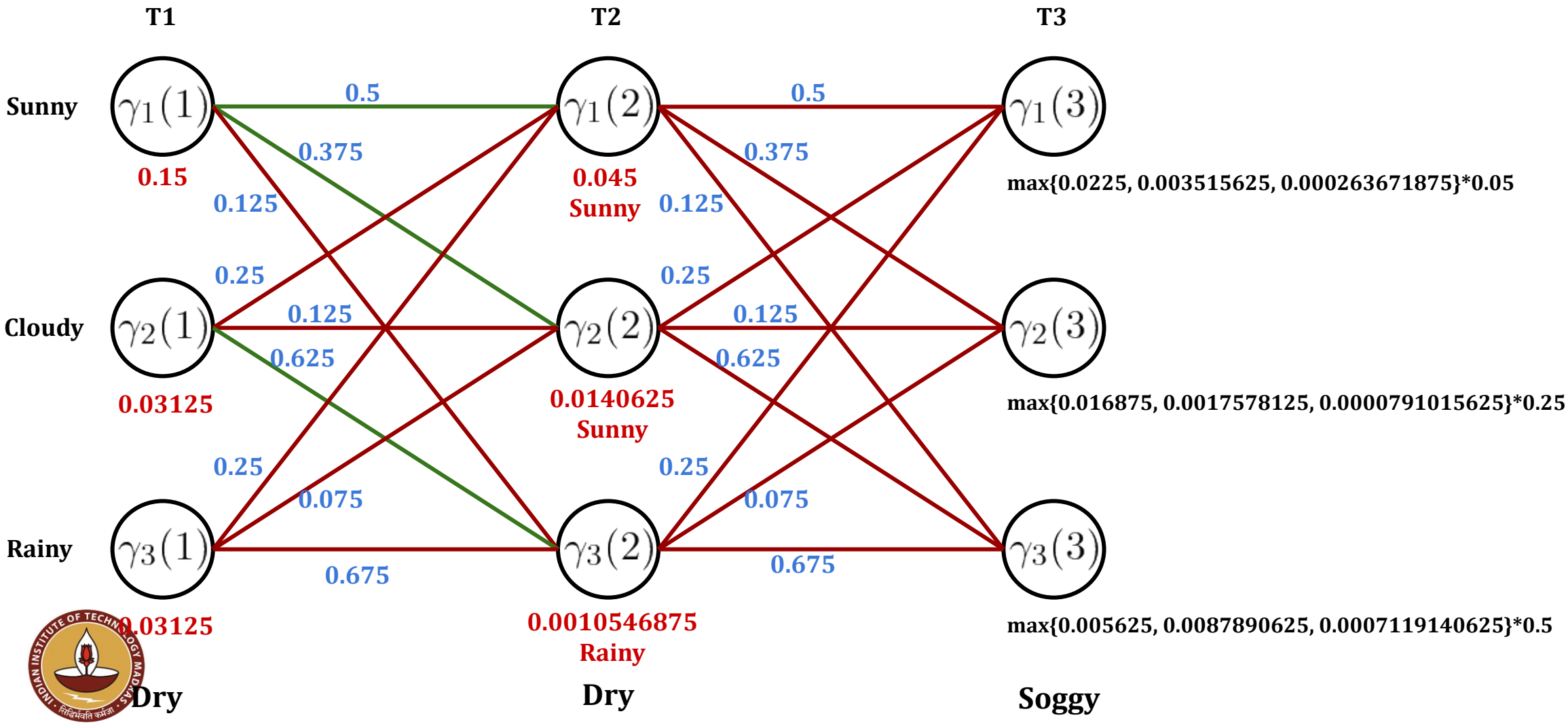
Today

Yesterday	Sunny	Cloudy	Rainy
Sunny	0.5	0.375	0.125
Cloudy	0.25	0.125	0.625
Rainy	0.25	0.075	0.675

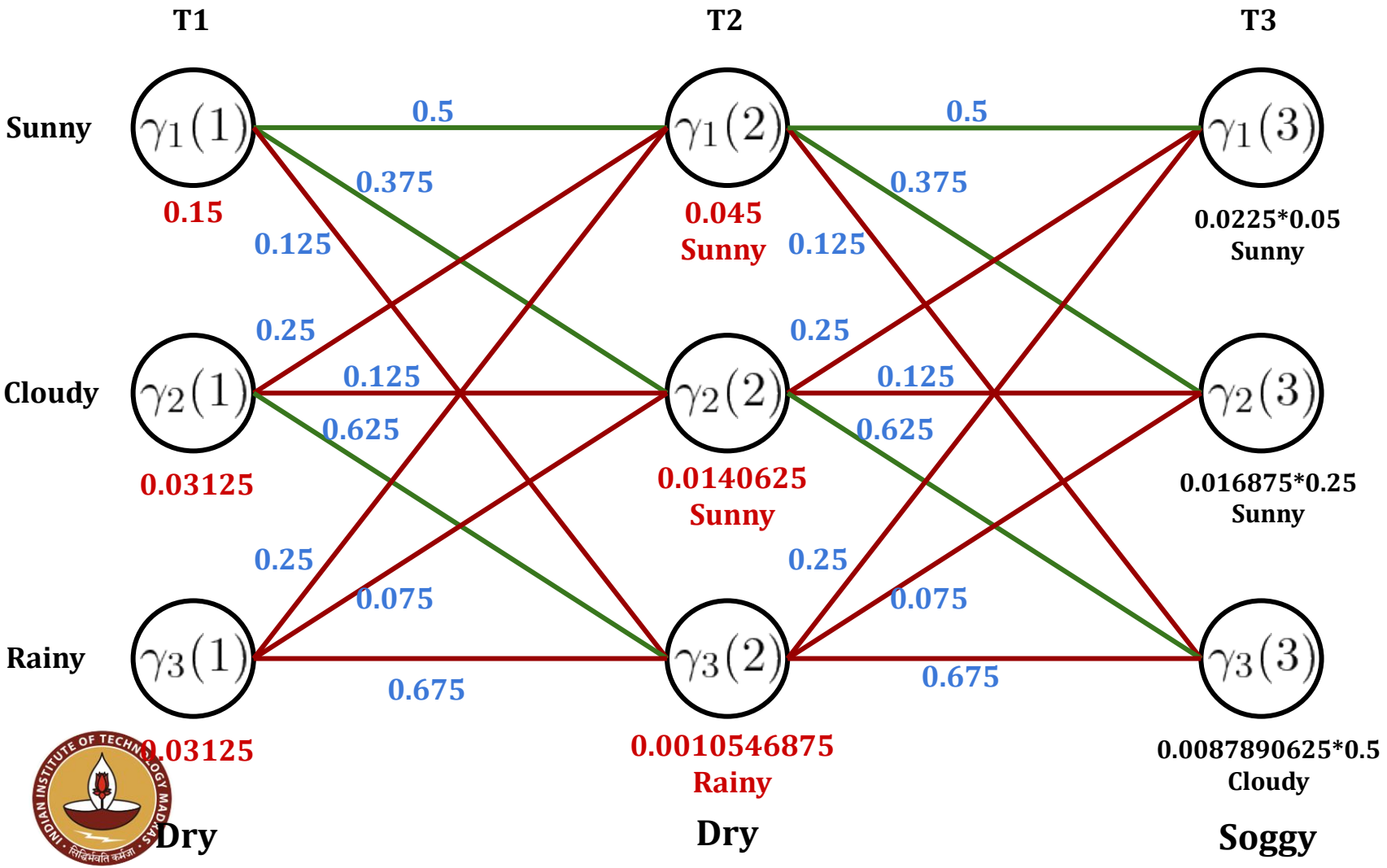
Viterbi Algorithm For HMM - Weather Example



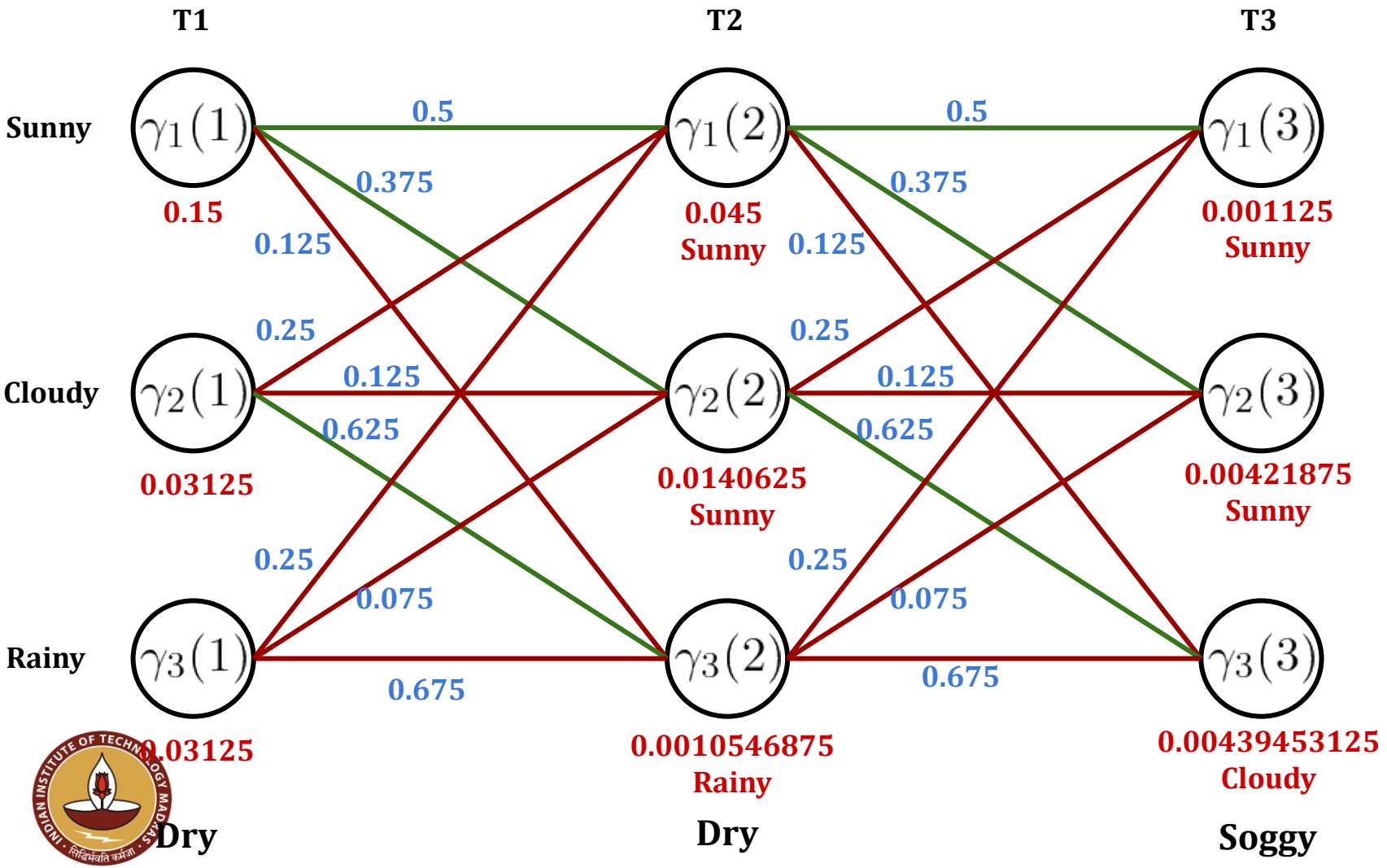
Viterbi Algorithm For HMM - Weather Example



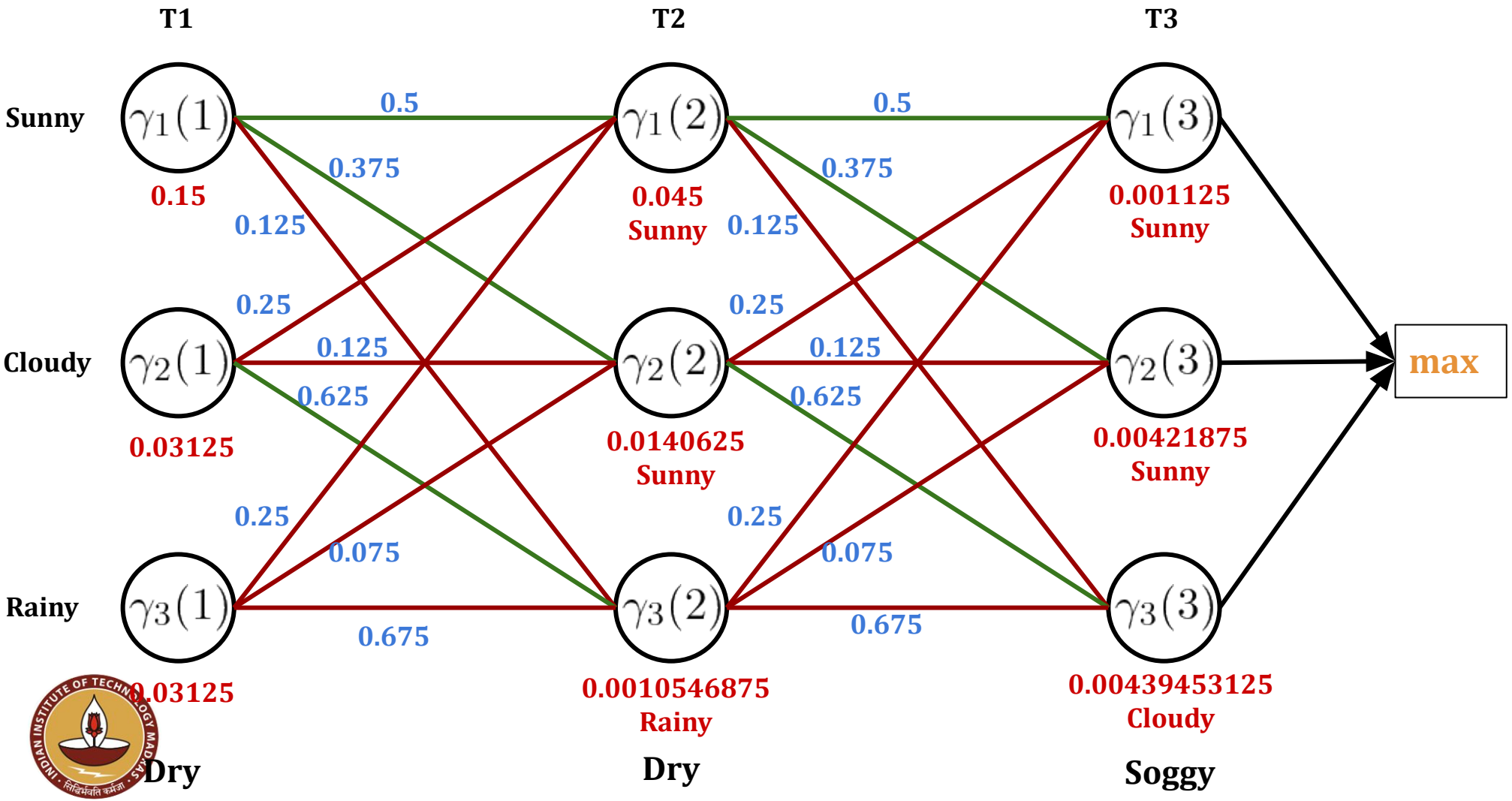
Viterbi Algorithm For HMM - Weather Example



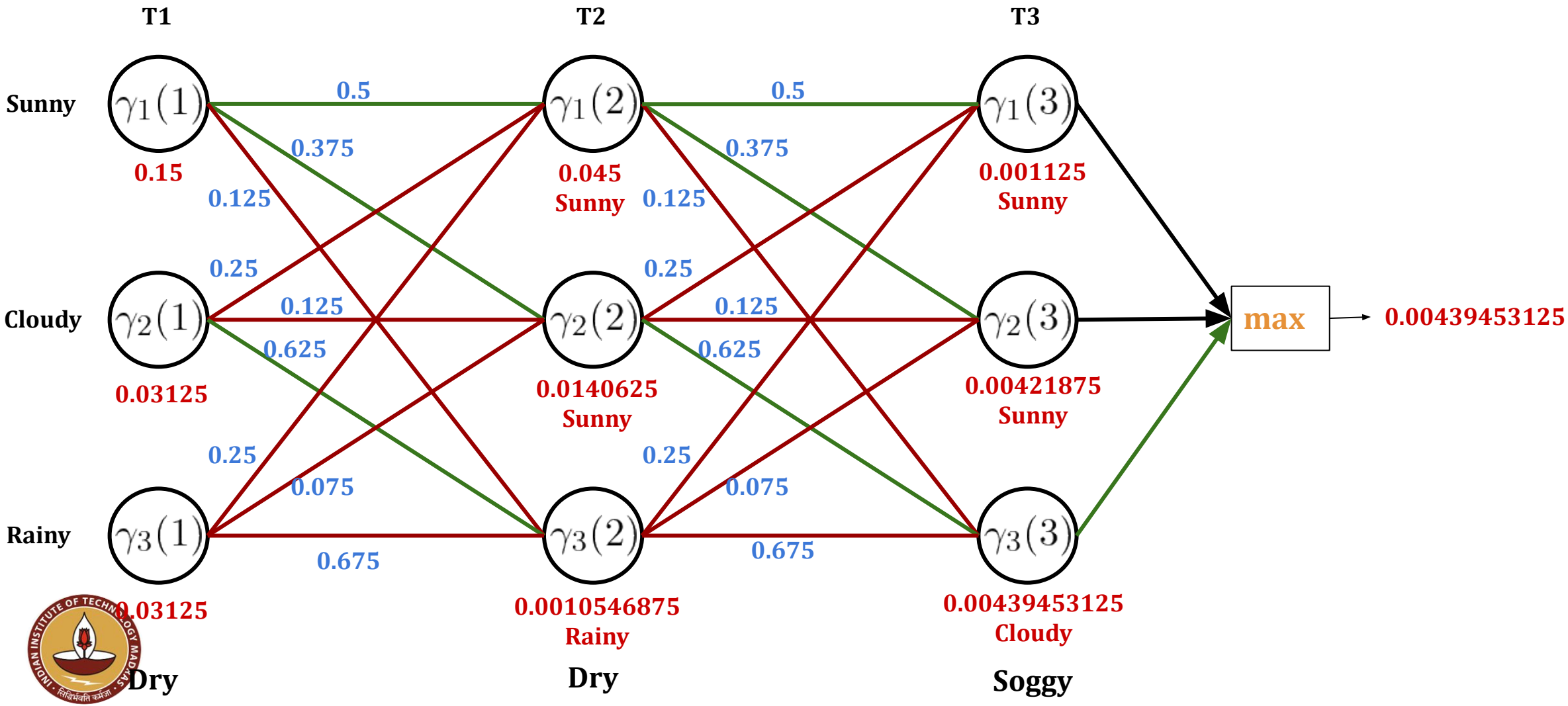
Viterbi Algorithm For HMM - Weather Example



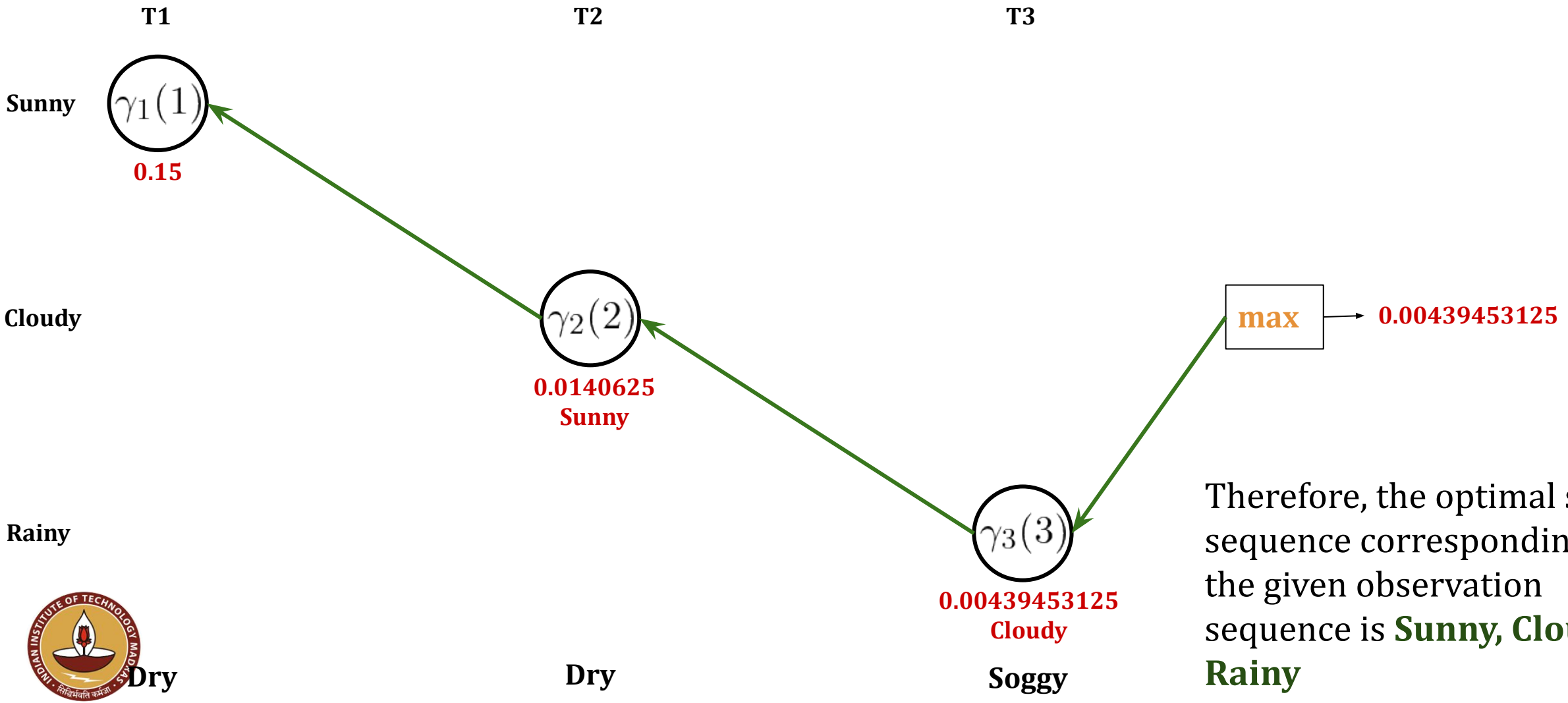
Viterbi Algorithm For HMM - Weather Example



Viterbi Algorithm For HMM - Weather Example

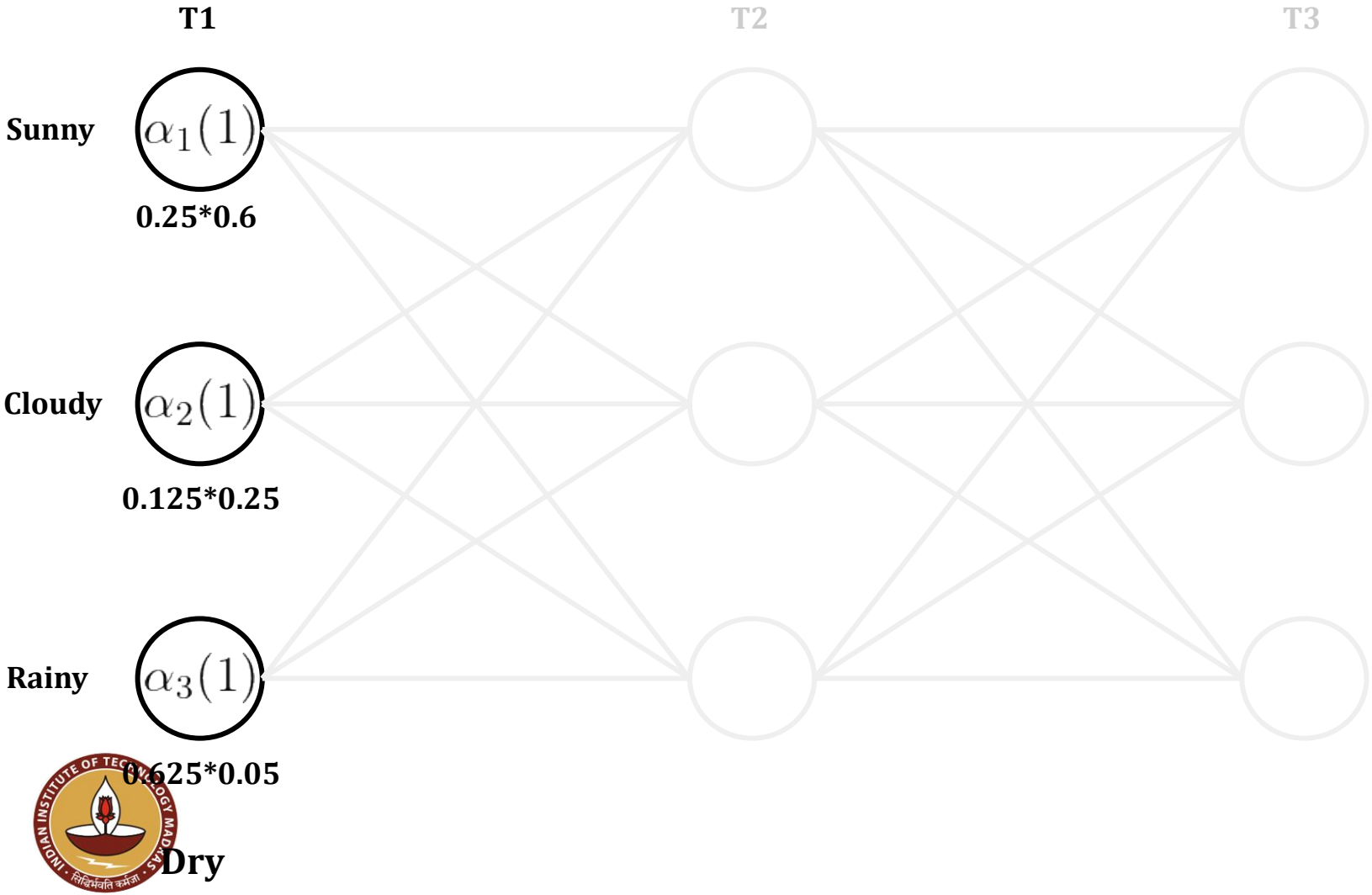


Viterbi Algorithm For HMM: Backtracking



Therefore, the optimal state sequence corresponding to the given observation sequence is **Sunny, Cloudy, Rainy**

Forward Algorithm For HMM - Weather Example



- Given that **Day 0 is Cloudy**, and **observed sequence is Dry, Dry, Soggy, what is the probability of this sequence given the model**?
- Similar to viterbi, but here **sum instead of max**

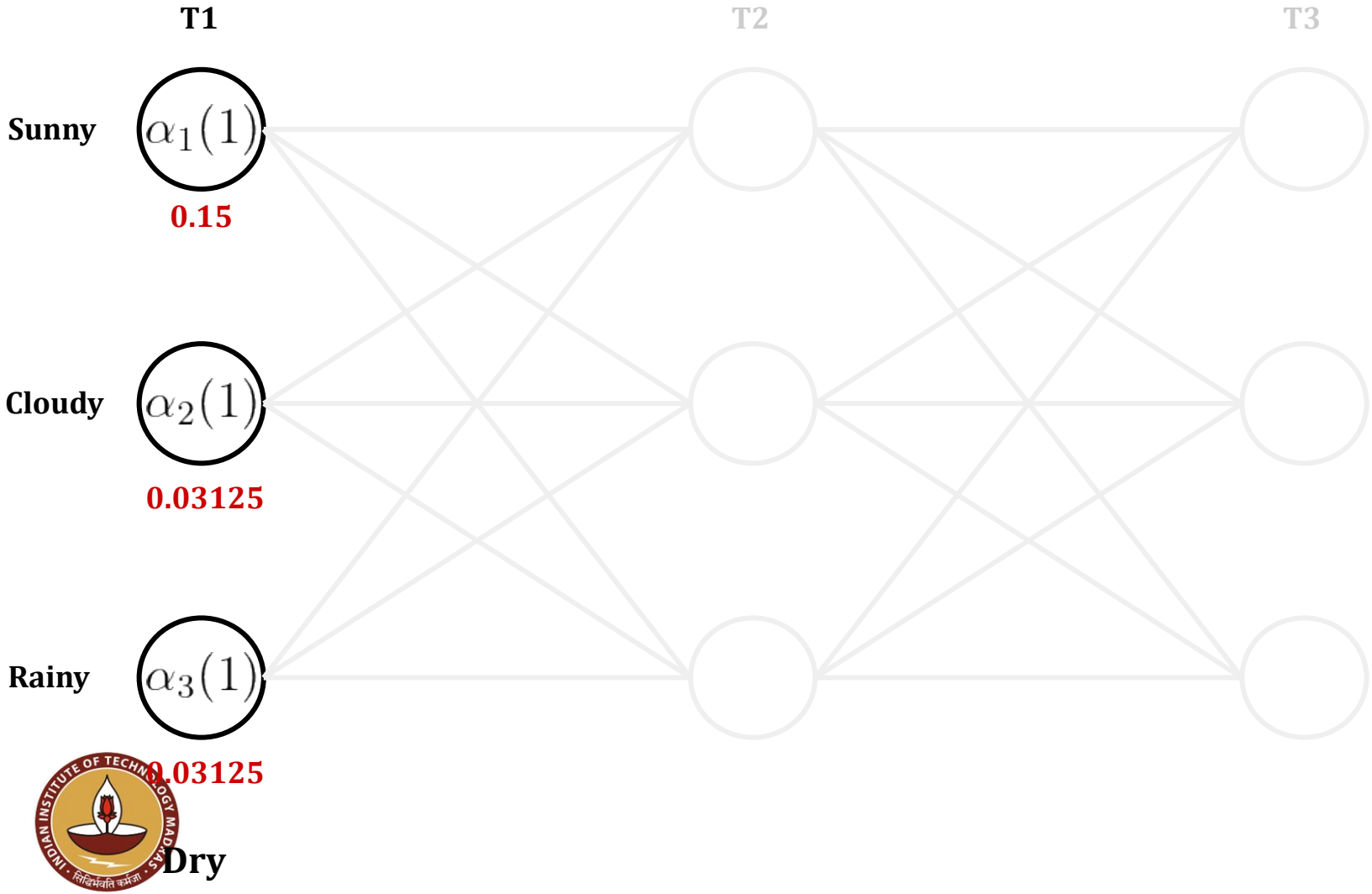
Grass

Weather	Dry	Dryish	Damp	Soggy
Sunny	0.60	0.20	0.15	0.05
Cloudy	0.25	0.25	0.25	0.25
Rainy	0.05	0.10	0.35	0.50

Today

Yesterday	Sunny	Cloudy	Rainy
Sunny	0.5	0.375	0.125
Cloudy	0.25	0.125	0.625
Rainy	0.25	0.075	0.675

Forward Algorithm For HMM - Weather Example



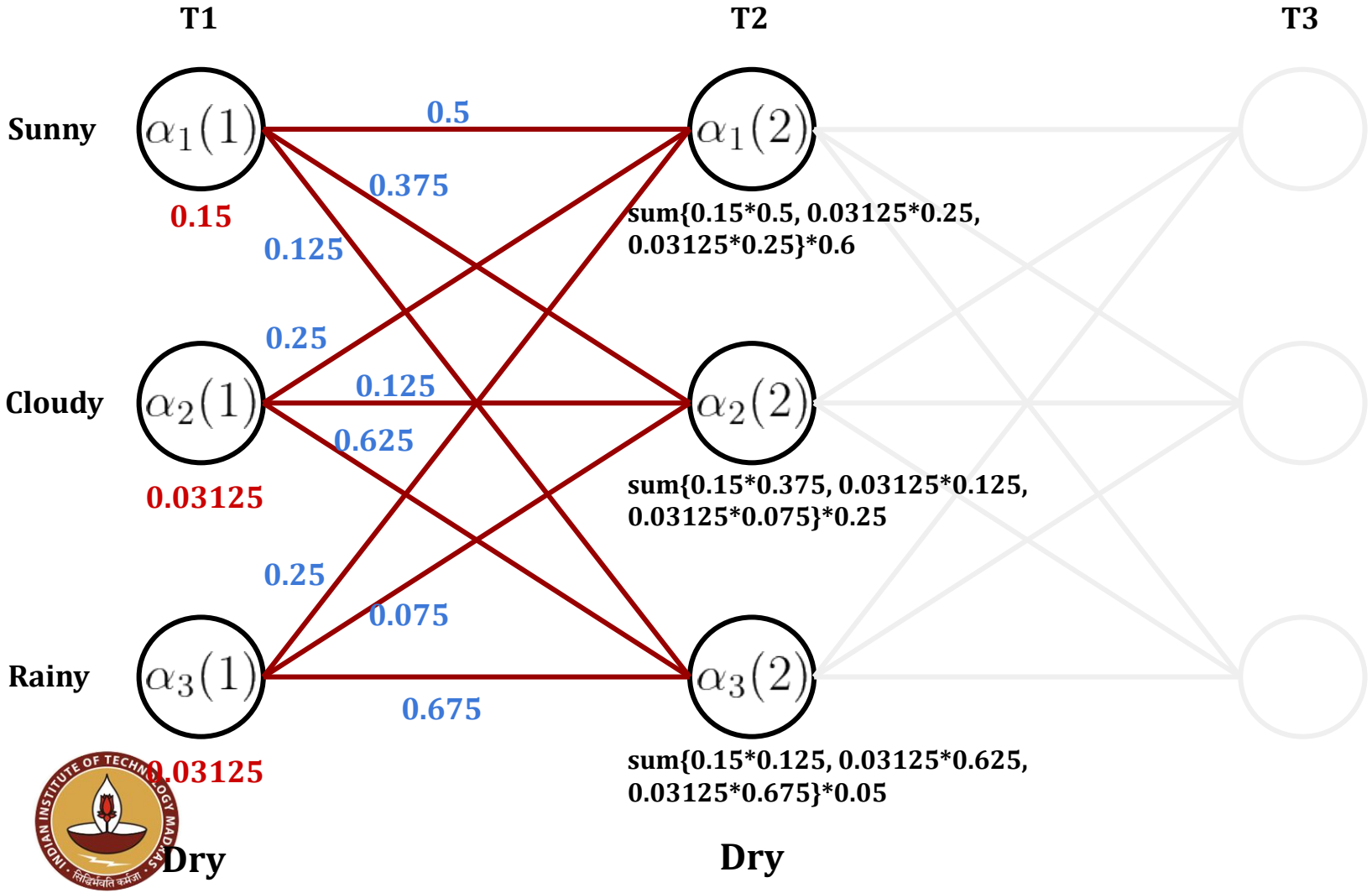
Grass

Weather	Dry	Dryish	Damp	Soggy
Sunny	0.60	0.20	0.15	0.05
Cloudy	0.25	0.25	0.25	0.25
Rainy	0.05	0.10	0.35	0.50

Today

Yesterday	Sunny	Cloudy	Rainy
Sunny	0.5	0.375	0.125
Cloudy	0.25	0.125	0.625
Rainy	0.25	0.075	0.675

Forward Algorithm For HMM - Weather Example



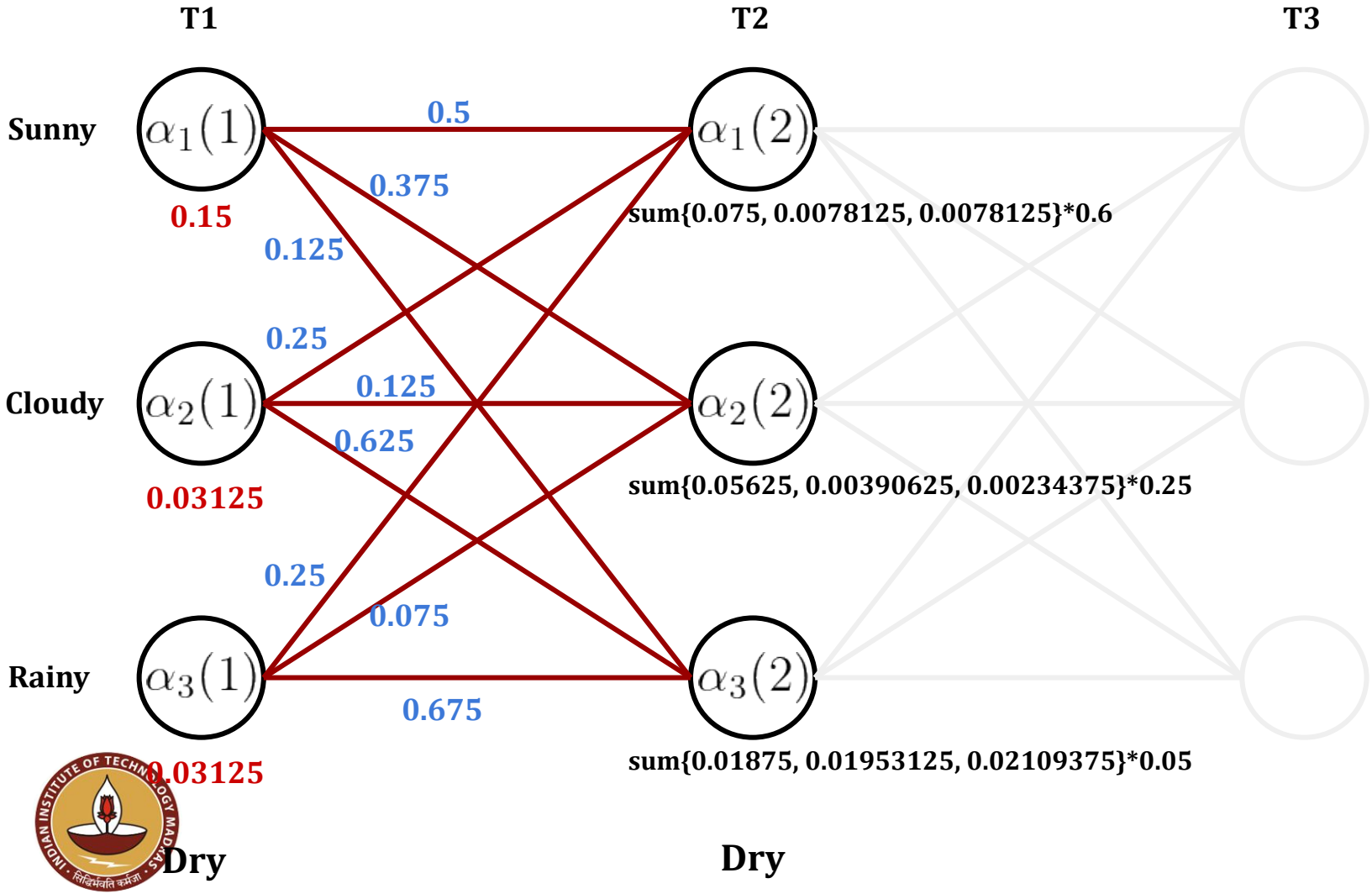
Grass

Weather	Dry	Dryish	Damp	Soggy
Sunny	0.60	0.20	0.15	0.05
Cloudy	0.25	0.25	0.25	0.25
Rainy	0.05	0.10	0.35	0.50

Today

Yesterday	Sunny	Cloudy	Rainy
Sunny	0.5	0.375	0.125
Cloudy	0.25	0.125	0.625
Rainy	0.25	0.075	0.675

Forward Algorithm For HMM - Weather Example



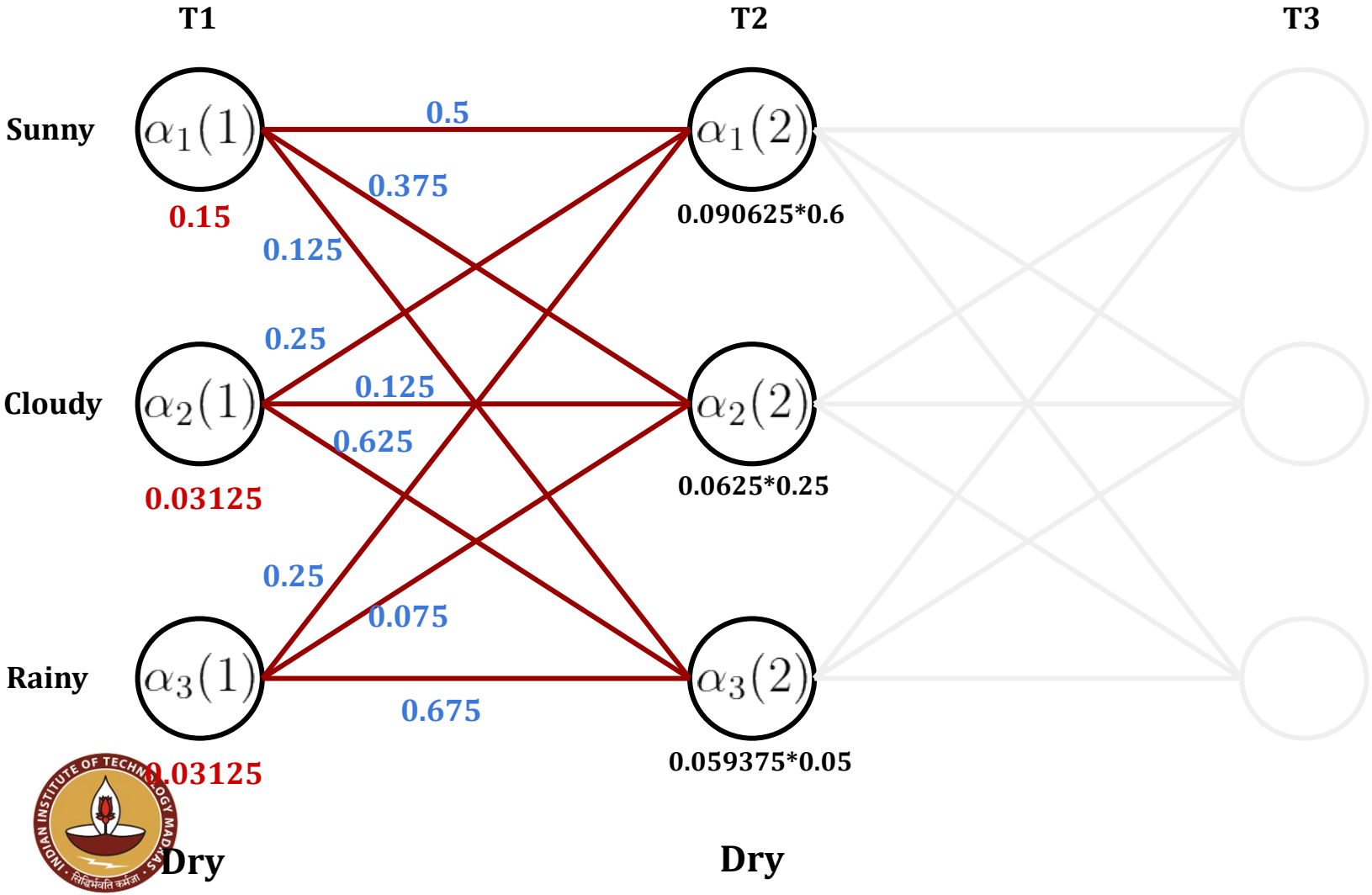
Grass

Weather	Dry	Dryish	Damp	Soggy
Sunny	0.60	0.20	0.15	0.05
Cloudy	0.25	0.25	0.25	0.25
Rainy	0.05	0.10	0.35	0.50

Today

Yesterday	Sunny	Cloudy	Rainy
Sunny	0.5	0.375	0.125
Cloudy	0.25	0.125	0.625
Rainy	0.25	0.075	0.675

Forward Algorithm For HMM - Weather Example



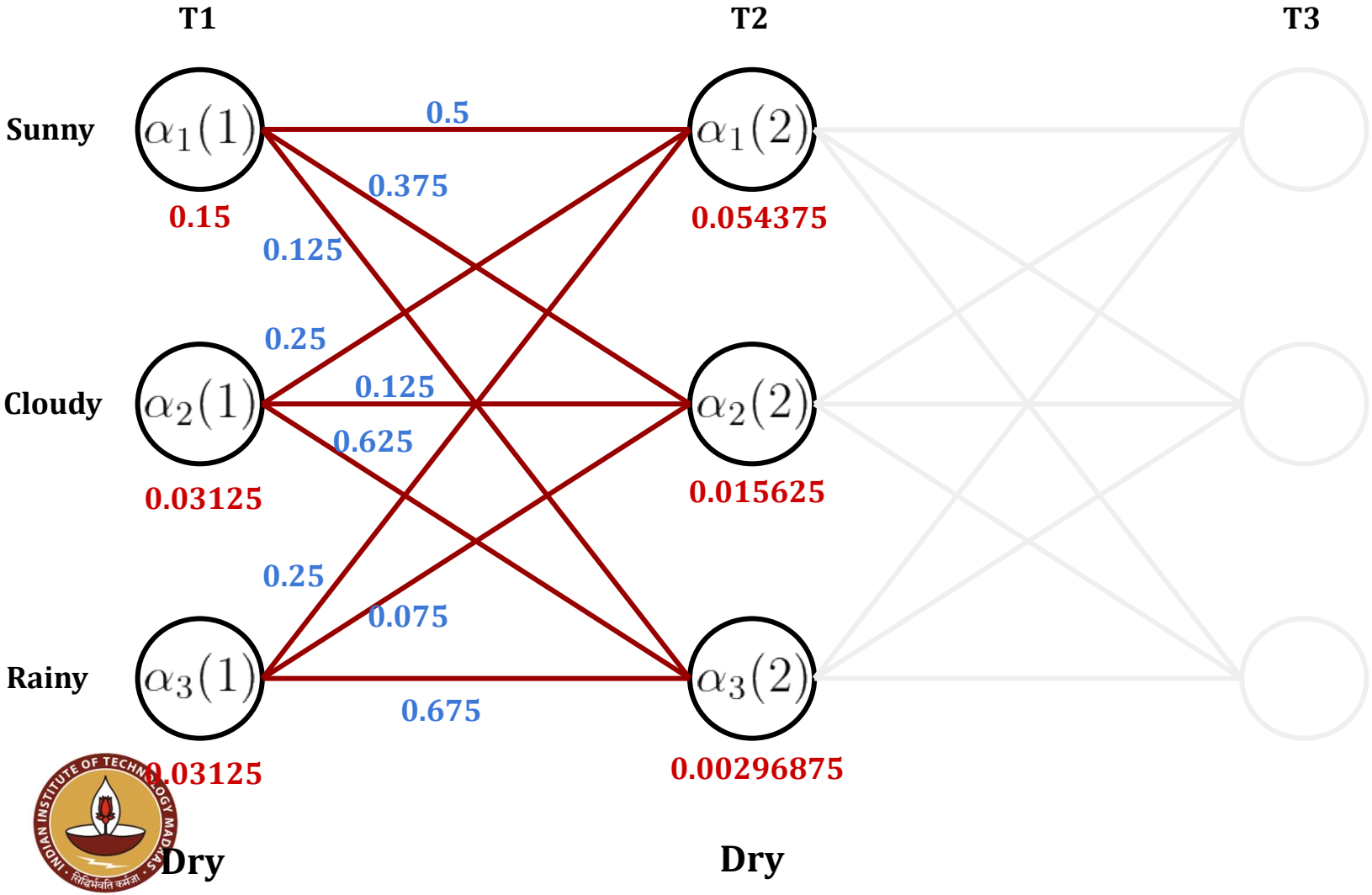
Grass

Weather	Dry	Dryish	Damp	Soggy
Sunny	0.60	0.20	0.15	0.05
Cloudy	0.25	0.25	0.25	0.25
Rainy	0.05	0.10	0.35	0.50

Today

Yesterday	Sunny	Cloudy	Rainy
Sunny	0.5	0.375	0.125
Cloudy	0.25	0.125	0.625
Rainy	0.25	0.075	0.675

Forward Algorithm For HMM - Weather Example



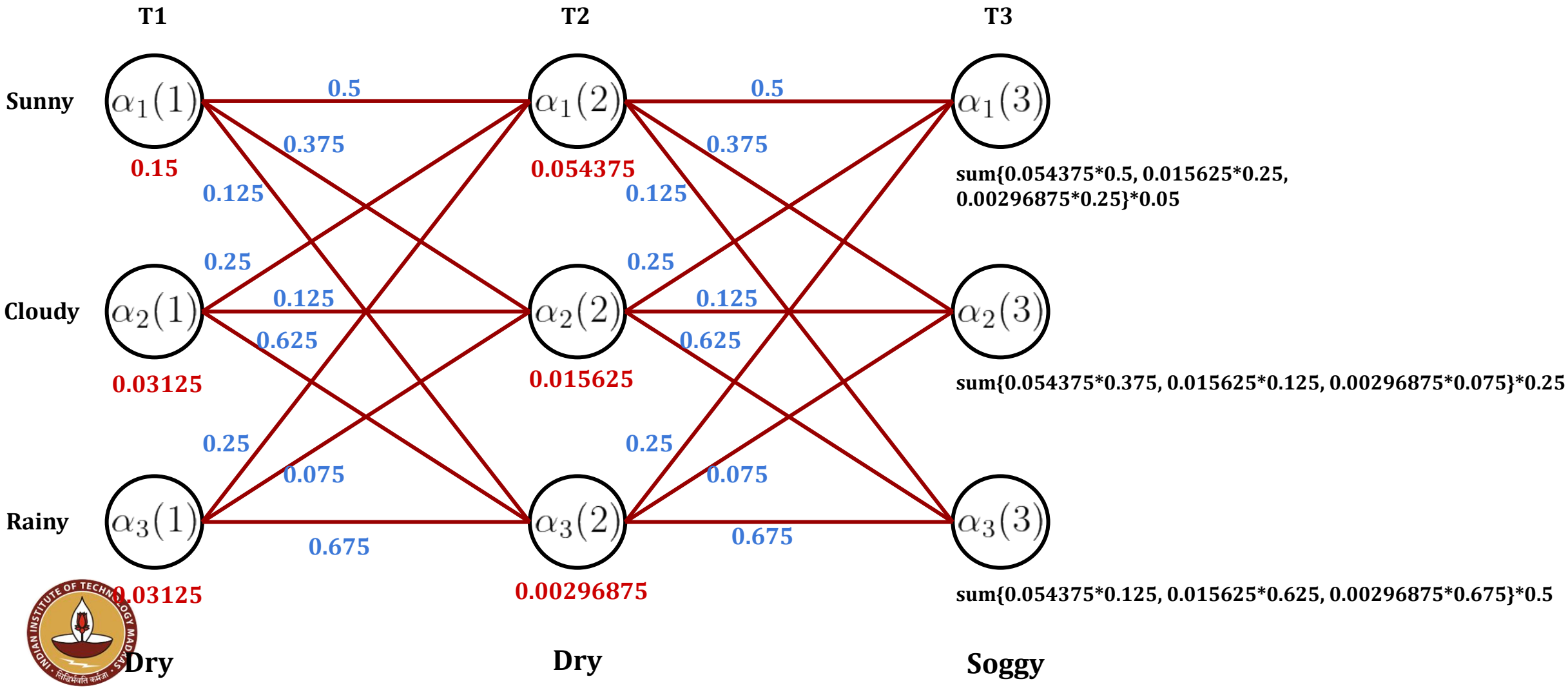
Grass

Weather	Dry	Dryish	Damp	Soggy
Sunny	0.60	0.20	0.15	0.05
Cloudy	0.25	0.25	0.25	0.25
Rainy	0.05	0.10	0.35	0.50

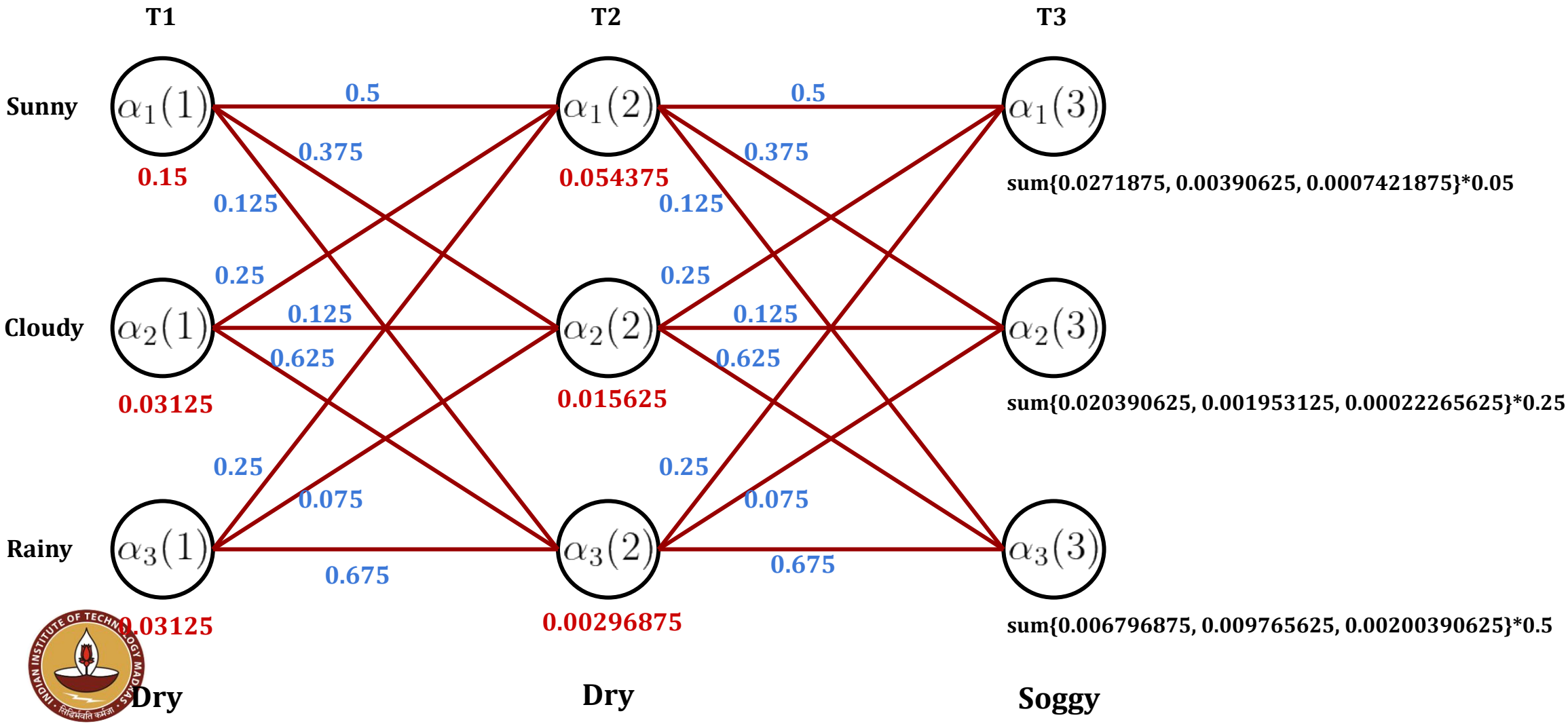
Today

Yesterday	Sunny	Cloudy	Rainy
Sunny	0.5	0.375	0.125
Cloudy	0.25	0.125	0.625
Rainy	0.25	0.075	0.675

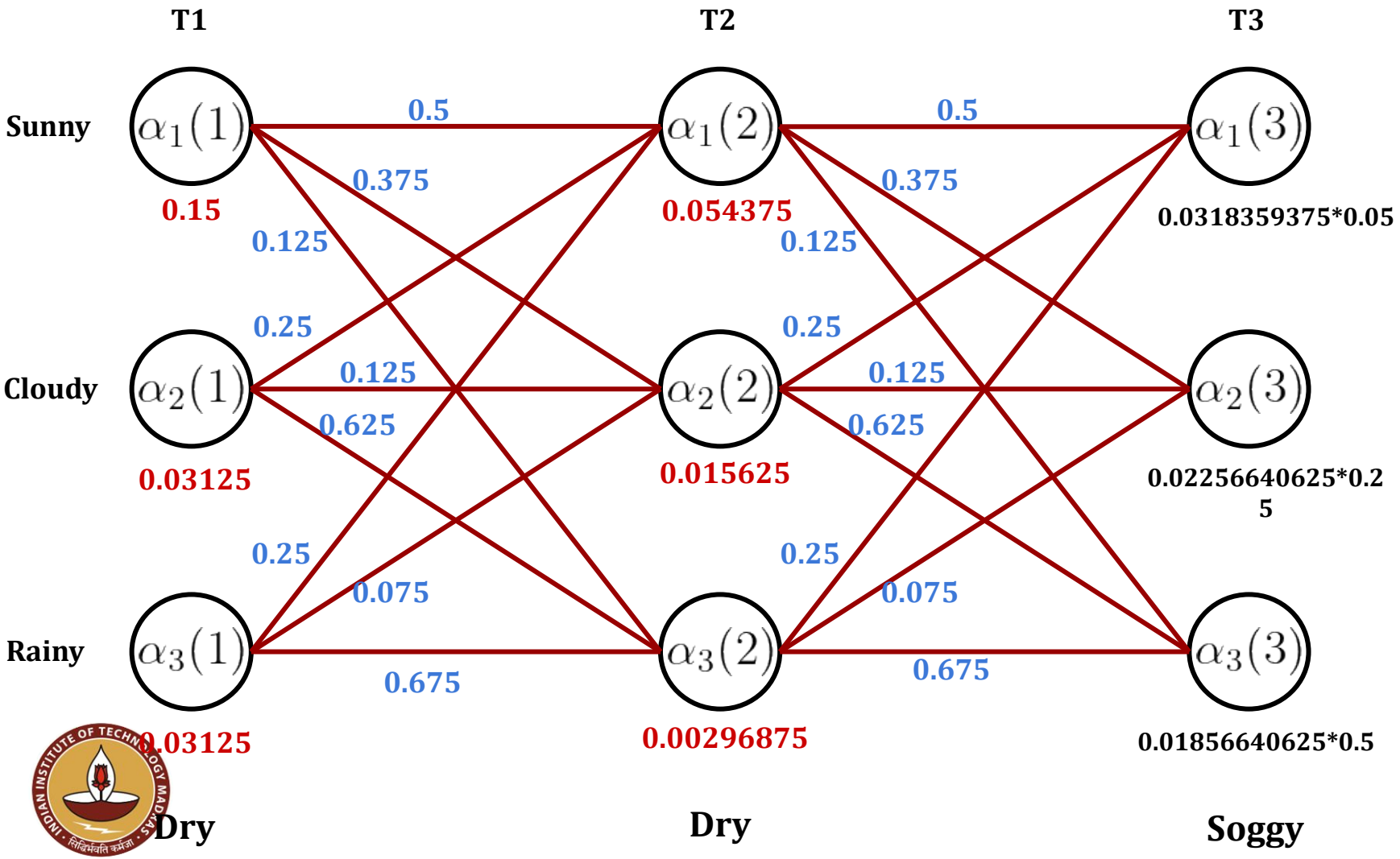
Forward Algorithm For HMM - Weather Example



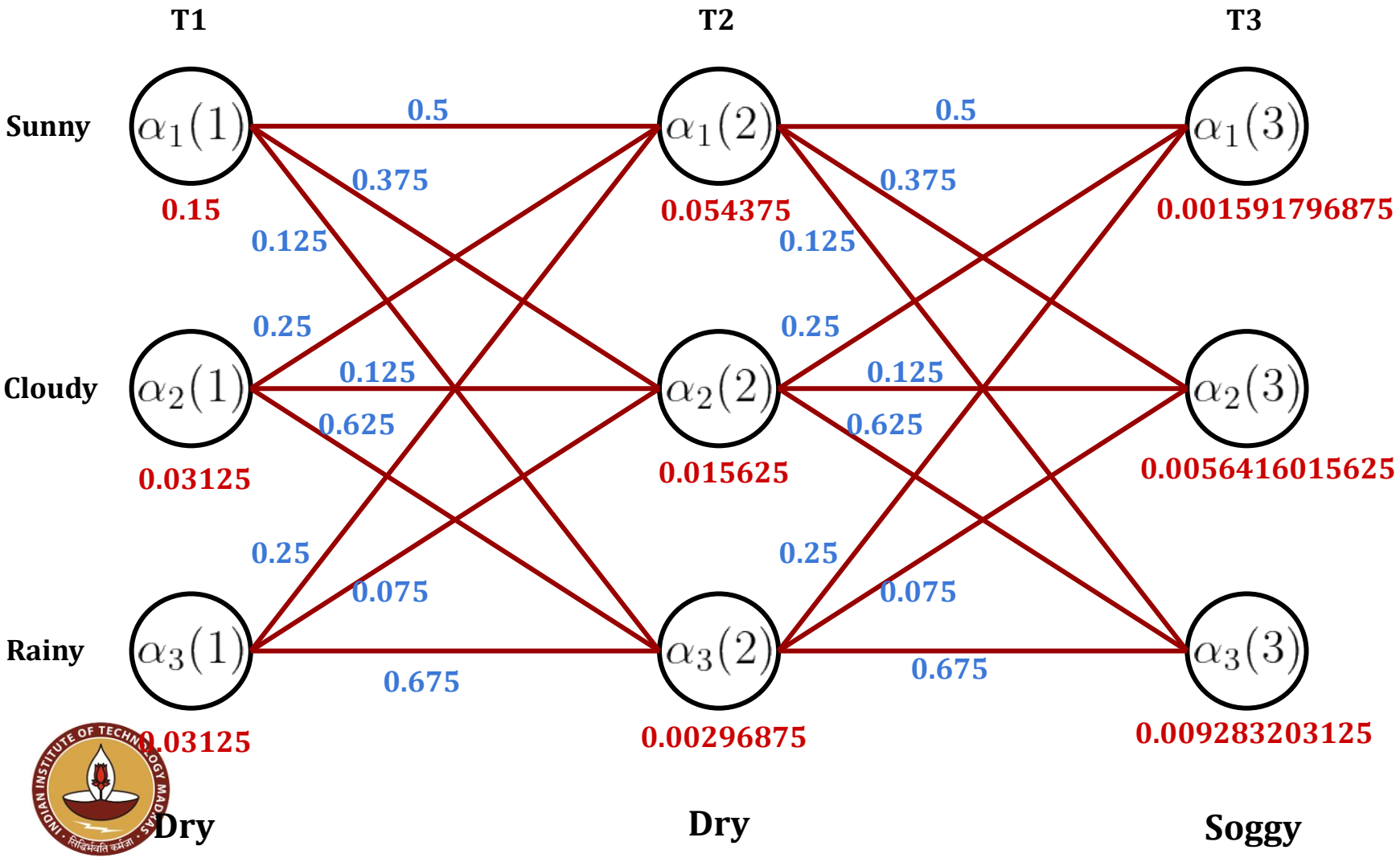
Forward Algorithm For HMM - Weather Example



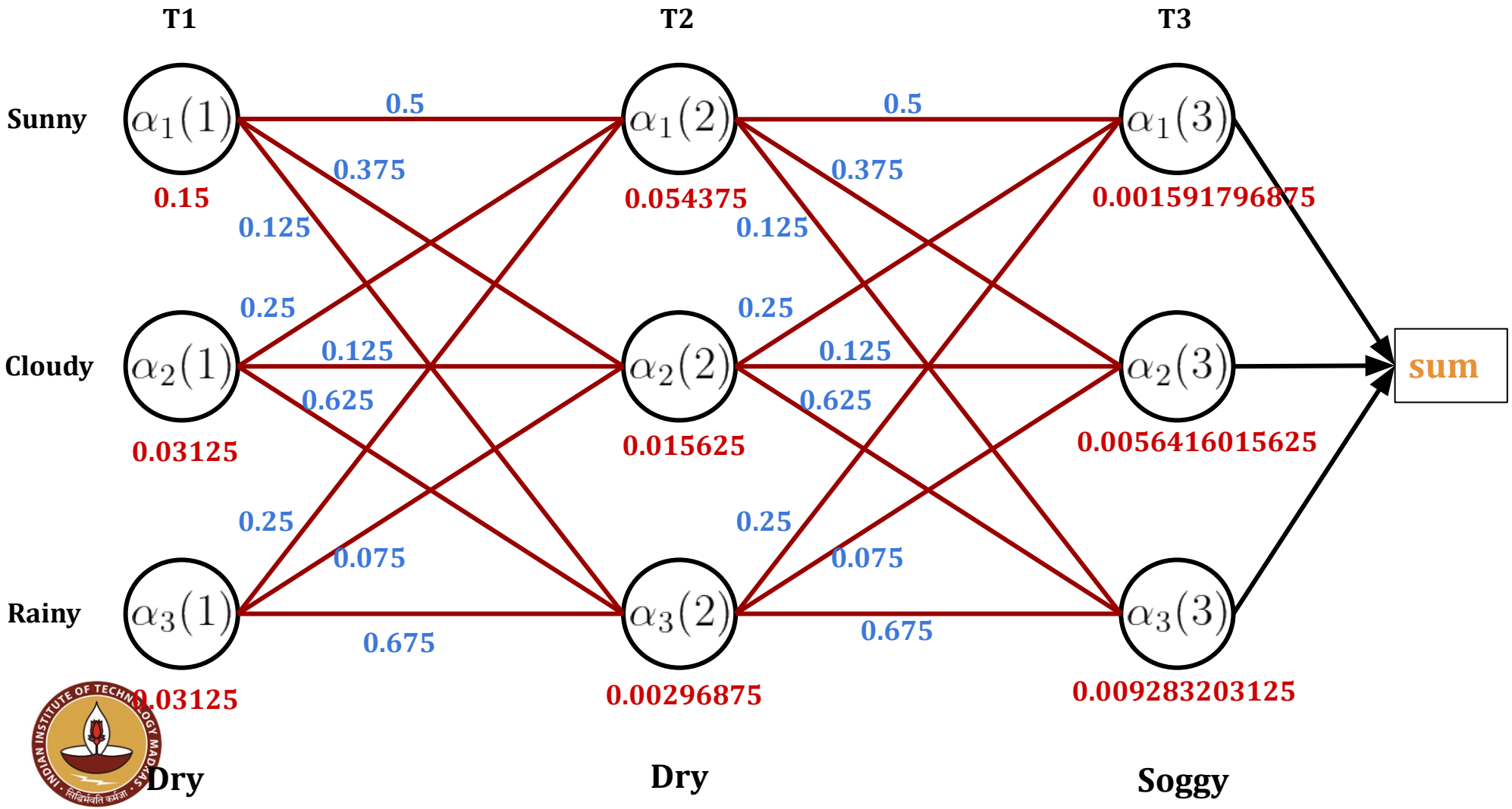
Forward Algorithm For HMM - Weather Example



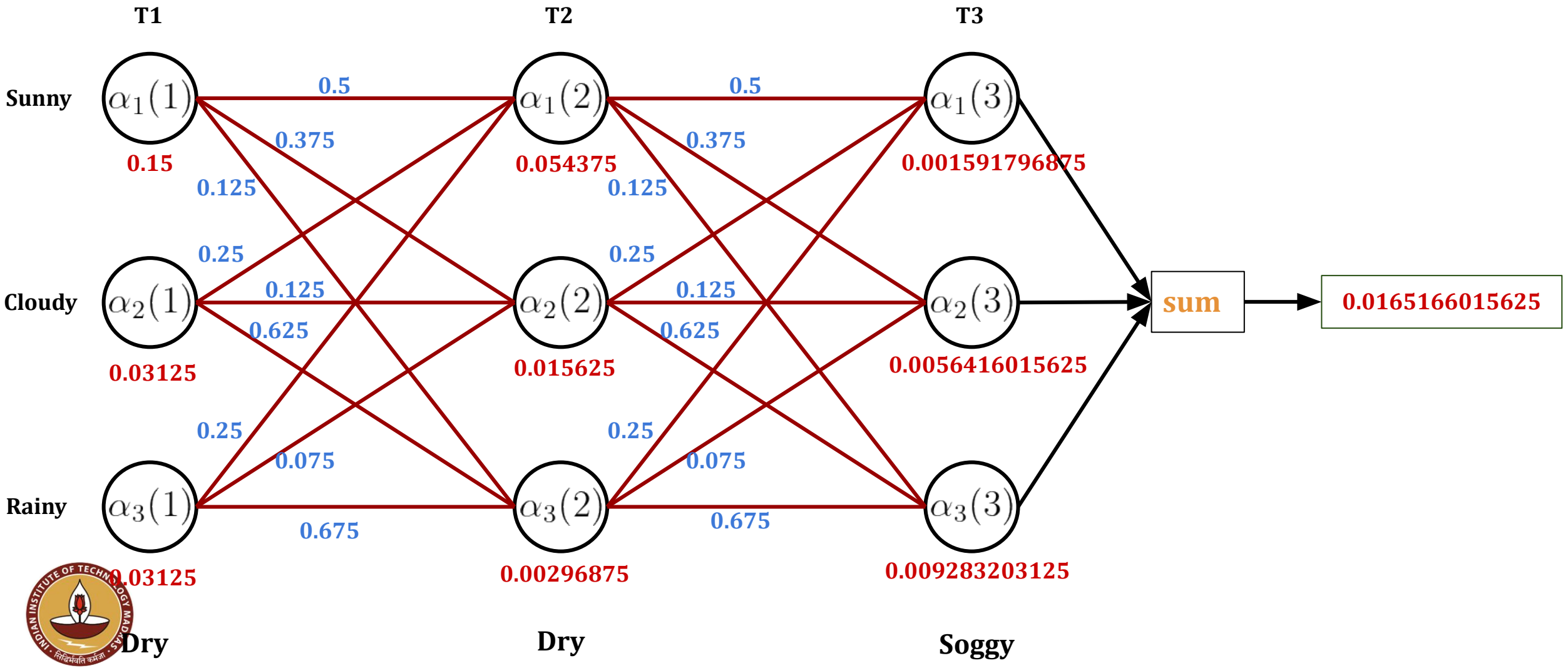
Forward Algorithm For HMM - Weather Example



Forward Algorithm For HMM - Weather Example



Forward Algorithm For HMM - Weather Example

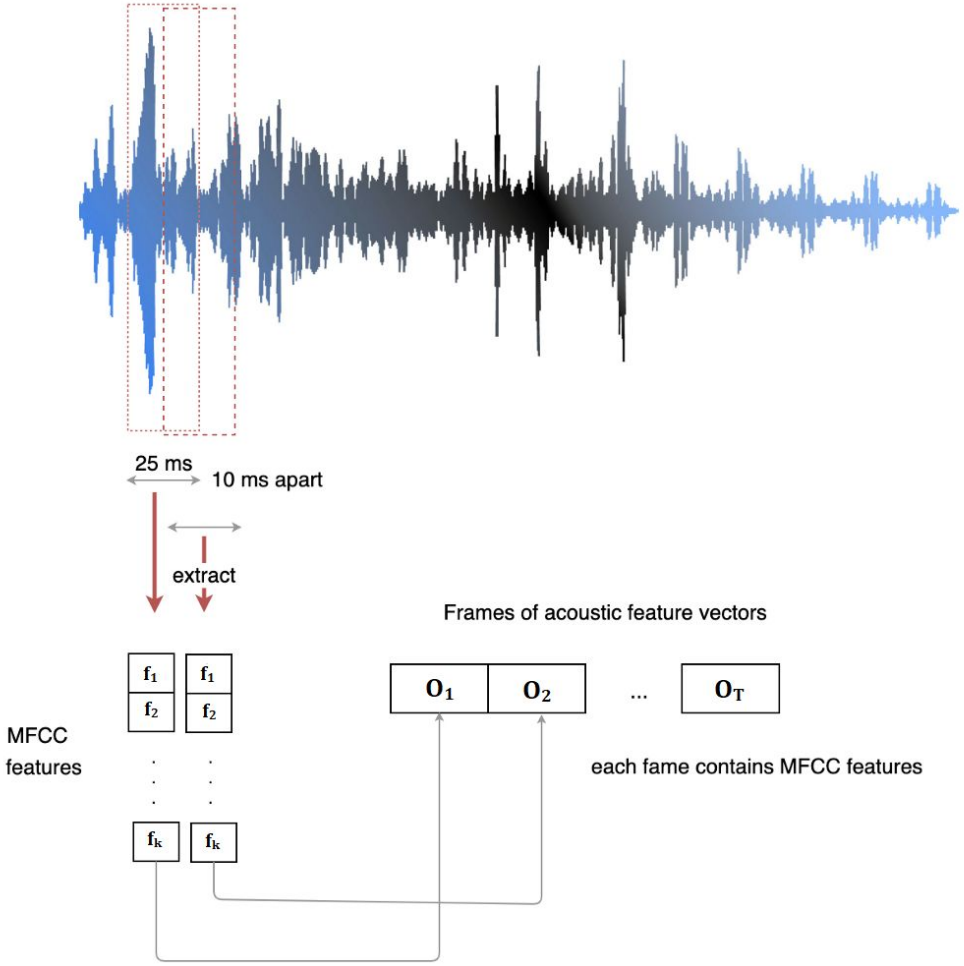


Speech Recognition

- We learned how sequential data can be modelled with HMM



Speech Recognition



Speech Recognition

Let us suppose that there are only two phonemes P_1 & P_2.

So I know all the observation vectors O_1 , 0_2 , …, 0_T belongs to either P_1 or P_2

Task: We have to classify each of the observation vectors O_1, 0_2, …,0_T to either P_1 or P_2

It can be thought of as a 2 class problem.

Given O_1 , 0_2 , … , 0_T classify P_1 , P_2



Recall : Given height measurement classify male or female

Given height y , we have two likelihood function $P_M(y/\Theta_M)$ vs $P_F(y/\Theta_F)$

We need labelled data for male & female class

$x_1^M, x_2^M, x_3^M \ldots x_{1000}^M$

$x_1^F, x_2^F, x_3^F \ldots x_{1000}^F$

Build $P_M(y/\Theta_M)$

Build $P_F(y/\Theta_F)$

Supervised Learning



Speech Recognition

Here, we have Observation vectors $O_1, 0_2, \ldots, 0_T$, we have two likelihood function $P_{P1}(O_1, 0_2, \ldots, 0_T/\Theta_{P1})$ vs $P_{P2}(O_1, 0_2, \ldots, 0_T/\Theta_{P2})$

We need supervised learning data for this.

We will ask say 100 volunteers to speak the phoneme P_1 and get the Observation vectors $O_1, 0_2, \ldots, 0_T$. We will then ask say 100 volunteers to speak the phoneme P_2 and get the Observation vectors $O_1, 0_2, \ldots, 0_T$.

With the help of these data we will get the parameters Θ_{P1} & Θ_{P2}.

We can extend this method to n phoneme classification. Whichever likelihood is larger it belongs to that class.



Speech Recognition

But here are two **problems**.

- Like in case of cursive writing letters are different when written standalone vs when used in word, here also the observation vectors will be different when a phoneme is spoken standalone vs when spoken in a sentence.
- Also there is no fixed boundary between different phoneme just like there is no fixed boundary between letters in cursive writing.



Need for HMM?

In case of modelling with Gaussian Mixture model we were able to predict the most probable phoneme.

But as text is a sequential observation. It will help if we capture the sequential information.

We need to do modelling of a sequence of observations hence the need arises for **Hidden Markov Model** with which we can model sequence of events.



Recall Weather prediction with dryness of grass

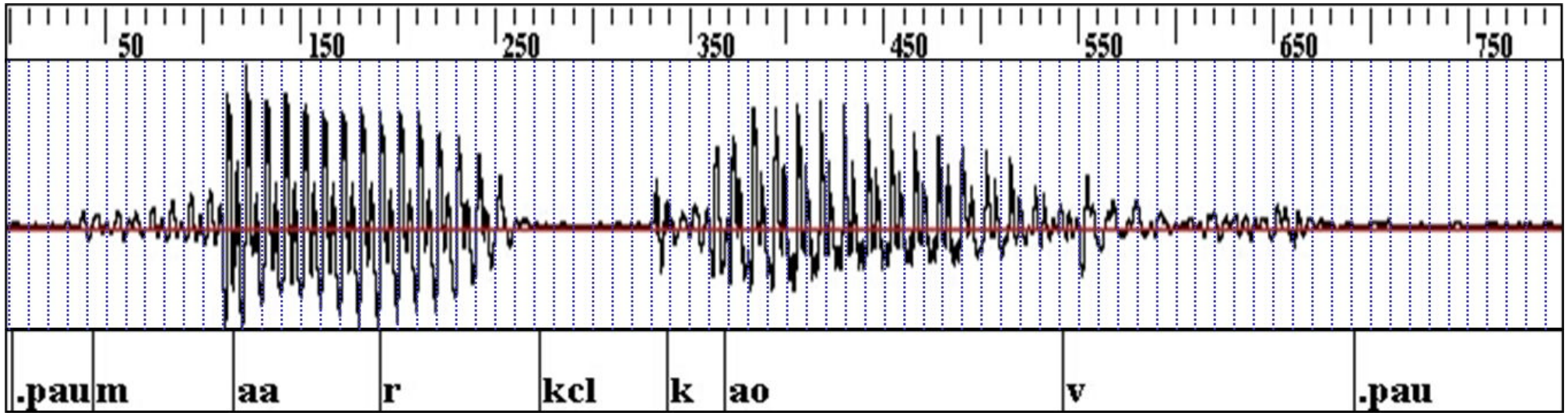
In case of weather prediction the observation was **dryness of grass** & we needed to **predict the weather**. There the weather condition was not directly visible to us.

Here also there is a similar pattern. Here we have observation vectors as **MFCC features** & we need to **predict the corresponding text**. We can use the concept learn in weather prediction using **Hidden Markov Model** here and predict the corresponding text.



HMM For Speech Recognition

- Speech is the output of an HMM; problem is to find most likely model for a given speech observation sequence.
- Speech is divided into sequence of 10-msec frames, one frame per state transition (faster processing). Assume speech can be recognized using 10-msec chunks.
- Each vertical line corresponds to one observation



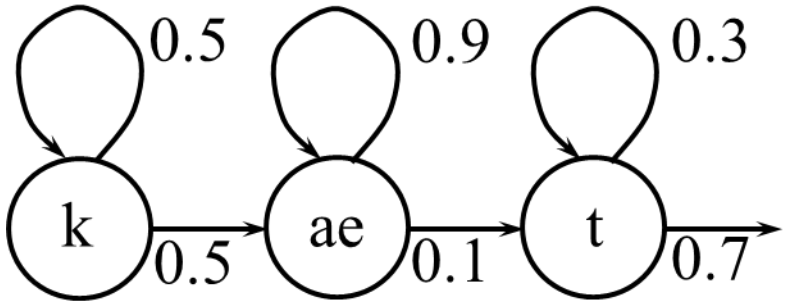
HMM For Speech Recognition

- Each state in a HMM can be associated with
  - sub-phoneme
  - phoneme
  - sub-word
- Usually, sub-phonemes or sub-words are used, to account for spectral dynamics (coarticulation).
- One HMM corresponds to one phoneme or word
- For each HMM, determine the probability of the best state sequence that results in the observed speech.
- Choose HMM with best match (probability) to observed speech.
- Given most likely HMM and state sequence, maybe determine the corresponding phoneme and word sequence.

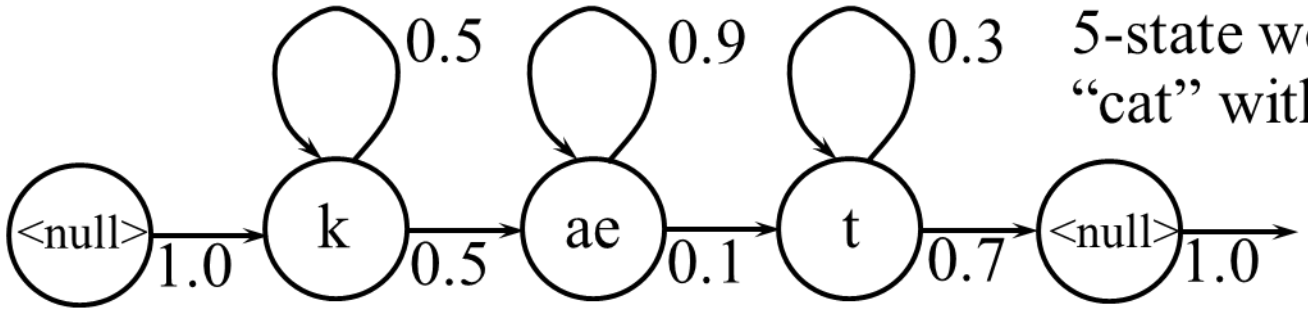


HMM For Speech Recognition

Example of states for word model:



3-state word model for “cat”



5-state word model for “cat” with null states

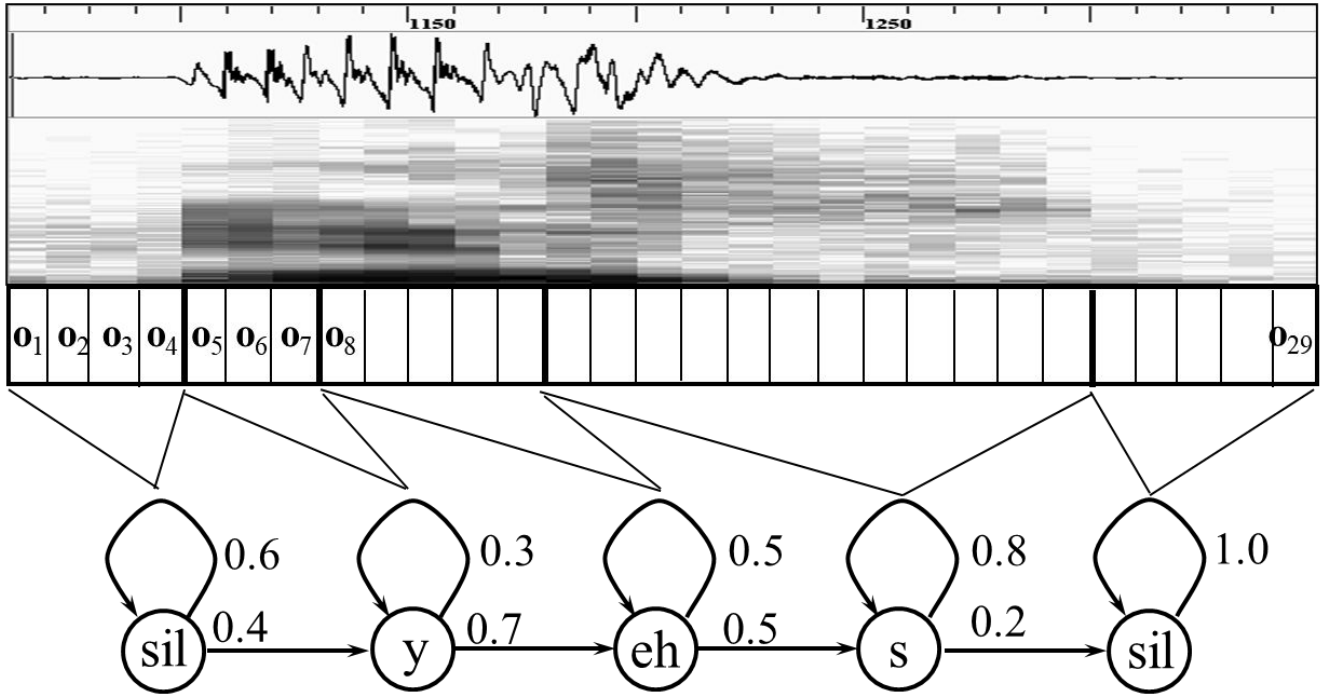
HMM For Speech Recognition

- Null states do not emit observations, and are entered and exited at the same time *t*. Theoretically, they are unnecessary. Practically, they can make implementation easier.
- States *don't have to* correspond directly to phonemes, but are commonly labeled using phonemes.



HMM For Speech Recognition

- Example of using HMM for word “yes” on an utterance:

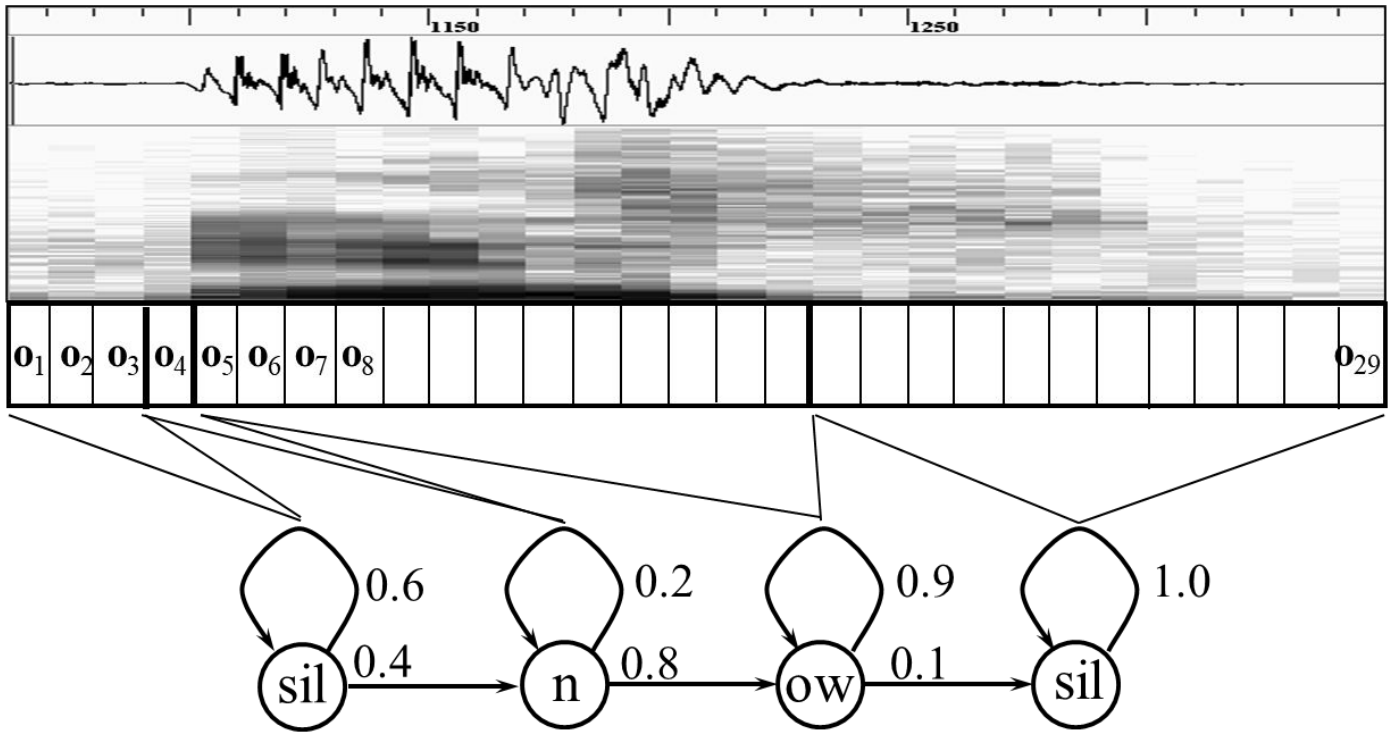


$$b_{\text{sil}}(\mathbf{o}_1)\cdot 0.6\cdot b_{\text{sil}}(\mathbf{o}_2)\cdot 0.6\cdot b_{\text{sil}}(\mathbf{o}_3)\cdot 0.6\cdot b_{\text{sil}}(\mathbf{o}_4)\cdot 0.4\cdot b_{\text{y}}(\mathbf{o}_5)\cdot 0.3\cdot b_{\text{y}}(\mathbf{o}_6)\cdot 0.3\cdot b_{\text{y}}(\mathbf{o}_7)\cdot 0.7\ \ldots$$

observation

HMM For Speech Recognition

- Example of using HMM for word “no” on same utterance:

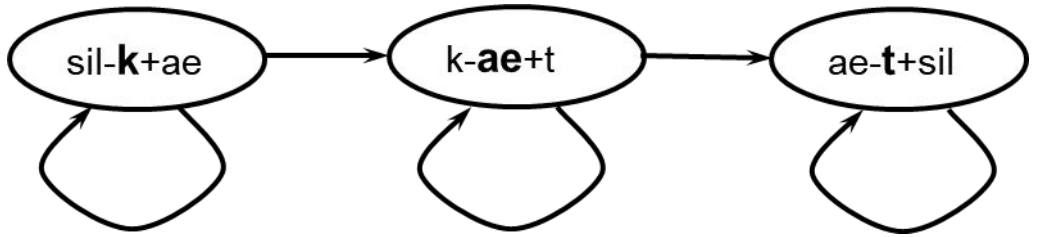


$$b_{\text{sil}}(\mathbf{o}_1)\cdot 0.6\cdot b_{\text{sil}}(\mathbf{o}_2)\cdot 0.6\cdot b_{\text{sil}}(\mathbf{o}_3)\cdot 0.4\cdot b_{\text{n}}(\mathbf{o}_4)\cdot 0.8\cdot b_{\text{ow}}(\mathbf{o}_5)\cdot 0.9\cdot b_{\text{ow}}(\mathbf{o}_6)\cdot 0.9\ \ldots$$

HMM For Speech Recognition

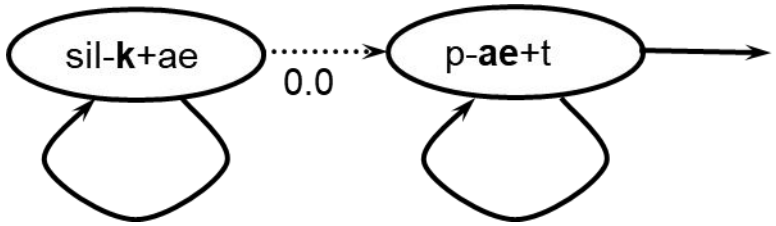
- Because of coarticulation, states are sometimes made dependent on preceding and/or following phonemes (context dependent).
  - **ae** (monophone model)
  - k-**ae**+t (triphone model)
  - k-**ae** (diphone model)
  - **ae**+t (diphone model)

- Constructing words requires matching the contexts:
- For example, triphone model for “cat”



HMM For Speech Recognition

- This permits several different models for each phoneme, depending on surrounding phonemes (context sensitive)
  - k-**ae**+t
  - p-**ae**+t
  - k-**ae**+p
- Probability of "illegal" state sequence is zero (never used)

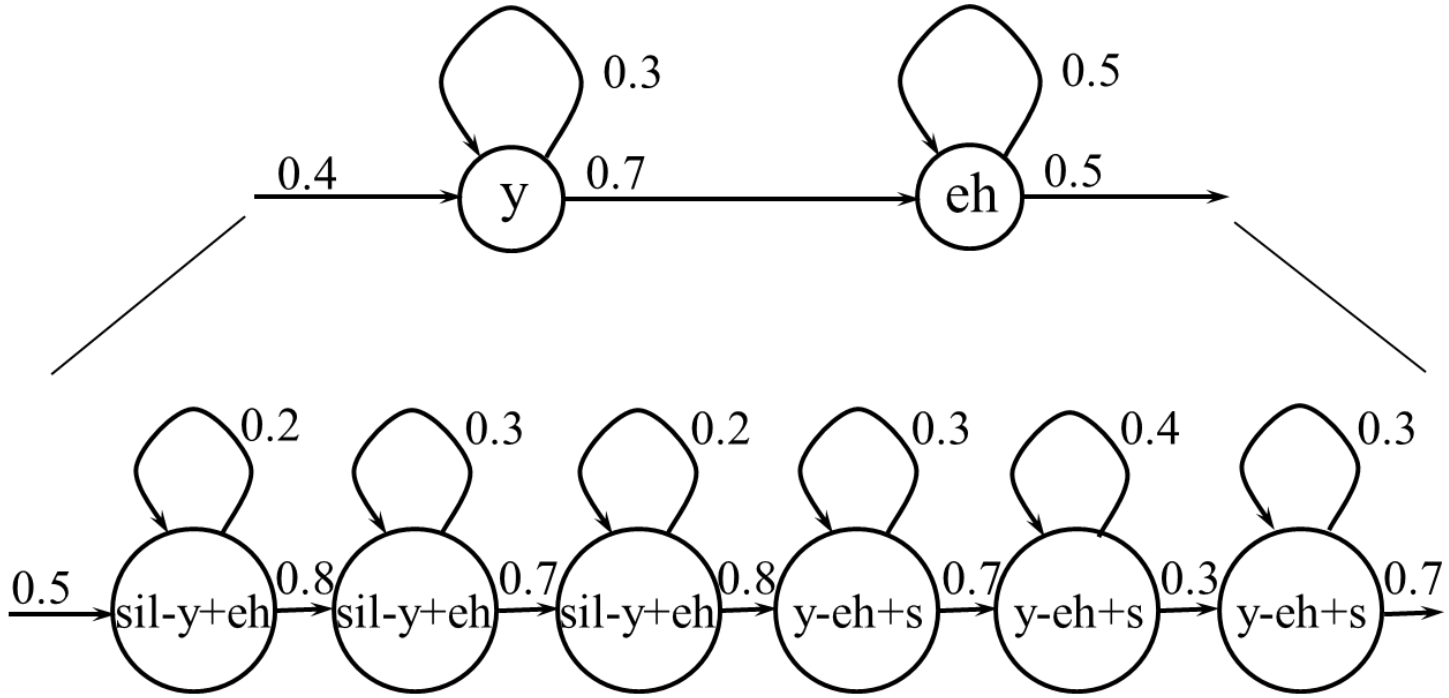


- Much larger number of states to train on… (50 vs. 125,000 for a full set of phonemes)



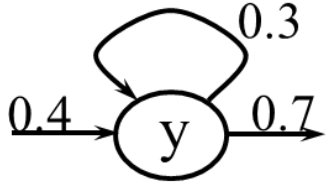
HMM For Speech Recognition

- Example of 3-state, triphone HMM (expand from previous):

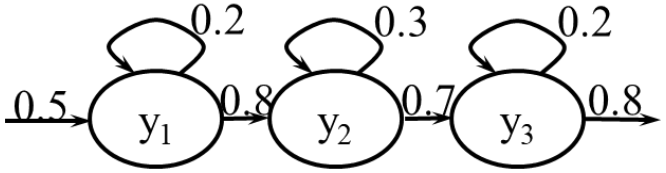


HMM For Speech Recognition

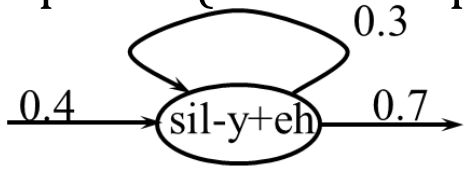
- 1-state monophone (context independent)



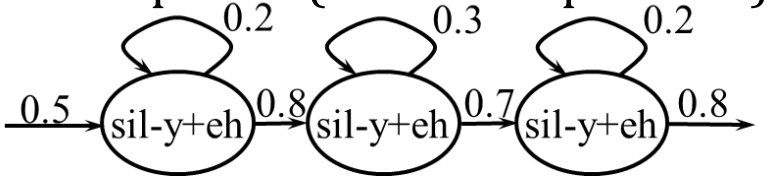
- 3-state monophone (context independent)



- 1-state triphone (context dependent)

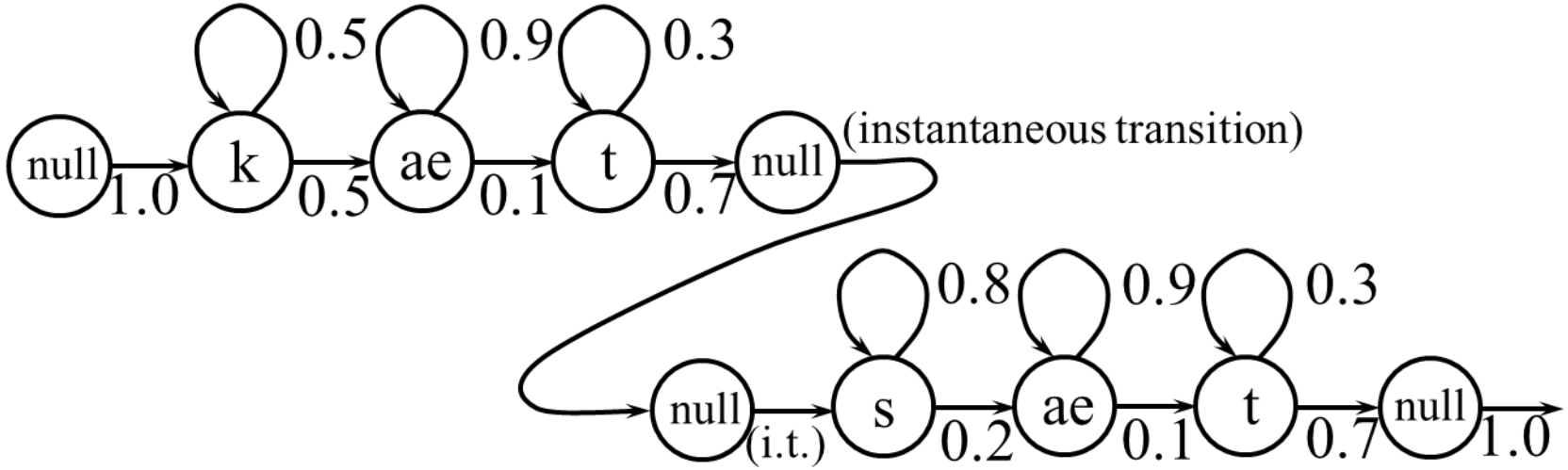


- 3-state triphone (context dependent)



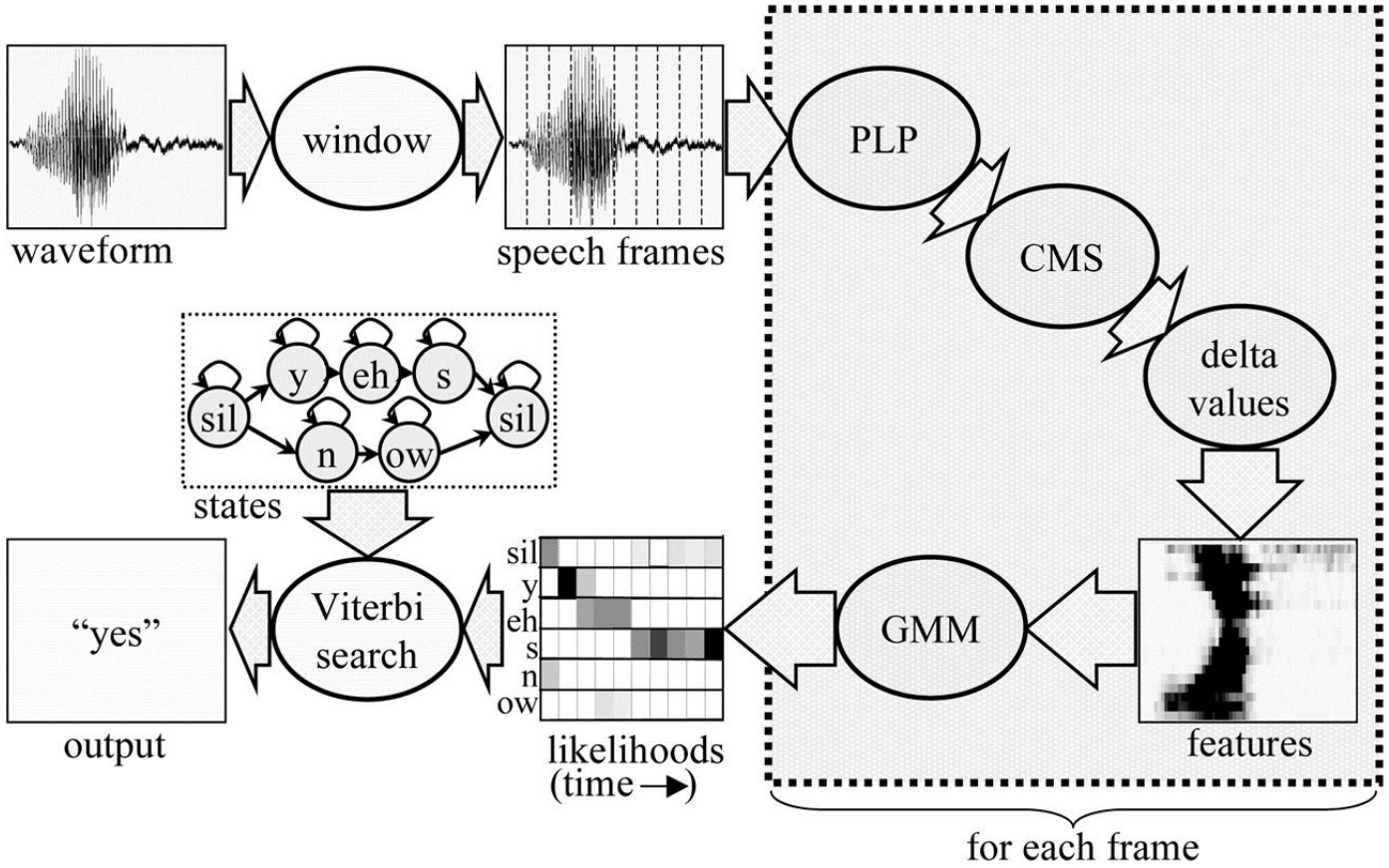
HMM For Speech Recognition

- Typically, one HMM = one word or phoneme
- Join HMMs to form sequence of phonemes = word-level HMM
- Join words to form sentences = sentence-level HMM
- Use <null> states at ends of HMM to simplify implementation



HMM For Speech Recognition

- Big Picture:



(from Encyclopedia of Information Systems, 2002)

HMM For Speech Recognition

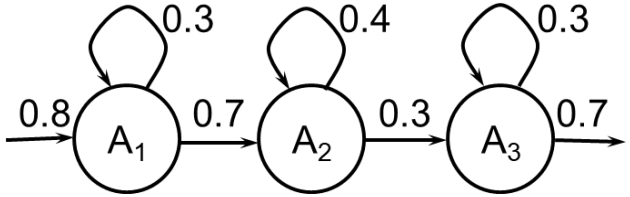
- Prior segmentation of speech into phonetic regions is not required before performing recognition. This provides robustness over other methods that first segment and then classify, because any attempt to do prior segmentation will yield errors.
- As we move through an HMM to determine most likely sequence, we get segmentation.
- First-order and independence assumptions correct for some phenomena, but not for speech. But math is easier.



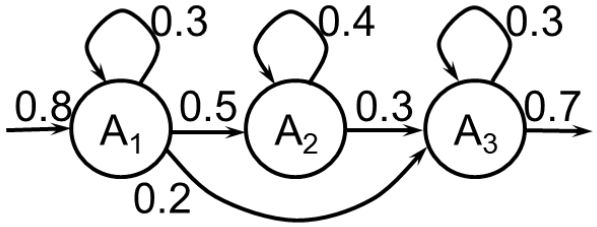
HMM For Speech Recognition

Different Topologies are Possible:

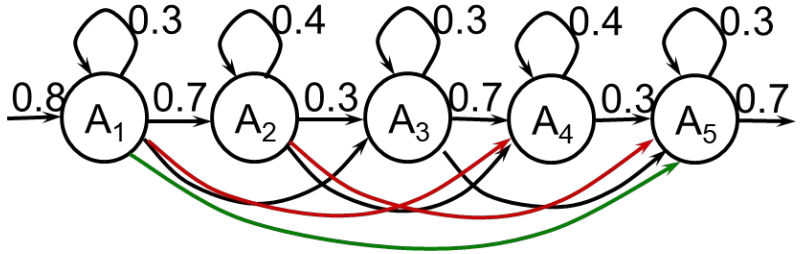
- Standard



- Short phoneme



- Left-to-right



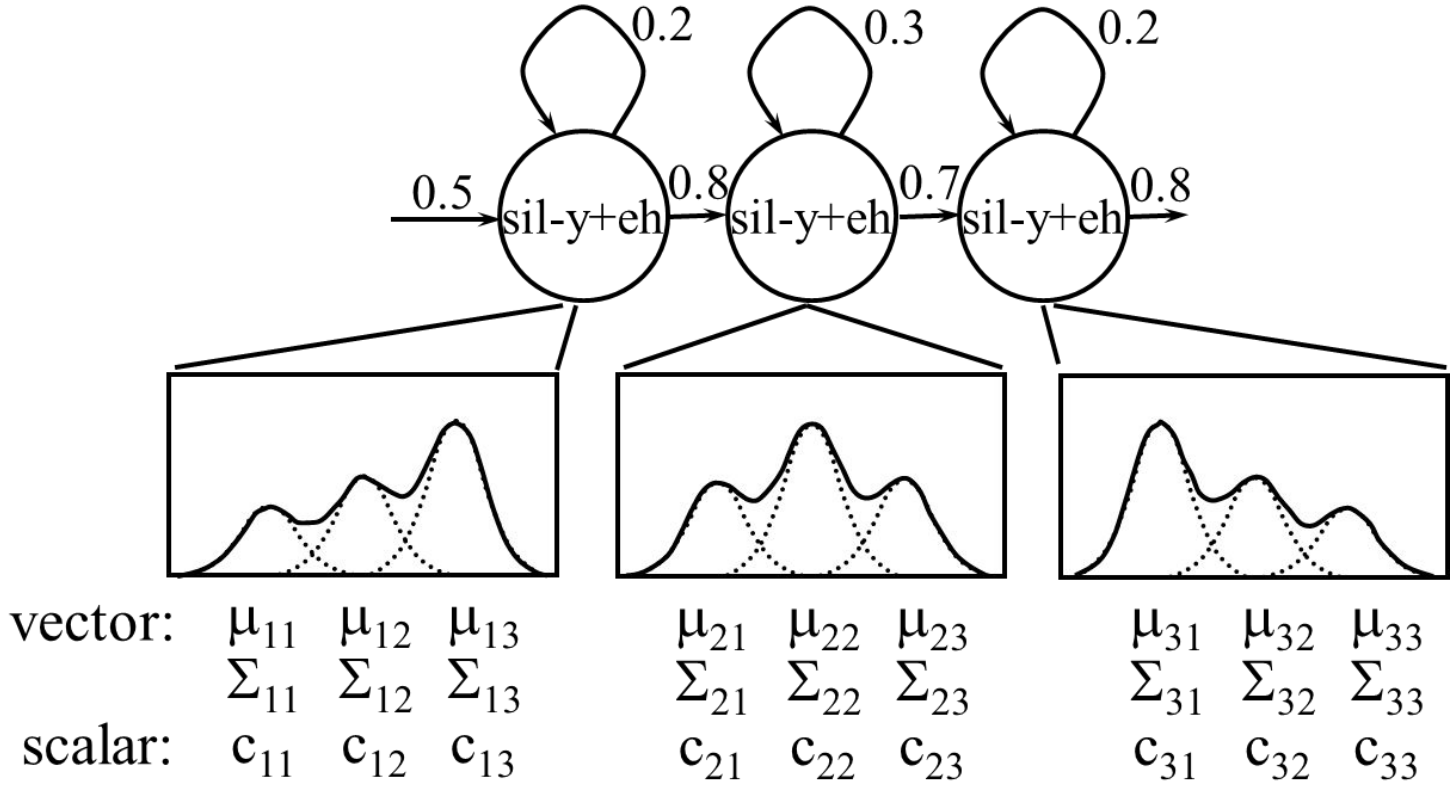
HMM For Speech Recognition

- first-order HMM
- one HMM per phoneme or word
- 3 states per phoneme-level HMM, more for word-level HMM
- sequential series of states, each with self-loop
- link HMMs together to form words and sentences
- GMM: many Gaussian components per state
- context-dependent HMMs: HMMs can be linked together only if their contexts correspond



HMM For Speech Recognition

Diagram of one HMM: /y/ in context of preceding silence, followed by /eh/



HMM For Speech Recognition

- N = number of states; 3 per phoneme, >3 per word
- S = states $\{S_1, S_2, S_3, \ldots, S_N\}$; even though any state can output (any) observation, associate most likely output with state name. Often use context-dependent phonetic states (triphones): {sil-y+eh y-eh+s eh-s+sil ...}
- T = final time of output; t = {1, 2, ... T}
- O = observations $\{o_1 o_2 \ \ldots o_T\}$; actual output generated by HMM; features (cepstral, LPC, MFCC, PLP, etc) of a speech signal



HMM For Speech Recognition

- M = number of observation symbols per state = number of codewords for discrete HMM = "infinite" for continuous HMM
- $\boldsymbol{v}$ = symbols $\{v_1\ v_2\ \ldots\ v_M\}$
  - "codebook indices" generated by discrete (VQ) HMM; for speech, indices point to locations in feature space.
  - No direct correspondence for continuous HMM; output of continuous HMM is sequence of observations {speech vector 1, speech vector 2, …} output can be any point in continuous n-dimensional space. output can be *any* point in continuous *n*-dimensional space.
- **A** = matrix of transition probabilities $\{a_{ij}\}$
  - $a_{ij} = P(q_t{=}j \mid q_{t\text{-}1}{=}i)$
  - *ergodic* HMM: all $a_{ij} > 0$



HMM For Speech Recognition

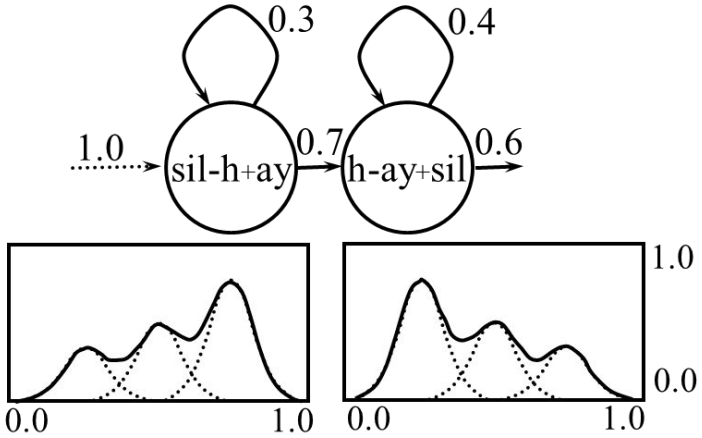
- **B** = set of parameters for determining probabilities $b_j(\mathbf{o}_t)$
  - $b_j(\mathbf{o}_t) = P(\mathbf{o}_t = \mathbf{v}_k \mid q_t = j)$ (discrete: codebook)
  - $b_j(\mathbf{o}_t) = P(\mathbf{o}_t \mid q_t = j)$ (continuous: GMM)

- $\boldsymbol{\pi}$ = initial state distribution $\{\pi_i\}$
  - $\pi_i = P(q_1 = i)$

- λ = entire model
  - $\lambda = (\mathbf{A}, \mathbf{B}, \boldsymbol{\pi})$



HMM For Speech Recognition



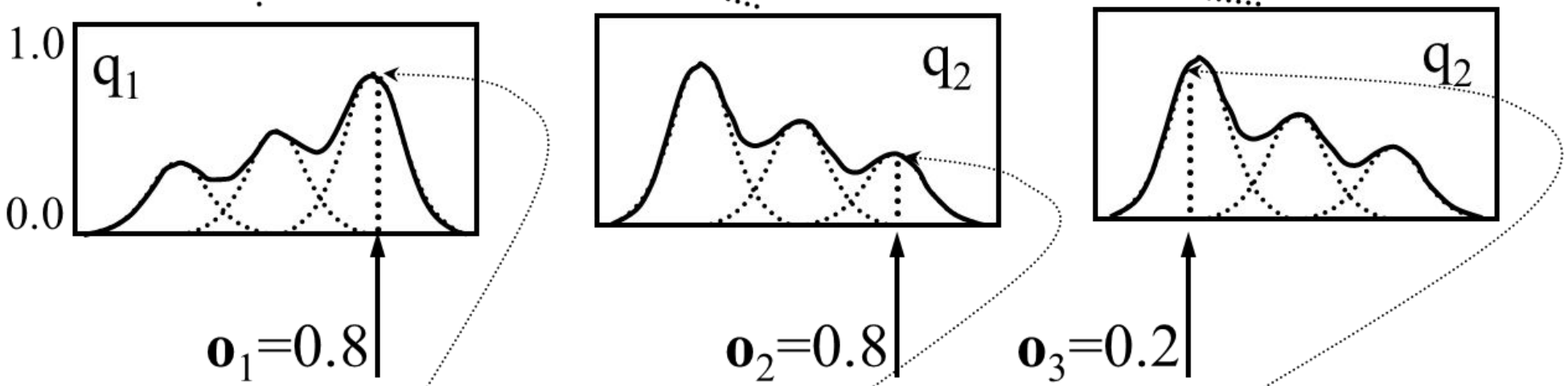
Example: “hi”

- observed features: $\mathbf{o}_1 = \{0.8\}$, $\mathbf{o}_2 = \{0.8\}$, $\mathbf{o}_3 = \{0.2\}$
- what is probability of **O *given*** the state sequence:

  {sil-h+ay h-ay+sil h-ay+sil}
  {1 2 2}

HMM For Speech Recognition

$$P = \pi_1 \, b_1(\mathbf{o}_1) \, a_{12} \, b_2(\mathbf{o}_2) \, a_{22} \, b_2(\mathbf{o}_3)$$



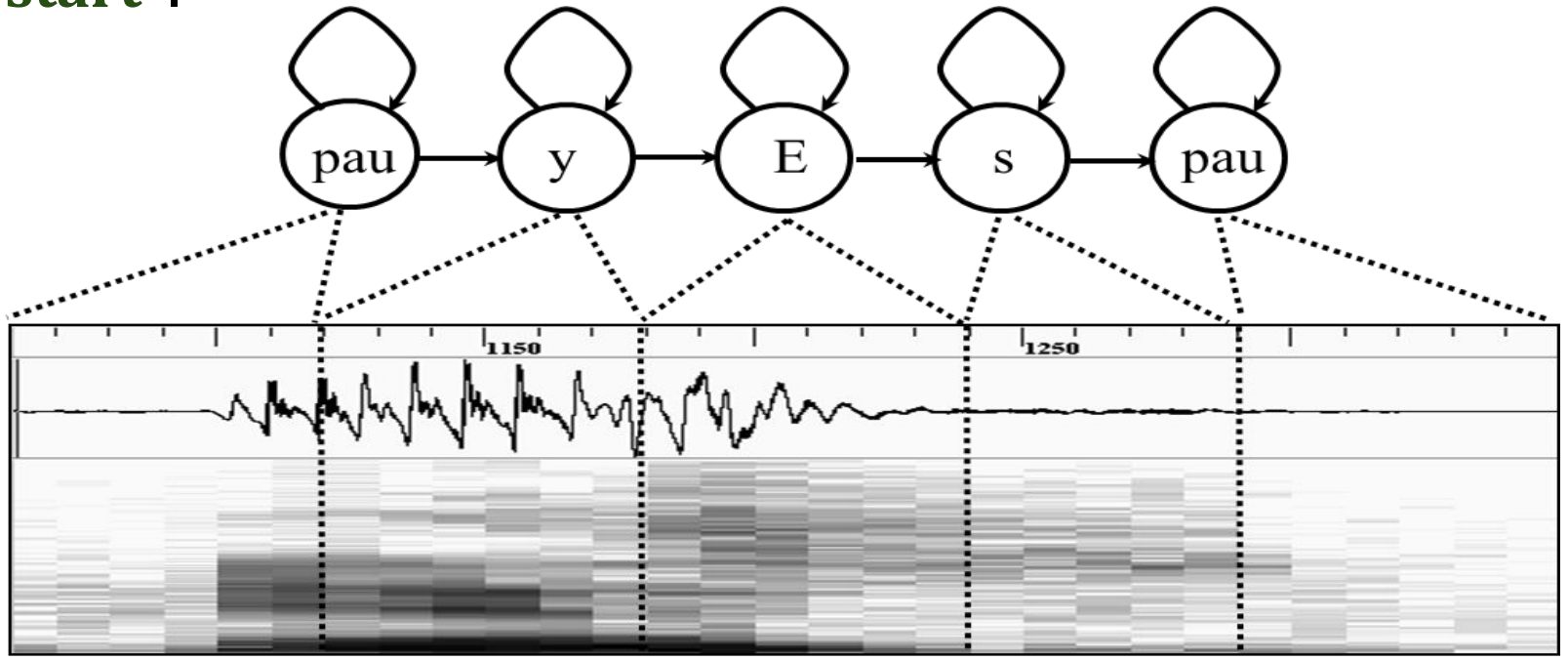
$$P = 1.0 \cdot 0.76 \cdot 0.7 \cdot 0.27 \cdot 0.4 \cdot 0.82$$

$$P = 0.0471$$

How To Start Training an HMM??

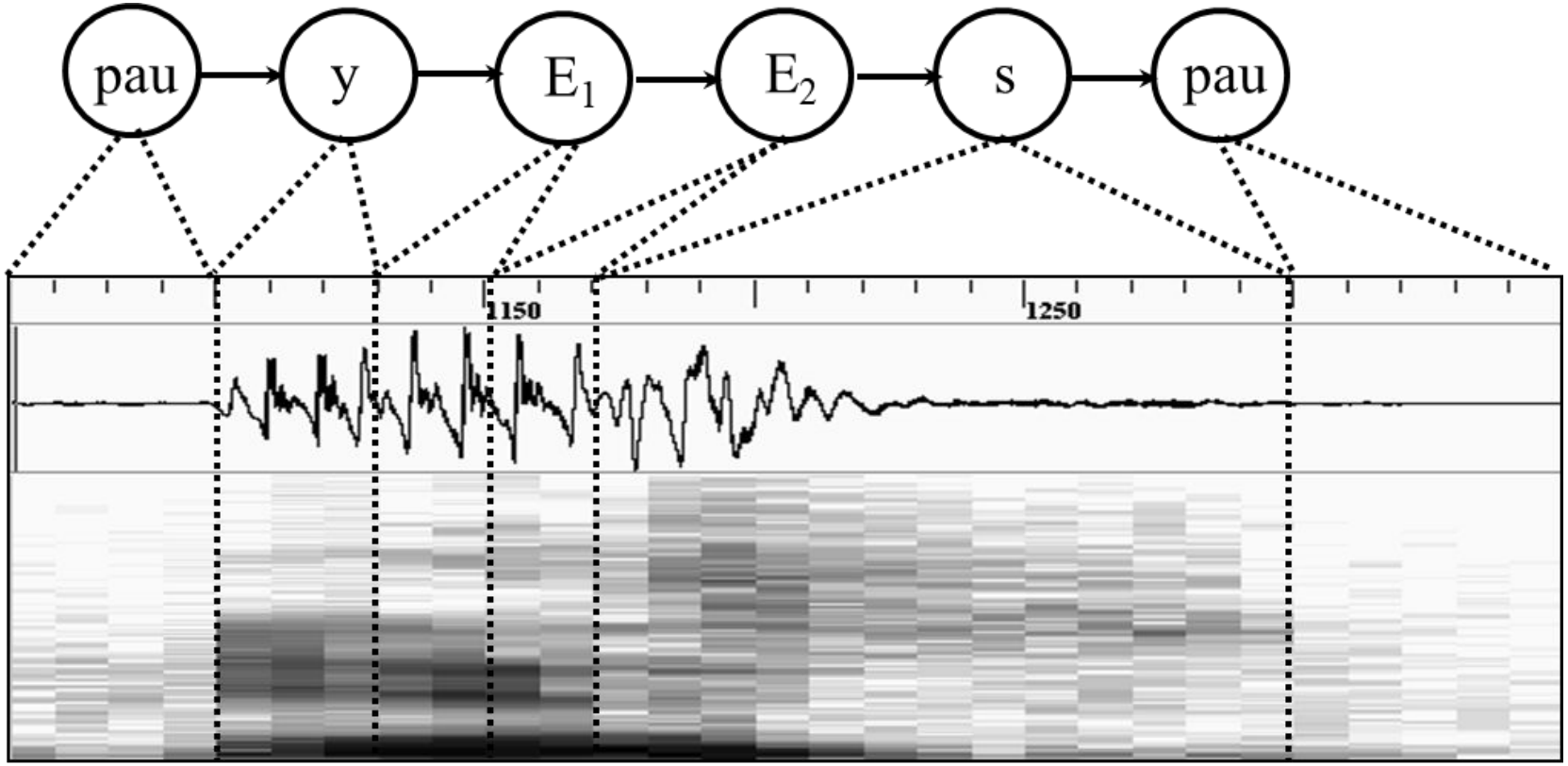
To compute initial values we can give **equally likely values of π_i .**

To compute initial $b_j(\mathbf{o}_t)$ we requires **segmentation of training data.** One approach is to divide it into **equal-length segments**, compute $b_j(\mathbf{o}_t)$ for each segment. This approach is called **"flat start"**.



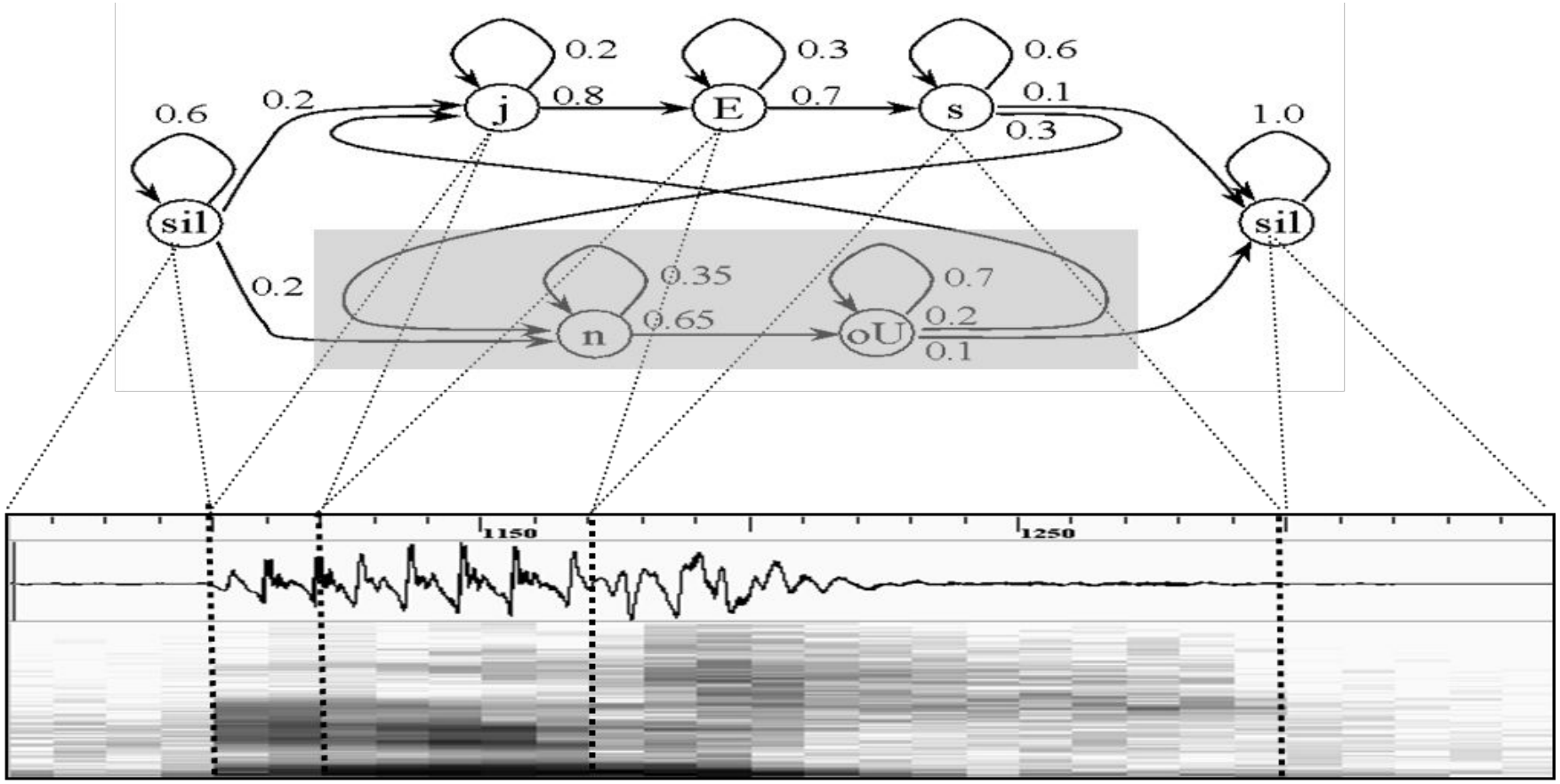
How To Start Training an HMM??

Another approach is to use **manually-aligned data**, if available. Split each phoneme into *X* equal parts to create *X* states per phoneme.



How To Start Training an HMM??

Third and most practical solution is use "f**orce-aligned data.**" We know phoneme sequence, so use Viterbi on *existing* HMM to determine best alignment.



How To Start Training an HMM??

How to improve initial a_{ij}, $b_j(\mathbf{o}_t)$ estimates?

- Viterbi Segmentation (k-means segmentation)

V1. Given training data, create initial model.

V2. Use Viterbi to determine best state sequence through data.

V3. For segment (sequence of observations) associated with one state:

- for each observation (frame),

  assign $\mathbf{o}(t)$ to most likely mixture component by evaluating each component of $b_j(\mathbf{o}_t)$
- update the parameters.

V4. If new model very different from current model, set current model to "new" model and then go to (2).



Step 1 = initialization

1. Assign each state to a sequence of observations.
2. VQ to create clusters; cluster weight = ratio of points in cluster to total points in state.
3. Estimate bj(•) by computing means, covariances.

Step 2 = Viterbi search

4. Perform Viterbi search to get best state alignment.

Step 3 of k-means

5. Assign each observation to the mixture component that yields the greatest probability of that observation.
6. Update means, covariances, mixture weights, transition probabilities (aij measured from data).

Step 4 of k-means

7. Repeat from (3) until converge; convergence to a locally "best" model is guaranteed.



HMM For Speech Recognition

- What is probability of an observation sequence ***and*** state sequence, given the model?

$$P(\mathbf{O}, \mathbf{q} \mid \lambda) = P(\mathbf{O} \mid \mathbf{q}, \lambda)\, P(\mathbf{q} \mid \lambda)$$

- **What is the "best" valid observation sequence from time 1 to time T, given the model?**
- At every time t, can connect to up to N states

$\Rightarrow$ There are up to N^T possible state sequences

We can use viterbi algorithm to find the best path

